

ROBOTIKA
INTEGRASI SENSOR KINECT, NAVIGASI DAN KONTROL



Oleh :

PROGRAM STUDI TEKNIK MEKATRONIKA
JURUSAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS TRUNOJOYO MADURA
BANGKALAN
2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	vi
DAFTAR TABEL	vii
ABSTRAK.....	1
BAB I: PENGENALAN SENSOR KINECT	2
1.1. Latar Belakang dan Sejarah Kinect	2
1.2. Arsitektur Perangkat Keras Kinect	3
1.3. Prinsip Kerja Sensor Kedalaman (<i>Depth</i>)	5
1.4. Spesifikasi Teknis dan Batasan Operasional	6
1.5. Transformasi Kinect dari Gaming ke Aplikasi Robotika	7
BAB II: SENSOR SEJENIS DENGAN MERK LAIN	8
2.1. Latar Belakang Kebutuhan Sensor Alternatif.....	8
2.2. Asus Xtion Pro Live	8
2.3. Intel RealSense (Seri D400 dan L500)	9
2.4. Stereolabs ZED (Kamera Stereo Pasif)	10
2.5. Orbbec (Astra dan Femto)	11
2.6. Tabel Perbandingan Karakteristik Sensor	12
BAB III: DRIVER YANG DIGUNAKAN DAN ALTERNATIFNYA.....	13
3.1. Latar Belakang Komunikasi Perangkat Keras dan Perangkat Lunak	13
3.2. Microsoft Kinect SDK (Kinect for Windows SDK)	14
3.3. OpenNI (Open Natural Interaction).....	15
3.4. Libfreenect (Proyek OpenKinect)	15
3.5. Integrasi Driver pada ROS (Robot Operating System)	16
3.6. Alternatif Driver Modern (Intel RealSense SDK dan Orbbec SDK)	17
3.7. Tantangan Bandwidth USB dan Isu Komputasi Edge.....	18
BAB IV: DATA YANG DIHASILKAN: RGB, DEPTH, SKELETON	20

4.1.	Arsitektur Multistream Sensor Kinect	20
4.2.	Data Citra RGB (Red, Green, Blue)	20
4.3.	Data Citra Kedalaman (Depth Image)	21
4.4.	Data Kerangka Tubuh (Skeleton Tracking)	22
4.5.	Data Inframerah (IR Stream)	23
4.6.	Sinkronisasi dan Registrasi Antar-Modalitas	24
BAB V: PEMROSESAN CITRA RGB DAN DEPTH MENGGUNAKAN OPENCV		25
5.1.	Mengakses Data RGB Image Stream	25
5.2.	Inisialisasi dan Konfigurasi Perangkat Melalui OpenNI	27
5.3.	Pembacaan Buffer dan Konversi Ruang Warna	29
5.4.	Visualisasi Real-Time dan Manajemen Sumber Daya	31
BAB VI: MENGAkses DEPTH IMAGE STREAM		34
6.1.	Arsitektur dan Representasi Matematis Citra Kedalaman	34
6.2.	Inisialisasi, Konfigurasi, dan Registrasi Spasial	35
6.3.	Pembacaan Buffer dan Normalisasi Visualisasi	36
6.4.	Pemrosesan Lanjutan: Filtering Noise pada Depth Map	39
BAB VII: MENGAkses SKELETON IMAGE		42
7.1.	Arsitektur dan Representasi Matematis Data Skeleton	42
7.2.	Inisialisasi Pelacakan Kerangka melalui NITE Middleware	43
7.3.	Pemetaan Koordinat 3D ke Bidang Citra 2D (Perspektif Proyeksi)	44
7.4.	Visualisasi dan Penggambaran Hierarki Pohon Kinematika	46
BAB VIII: MENGAkses INFRARED IMAGE		50
8.1.	Arsitektur Fisis dan Representasi Matematis Citra Inframerah	50
8.2.	Inisialisasi IR Stream dan Konstrain Bandwidth Hardware	51
8.3.	Pembacaan Buffer dan Peningkatan Kontras melalui Histogram Equalization	53
BAB IX: MENGESTIMASI JARAK DENGAN KINECT		56

9.1.	Prinsip Pengukuran Jarak Berdasarkan Agregasi Statistik Depth Data	56
9.2.	Implementasi Deteksi Objek Berbasis Warna untuk ROI Extraction	57
9.3.	Ekstraksi Jarak Objek dari ROI pada Depth Map	60
9.4.	Implementasi Estimasi Jarak untuk Kontrol Proporsional (P-Controller)..	61
BAB X: MENGESTIMASI ARAH DENGAN KINECT		65
10.1.	Prinsip Estimasi Sudut Arah (Bearing) pada Bidang Citra	65
10.2.	Ekstraksi Posisi Piksel Centroid Objek	66
BAB XI: MENGGUNAKAN POSE ESTIMATION PADA RGB IMAGE.....		70
11.1.	Konsep Pose Estimation 2D (Keypoints Detection).....	70
11.2.	Algoritma Pelacakan Pose (MediaPipe)	70
11.3.	Ekstraksi Koordinat Pixel Sendi Tubuh	71
11.4.	Perhitungan Sudut Sendi (Angle Estimation).....	73
BAB XII: MEMBANGUN POINT CLOUD DENGAN DEPTH IMAGE.....		77
12.1.	Pengantar Point Cloud 3D	77
12.2.	Persamaan Transformasi Koordinat (u,v,z) ke (X,Y,Z)	77
12.3.	Implementasi Pembangkitan Point Cloud di OpenCV/PCL.....	78
12.4.	Visualisasi 3D Point Cloud	78
BAB XIII: KONTROL GERAK SEDERHANA DENGAN KINECT DENGAN SIMULASI ROBOT		82
13.1.	Arsitektur Teleoperation Berbasis PyBullet	82
13.2.	Setup Lingkungan Simulasi PyBullet.....	82
13.3.	Pemetaan Gestur Skeleton ke Kecepatan Motor (Kinematic Mapping).....	83
13.4.	Implementasi Skrip Python untuk Kendali.....	84
BAB XIV: KONTROL GERAK SEDERHANA UNTUK ROBOT BERODA DI DUNIA NYATA.....		89
14.1.	Arsitektur Robot Beroda (TurtleBot URDF) di PyBullet.....	89

14.2.	Kinematika dan Kalibrasi Odometri Motor DC	89
14.3.	Implementasi Obstacle Avoidance Berbasis Depth.....	90
DAFTAR PUSTAKA		92

DAFTAR GAMBAR

Gambar 1.1 Kinect.....	3
Gambar 1.2 Cara Kerja Sensor Depth	5
Gambar 1.3. Output Sensor Kinect.....	6
Gambar 2.1 Xtion Pro Live.....	9
Gambar 2.2 Intel RealSense	10
Gambar 2.3. Stereolabs ZED.....	11
Gambar 2.4. Orbbec.....	12
Gambar 3.1. Software Kinect SDK.....	13
Gambar 3.2. Microsoft Kinect SDK.....	14
Gambar 3.3. OpenNI	15
Gambar 3.4. Libfreenect.....	16
Gambar 3.5. ROS Kinect.....	17
Gambar 3.6. RealSense SDK.....	18
Gambar 4.1. Kinect Output RGB.....	21
Gambar 4.2. Depth Output Kinect.....	22
Gambar 4.3. Skeleton Tracking Output Kinect	23
Gambar 4.4. Infrared Output Kinect.....	24
Gambar 5.1. Output Program Bab 5.....	33
Gambar 6.1. Output Program Bab 6.....	41
Gambar 7.1. Output Bab 7.....	49
Gambar 8.1. Output Bab 8.....	55
Gambar 9.1. Output Bab 9.....	64
Gambar 10.1. Output Bab 10.....	69
Gambar 11.1. Output Bab 11	76
Gambar 12.1. Output Bab 12	81
Gambar 13.1. Output Bab 13	88
Gambar 14.1. Obstacle Avoidance mobile robot menggunakan Pybullets.....	91

DAFTAR TABEL

Table 2.1. Perbandingan Karakteristik	12
---	----

ABSTRAK

Kemampuan robot untuk berinteraksi dengan lingkungan secara otonom dan adaptif. Salah satu kunci utama dalam mencapai otonomi tersebut adalah keberadaan sistem persepsi visual yang andal. Sensor Kinect, yang awalnya dikembangkan untuk industri permainan video (gaming), telah berevolusi menjadi salah satu perangkat penginderaan tiga dimensi (3D) paling populer di kalangan akademisi dan praktisi robotika. Dengan mengintegrasikan kamera RGB, sensor kedalaman (Depth), proyektor Inframerah (IR), dan kemampuan pelacakan kerangka tubuh (Skeleton) dalam satu perangkat yang terjangkau, Kinect membuka jalan bagi robot untuk "melihat" dan memahami dunia di sekitarnya secara spasial. Buku ajar ini disusun secara komprehensif dan sistematis untuk membimbing mahasiswa, peneliti, maupun praktisi dalam menguasai integrasi sensor Kinect pada sistem robotika.

Materi pembelajaran dirancang secara progresif, dimulai dari pengenalan dasar perangkat keras dan *driver* yang diperlukan, hingga teknik ekstraksi berbagai aliran data (*stream*) seperti RGB, Depth, IR, dan Skeleton. Pembaca juga akan diajak untuk mempraktikkan langsung pemrosesan data citra melalui pendekatan pemrograman tingkat tinggi menggunakan pustaka populer seperti OpenCV dan OpenNI. Buku ini membahas implementasi praktis dari data yang diperoleh, seperti mengestimasi jarak objek, menentukan arah navigasi bebas hambatan, menerapkan *Pose Estimation* menggunakan kecerdasan buatan, serta membangun struktur data tiga dimensi berupa *Point Cloud*. Sebagai puncak dari pembelajaran, buku ini menyajikan dua studi kasus aplikatif: pertama, kontrol gerak robot berbasis gerakan tubuh di lingkungan simulasi (seperti ROS/Gazebo); dan kedua, implementasi kendali navigasi pada robot beroda di dunia nyata. Melalui perpaduan teori, cuplikan kode pemrograman, dan studi kasus yang relevan, buku ini diharapkan dapat menjadi panduan esensial bagi siapa pun yang ingin mengeksplorasi batas-batas interaksi antara manusia, komputer, dan robot.

Kata Kunci: Robotika, Sensor Kinect, Pengolahan Citra 3D, Depth Image, Skeleton Tracking, Point Cloud, Navigasi Otonom.

BAB I

PENGENALAN SENSOR KINECT

1.1. Latar Belakang dan Sejarah Kinect

Era modern robotika menuntut sebuah sistem otonom yang tidak hanya mampu bergerak, tetapi juga mampu "memahami" lingkungan sekitarnya secara spasial. Selama bertahun-tahun, kamera dua dimensi (2D) konvensional menjadi standar dalam sistem visi komputer. Namun, kamera 2D memiliki keterbatasan fundamental: ia hanya mampu menangkap proyeksi dunia tiga dimensi (3D) ke dalam bidang datar, sehingga informasi mengenai jarak dan struktur spasial objek hilang secara permanen. Untuk mengatasi hal ini, para peneliti robotika dulunya harus mengandalkan sensor laser mahal seperti *Light Detection and Ranging* (LiDAR) atau kamera stereo yang membutuhkan kalibrasi yang rumit. Kondisi ini berubah drastis pada tahun 2010 ketika perusahaan Microsoft merilis perangkat bernama Kinect.

Sensor Kinect awalnya tidak dirancang untuk keperluan riset robotika maupun industri. Perangkat ini dikembangkan oleh Microsoft sebagai pengendali gerak (*motion controller*) untuk konsol permainan video (*gaming console*) Xbox 360. Ambisi utama Kinect adalah menghilangkan kebutuhan akan *controller* fisik (seperti *joystick*) dan memungkinkan pemain berinteraksi dengan permainan menggunakan gerakan tubuh mereka secara langsung. Nama "Kinect" sendiri merupakan portmanteau dari kata "kinetic" (gerak) dan "connect" (koneksi), yang merepresentasikan visi perangkat ini dalam menghubungkan gerakan manusia dengan dunia digital.

Rilisnya Kinect ke pasar konsumen pada November 2010 menciptakan gelombang besar di komunitas ilmuwan komputer dan robotika. Hal yang membuat perangkat ini mendapat perhatian luar biasa bukanlah fungsinya sebagai alat *gaming*, melainkan perpaduan antara kemampuan sensor 3D yang canggih dengan harga yang sangat terjangkau (sekitar \$150 saat rilis). Sebagai perbandingan, sensor jarak berkualitas serupa yang digunakan di laboratorium riset pada saat itu dapat berharga ribuan dolar. Dengan adanya Kinect, demokratisasi teknologi visi 3D pun terjadi secara massal.

Hanya dalam hitungan minggu setelah rilis, komunitas *open-source* di seluruh dunia berhasil membajak (*hack*) protokol USB Kinect dan membuat *driver* tidak resmi yang memungkinkan perangkat tersebut terhubung ke komputer pribadi (PC). Melihat

antusiasme ini, Microsoft akhirnya merilis *Software Development Kit* (SDK) resmi untuk Windows. Sejak saat itu, Kinect berevolusi dari sekadar aksesoris game menjadi salah satu sensor robotika paling berpengaruh di era modernnya, digunakan secara luas dalam navigasi robot, pemetaan ruangan, pengenalan gestur, hingga analisis biomekanika. Berikut merupakan tampilan Kinect yang ditujukan pada Gambar 1.1



Gambar 1.1 Kinect

1.2. Arsitektur Perangkat Keras Kinect

Untuk memahami bagaimana Kinect dapat memberikan data 3D, kita harus membedah arsitektur perangkat kerasnya. Secara fisik, baris paling depan perangkat Kinect (sering disebut sebagai "mata" Kinect) terdiri dari tiga komponen optik utama yang bekerja secara sinkron. Komponen-komponen ini meliputi kamera RGB standar, proyektor Inframerah (IR), dan kamera Inframerah (IR).

Kamera RGB pada Kinect berfungsi sama seperti kamera digital pada umumnya. Kamera ini menangkap citra berwarna dari lingkungan sekitar dengan resolusi standar *VGA* (640x480 piksel) pada 30 *frame per second* (fps) untuk versi pertama (Kinect V1), dan mampu menangkap resolusi *Full HD* (1920x1080) pada versi kedua (Kinect V2). Citra RGB ini memberikan informasi tekstur dan warna objek, yang sangat penting untuk pengenalan objek 2D (seperti mendeteksi rambu lalu lintas atau wajah manusia).

Komponen kedua adalah Proyektor Inframerah (IR Emitter). Proyektor ini tidak memancarkan cahaya tampak, melainkan pola titik-titik (*speckle pattern*) inframerah yang terstruktur ke arah lingkungan. Pola titik-titik cahaya ini biasanya tidak terlihat oleh mata manusia, namun berfungsi sebagai "penanda" di permukaan setiap objek. Ketika pola ini diproyeksikan ke objek yang berada di dekat sensor, bentuk titik-titik tersebut akan berubah sesuai dengan kontur permukaan objek.

Komponen ketiga adalah Kamera Inframerah (IR Depth Sensor). Tugas dari kamera monokrom khusus ini adalah menangkap pantulan pola bintang inframerah yang dipancarkan oleh proyektor IR sebelumnya. Dengan menganalisis distorsi atau pergeseran pola titik-titik tersebut, prosesor internal Kinect (atau perangkat lunak di PC) dapat menghitung kedalaman (jarak) setiap titik terhadap sensor. Selain ketiga komponen visual tersebut, Kinect juga dilengkapi dengan *array microphone* multi-mikrofon untuk menangkap suara dan mendeteksi arah sumber suara, serta sebuah motor *tilt* kecil di bagian bawah yang memungkinkan kepala sensor dimiringkan secara otomatis untuk menyesuaikan sudut pandang. Berikut ini merupakan spesifikasi pada Table 1.1

Tabel 1.1. Perangkat Keras Kinect

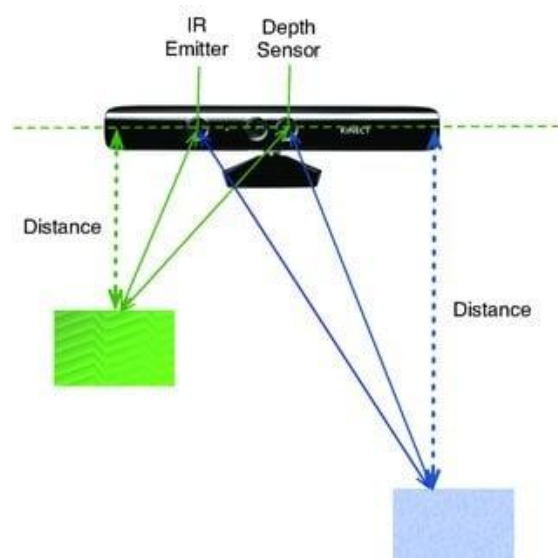
Komponen	Spesifikasi
Nama Perangkat	Microsoft Kinect V1 (Xbox 360)
Tahun Rilis	2010
Sensor Warna (RGB)	CMOS Camera
Resolusi RGB	640 × 480 piksel @ 30 fps (hingga 1280 × 960 untuk foto)
Sensor Kedalaman (Depth)	Infrared Structured Light
Resolusi Depth	320 × 240 piksel @ 30 fps
Sensor Infrared (IR)	IR Camera + IR Projector
Field of View (Horizontal)	57°
Field of View (Vertikal)	43°
Sudut Kemiringan (Tilt Motor)	±27° (motorized)
Jarak Operasi	0,8–4 meter (optimal 1,2–3,5 meter)
Frame Rate	30 FPS
Microphone Array	4 mikrofon internal
Format Audio	16 kHz, 24-bit PCM
Accelerometer	3-axis accelerometer
Antarmuka	USB 2.0 (dengan adaptor daya untuk PC)
Tegangan Catu Daya	12 V DC (melalui adaptor eksternal saat digunakan pada PC)

Dimensi	279 × 64 × 64 mm
Berat	±1,3 kg
Teknologi Depth	Structured Light (PrimeSense)
Pelacakan Rangka (Skeleton Tracking)	Hingga 20 joint untuk maksimal 2 pengguna aktif

1.3. Prinsip Kerja Sensor Kedalaman (*Depth*)

Kemampuan Kinect untuk "melihat" dalam tiga dimensi bersandar pada teknologi *depth sensing*. Pada Kinect V1, teknologi yang digunakan dikenal sebagai *Structured Light 3D Scanning*, yang dikembangkan oleh Perusahaan bernama PrimeSense (yang kemudian diakuisisi oleh Apple). Prinsip kerjanya adalah triangulasi optis. Proyektor IR memancarkan pola titik yang sudah diketahui pola aslinya. Kamera IR ditempatkan pada jarak tertentu (disebut *baseline*) dari proyektor IR.

Ketika cahaya IR mengenai sebuah objek datar (seperti tembok), kamera IR akan melihat pola titik yang lurus. Namun, jika objek tersebut memiliki kedalaman atau menonjol (seperti manusia yang berdiri di depan tembok), titik-titik cahaya IR yang dipantulkan akan mengalami pergeseran optik (*disparity*) jika dilihat dari perspektif kamera IR. Prosesor kemudian menggunakan trigonometri sederhana untuk menghitung jarak: semakin besar pergeseran titik di kamera IR, semakin dekat objek tersebut ke sensor. Hasil dari perhitungan ini disebut sebagai *Depth Map* (Peta Kedalaman). Berikut ini merupakan tampilan sensor depth yang ditunjukkan pada Gambar 1.2.



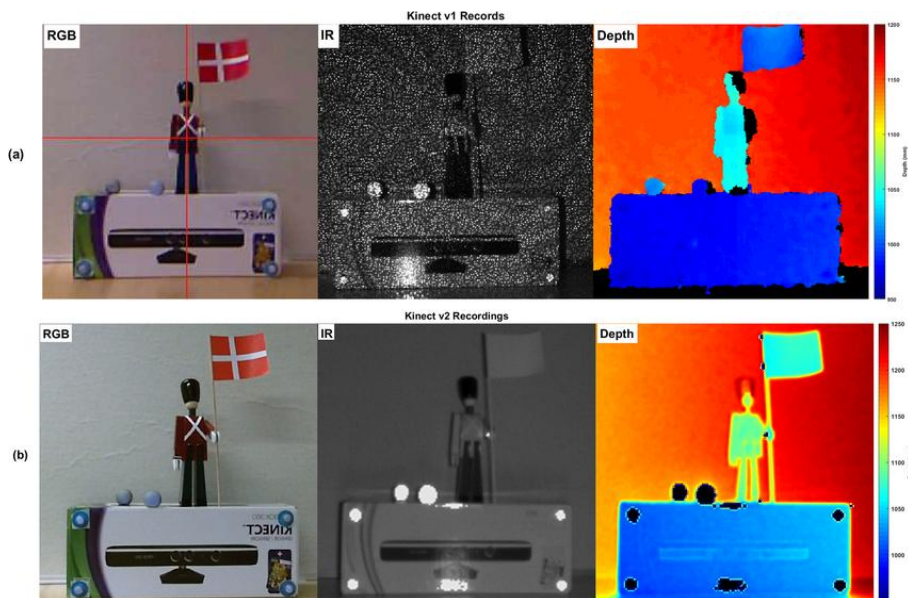
Gambar 1.2 Cara Kerja Sensor Depth

1.4. Spesifikasi Teknis dan Batasan Operasional

Meskipun revolusioner, pengguna Kinect dalam robotika harus memahami batasan-batasan operasional perangkat ini. Pertama, terdapat batasan jarak deteksi yang dikenal sebagai "*sweet spot*". Untuk Kinect V1, sensor kedalaman optimal beroperasi pada jarak antara 0,8 meter (800 mm) hingga 4,0 meter (4000 mm) dari sensor. Jika objek berada lebih dekat dari 0,8 meter, kamera IR tidak dapat memfokuskan pola *structured light*, sehingga data kedalaman menjadi *noise* atau "buta". Jika objek berada lebih jauh dari 4 meter, intensitas pantulan IR terlalu lemah untuk diukur.

Kedua, sensor Kinect sangat bergantung pada properti reflektansi material objek. Bahan yang menyerap cahaya inframerah (seperti kain hitam tebal atau bahan beludru) akan terlihat sebagai "lubang" atau celah pada *depth map* karena pantulan tidak kembali ke sensor. Sebaliknya, bahan yang sangat memantulkan cahaya (seperti cermin atau kaca bening) akan menyesatkan sensor. Cermin akan memantulkan pola IR ke arah lain, sehingga sensor salah membaca jarak, dan kaca bening akan meneruskan pola IR ke objek di belakang kaca, membuat sensor mendeteksi objek di balik kaca seolah-olah berada di permukaan kaca tersebut.

Ketiga, interferensi cahaya ambient. Karena kamera Kinect V1 bekerja di spektrum inframerah, penggunaan Kinect di ruangan yang terkena sinar matahari langsung akan menimbulkan masalah serius. Berikut merupakan tampilan hasil output sensor Kinect yang ditunjukkan pada Gambar 1.3.



Gambar 1.3. Output Sensor Kinect

1.5. Transformasi Kinect dari Gaming ke Aplikasi Robotika

Integrasi sensor Kinect dalam bidang robotika telah mendefinisikan ulang berbagai disiplin ilmu, khususnya di bidang *Human-Robot Interaction* (HRI) dan navigasi otonom. Dalam konteks HRI, kemampuan Kinect untuk melakukan *skeleton tracking* menjadi fitur paling dikagumi. Melalui algoritma *machine learning* yang tertanam di dalam perangkat lunaknya (seperti OpenNI atau Kinect SDK), Kinect mampu mengidentifikasi manusia secara real-time dan mengekstrak posisi 3D dari 15 hingga 25 titik sendi (*joints*) tubuh, seperti kepala, bahu, siku, pergelangan tangan, dan lutut.

Fitur *skeleton tracking* ini memungkinkan robot untuk mengenali gestur manusia. Alih-alih harus menggunakan *remote control*, manusia dapat memerintahkan robot beroda untuk maju, mundur, atau belok hanya dengan menggerakkan tangan. Bahkan, robot dapat diprogram untuk mengikuti gerakan (*mimicking*) operator manusia, yang sangat berguna dalam bidang *teleoperation* (pengendalian jarak jauh), seperti robot penjelajah bawah laut atau robot pengangkat barang berbahaya.

Di sisi navigasi otonom, data *depth map* dari Kinect digunakan untuk membangun peta lingkungan secara real-time. Algoritma seperti *Simultaneous Localization and Mapping* (SLAM) memanfaatkan data kedalaman ini untuk mendeteksi rintangan, menghitung jarak bebas menuju dinding, serta memetakan geometri ruangan. Data dari sensor ini juga sering dikonversi menjadi data *Point Cloud* (awan titik 3D), yang memungkinkan robot melihat bentuk geometri asli dari suatu objek, memungkinkan *grasping* (pengambilan objek) oleh lengan robot secara presisi.

Perlu dipahami bahwa meskipun Kinect saat ini telah dihentikan produksinya oleh Microsoft, warisannya dalam dunia robotika terus hidup. Teknologi yang dipelopokannya telah melahirkan generasi baru sensor 3D yang lebih ringkas dan efisien (seperti Intel RealSense dan Orbbec) yang akan kita bahas pada bab selanjutnya. Namun, memahami fondasi dari Kinect adalah kunci utama untuk menguasai dasar-dasar visi komputer 3D yang menjadi tulang punggung robotika modern saat ini.

BAB II

SENSOR SEJENIS DENGAN MERK LAIN

2.1. Latar Belakang Kebutuhan Sensor Alternatif

Meskipun sensor Microsoft Kinect telah memberikan kontribusi yang luar biasa dalam demokratisasi teknologi visi 3D untuk robotika, perkembangan industri dan kebutuhan riset yang semakin kompleks menuntut adanya alternatif perangkat lain. Pada tahun 2015, Microsoft resmi menghentikan produksi Kinect V1, dan kemudian diikuti oleh penghentian adaptor Kinect V2 pada tahun 2017. Kondisi ini menyebabkan kelangkaan perangkat di pasar sekunder, terutama bagi peneliti dan institusi pendidikan yang membutuhkan sensor baru dengan umur pakai yang panjang. Selain itu, desain Kinect yang awalnya ditujukan untuk indoor gaming memiliki keterbatasan fisik, seperti ukurannya yang cukup besar, kebutuhan daya yang relatif tinggi (terutama V2 yang membutuhkan port USB khusus dengan daya ekstra), serta kerentanannya terhadap interferensi cahaya matahari.

Dalam ekosistem robotika modern, keterbatasan tersebut mendorong pencarian sensor yang lebih ringkas, hemat daya, dan memiliki kemampuan adaptasi yang lebih luas terhadap lingkungan luar ruangan (outdoor). Para insinyur robotika membutuhkan sensor yang dapat diintegrasikan pada platform robot kecil seperti *Unmanned Aerial Vehicles* (UAV) atau drone, *mobile robot* berukuran mini, dan *Single Board Computer* (SBC) seperti Raspberry Pi atau Nvidia Jetson Nano. Selain itu, kebutuhan akan resolusi kedalaman yang lebih tinggi dan *frame rate* yang lebih cepat untuk aplikasi *real-time* mendorong munculnya berbagai inovasi dari perusahaan teknologi lain yang mengisi ruang kosong yang ditinggalkan oleh Kinect.

2.2. Asus Xtion Pro Live

Salah satu penerus paling langsung dari teknologi Kinect V1 adalah sensor buatan Asus yang bernama Xtion Pro Live. Sensor ini dikembangkan berdasarkan kerjasama dengan PrimeSense—perusahaan Israel yang menciptakan chip dan teknologi *structured light* di balik Kinect V1. Karena menggunakan arsitektur yang sama, Asus Xtion Pro Live memiliki prinsip kerja yang identik dengan Kinect V1: memproyeksikan pola *speckle* inframerah dan membaca pantulannya menggunakan sensor IR untuk menghasilkan

depth map.

Perbedaan utama terletak pada desain perangkat kerasnya. Xtion Pro Live dirancang jauh lebih ramping dan ringan dibandingkan Kinect. Perangkat ini tidak memiliki motor *tilt* otomatis, sehingga mengurangi beban mekanis dan potensi kerusakan *hardware*. Selain itu, Xtion Pro Live sepenuhnya mengandalkan daya dari port USB standar (USB 2.0 atau 3.0) tanpa memerlukan adaptor daya eksternal seperti Kinect V2. Dukungan *software* untuk sensor ini juga sangat luas, terutama karena kompatibilitasnya dengan *framework* OpenNI (Open Natural Interaction) dan ROS (Robot Operating System). Hal ini membuat transisi dari pengguna Kinect V1 ke Asus Xtion menjadi sangat mulus, cukup mengganti *package driver* tanpa perlu merombak struktur kode pemrograman robotika secara signifikan. Berikut ini merupakan tampilan Xtion Pro Live pada Gambar 2.1.



Gambar 2.1 Xtion Pro Live

2.3. Intel RealSense (Seri D400 dan L500)

Ketika Microsoft perlahan meninggalkan pasar sensor 3D, Intel melihat peluang besar dan memperkenalkan lini sensor bernama Intel RealSense. Lini produk ini, khususnya seri D400 (seperti D415 dan D435) dan seri L500 (L515), dengan cepat menjadi *standar emas* baru dalam riset dan industri robotika. Alih-alih menggunakan *structured light* seperti Kinect V1, seri D400 menggunakan teknologi *Active Stereo Vision* (Stereo Vision Aktif). Sensor ini memiliki dua kamera inframerah (kiri dan kanan) yang berfungsi seperti mata manusia, dipadukan dengan sebuah proyektor IR yang memancarkan tekstur tak kasat mata untuk membantu kamera membaca objek yang permukaannya rata atau minim tekstur (seperti dinding polos putih).

Keunggulan utama Intel RealSense dibandingkan Kinect adalah kemampuannya bekerja di lingkungan luar ruangan (*outdoor*). Karena tidak bergantung pada pembacaan pola *speckle* yang spesifik, radiasi inframerah dari sinar matahari tidak serta-merta

membutakan sensor. Selain itu, ukurannya sangat kecil (seukuran stik *chewing gum* untuk modul terpisah), menjadikannya ideal untuk dipasang pada drone atau robot *pick-and-place* yang ruangnya terbatas. Seri L500 di sisi lain menggunakan teknologi *Solid-State LiDAR* (LiDAR Mini) yang memanfaatkan *MEMS* (*Micro-Electromechanical Systems*), menawarkan presisi jarak jauh dengan resolusi tinggi yang sebelumnya hanya dimiliki oleh sensor LiDAR otomotif berukuran raksasa. Ketersediaan *SDK* Intel RealSense yang sangat aktif diperbarui dan mendukung berbagai sistem operasi (Windows, Linux, macOS) menjadikannya pilihan paling populer di laboratorium robotika modern. Berikut merupakan tampilan Intel RealSense yang ditunjukkan pada Gambar 2.2.



Gambar 2.2 Intel RealSense

2.4. Stereolabs ZED (Kamera Stereo Pasif)

Pendekatan berbeda ditawarkan oleh perusahaan Stereolabs melalui produknya yang bernama ZED Camera. Berbeda dengan Kinect atau Intel RealSense yang memanfaatkan proyektor IR aktif, ZED adalah kamera *passive stereo* murni. Perangkat ini terdiri dari dua kamera berdefinisi tinggi (hingga 4K) yang dipasang dengan jarak tertentu (*baseline* lebar), meniru sistem penglihatan binokular manusia secara persis. Karena bersifat pasif (tidak memancarkan cahaya apa pun), ZED dapat digunakan dalam kondisi cahaya rendah selama objek masih dapat ditangkap oleh lensa kamera, dan sama sekali tidak terpengaruh oleh interferensi sinar matahari.

Kamera ZED mengandalkan kekuatan komputasi eksternal (GPU komputer) untuk mencocokkan piksel dari kedua kamera (*stereo matching*) secara *real-time* dan menghasilkan *depth map* maupun *point cloud*. Kelemahannya adalah ketergantungan yang tinggi pada tekstur objek; objek yang sangat rata tanpa tekstur (seperti dinding putih mulus) sulit dihitung kedalamannya, namun software ZED yang canggih telah

mengompensasi hal ini melalui algoritma *semi-global matching*. Keunggulan utama ZED adalah jangkauannya yang bisa mencapai puluhan meter (20+ meter), jauh melampaui sensor IR pada umumnya yang terbatas pada 5-10 meter. Hal ini membuat ZED sangat populer untuk robot otonom berskala besar, *drone* survei, dan kendaraan otonom di luar ruangan. Berikut ini merupakan tampilan Stereolabs ZED yang ditunjukkan pada Gambar 2.3.



Gambar 2.3. Stereolabs ZED

2.5. Orbbec (Astra dan Femto)

Perusahaan Orbbec muncul sebagai raksasa baru di industri sensor 3D, terutama setelah mereka mengakuisisi divisi PrimeSense dari Apple pada tahun 2016 (meskipun Apple tetap memegang hak paten inti). Orbbec mengisi ceruk pasar yang ditinggalkan Kinect dengan menawarkan sensor seperti Astra dan Astra Pro. Sensor Astra menggunakan teknologi *structured light* generasi terbaru yang lebih efisien dibanding generasi pertama Kinect. Dengan harga yang ekonomis, Astra menjadi opsi favorit universitas-universitas yang ingin mengadopsi teknologi 3D tanpa harus menguras anggaran riset. Robotika edukatif dan *hobbyist* juga sangat banyak mengadopsi sensor ini karena dukungan driver *open-source* yang kuat (libastra).

Selain lini Astra, Orbbec juga merilis seri Femto yang menggunakan teknologi *Time-of-Flight* (ToF), sebagai kompetitor langsung untuk Intel RealSense L515 dan Kinect V2. Sensor Femto menawarkan *frame rate* kedalaman yang tinggi dan ketahanan terhadap cahaya ambient yang lebih baik dibanding teknologi *structured light*. Ekosistem Orbbec juga menyediakan *Orbbec SDK* dan *wrapper* untuk ROS yang sangat kompatibel dengan arsitektur lama OpenNI, sehingga para peneliti yang sudah memiliki basis kode

dari era Kinect V1 dapat melakukan migrasi ke perangkat Orbbec tanpa harus membangun ulang seluruh *pipeline* pengolahan citra robotika dari nol. Berikut merupakan tampilan Orbbec yang ditunjukkan pada Gambar 2.4.



Gambar 2.4. Orbbec

2.6. Tabel Perbandingan Karakteristik Sensor

Untuk memudahkan pemilihan sensor yang sesuai dengan kebutuhan aplikasi robotika, berikut adalah ringkasan perbandingan karakteristik antar berbagai sensor yang telah dibahas yang ditunjukkan pada Tabel 2.1.

Table 2.1. Perbandingan Karakteristik

Tipe	Depth	Optimal	daya	Lingkungan	Ukuran
Microsoft Kinect V1	Structured Light	0.8 - 4.0 m	Adaptor Eksternal	Indoor	Besar
Microsoft Kinect V2	Time-of-Flight (ToF)	0.5 - 4.5 m	Adaptor Eksternal	Indoor	Sangat Besar
Asus Xtion Pro Live	Structured Light	0.8 - 3.5 m	USB Standar	Indoor	Sedang
Intel RealSense D435	Active Stereo	0.3 - 3.0 m+	USB 3.0	Indoor & Outdoor	Sangat Kecil
Orbbec Astra	Structured Light	0.6 - 8.0 m	USB Standar	Indoor	Sedang
Stereolabs ZED	Passive Stereo	1.0 - 20+ m	USB 3.0	Indoor & Outdoor	Kecil

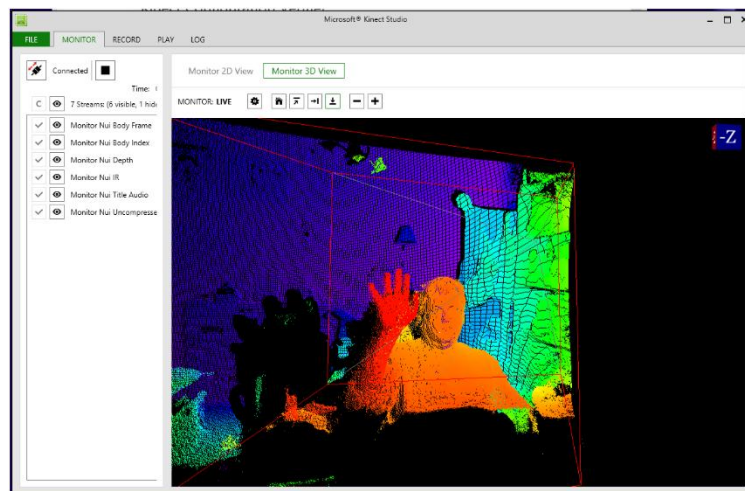
BAB III

DRIVER YANG DIGUNAKAN DAN ALTERNATIFNYA

3.1. Latar Belakang Komunikasi Perangkat Keras dan Perangkat Lunak

Dalam arsitektur sistem robotika modern, sensor tidak berdiri sendiri sebagai entitas terpisah, melainkan harus terintegrasi penuh dengan otak komputasi robot, baik itu berupa *Single Board Computer* (SBC) seperti Raspberry Pi atau Nvidia Jetson, maupun PC industrial. Agar data mentah dari proyektor inframerah, kamera RGB, dan sensor kedalaman dapat diinterpretasikan oleh prosesor, dibutuhkan sebuah lapisan perangkat lunak yang disebut *driver*. Driver berfungsi sebagai penerjemah protokol komunikasi perangkat keras (umumnya melalui bus USB) menjadi struktur data yang dapat dibaca oleh bahasa pemrograman tingkat tinggi seperti C++, Python, atau MATLAB.

Namun, proses pembuatan driver untuk sensor kompleks seperti Kinect bukanlah tugas yang mudah. Protokol komunikasi internal antara kamera IR, proyektor, dan *frame grabber* awalnya dirancang sebagai *closed-source* (proprietary) oleh Microsoft dan PrimeSense. Hal ini menyebabkan pada masa-masa awal perilisannya, sensor ini tidak plug-and-play di luar ekosistem Xbox. Kebutuhan untuk membuka akses ke sensor ini di lingkungan komputasi terbuka mendorong munculnya berbagai inisiatif *open-source* yang akhirnya melahirkan ekosistem driver yang sangat beragam, masing-masing dengan kelebihan, keterbatasan, dan arsitektur yang berbeda. Berikut merupakan tampilan software SDK pada Kinect yang ditunjukkan pada Gambar 3.1.

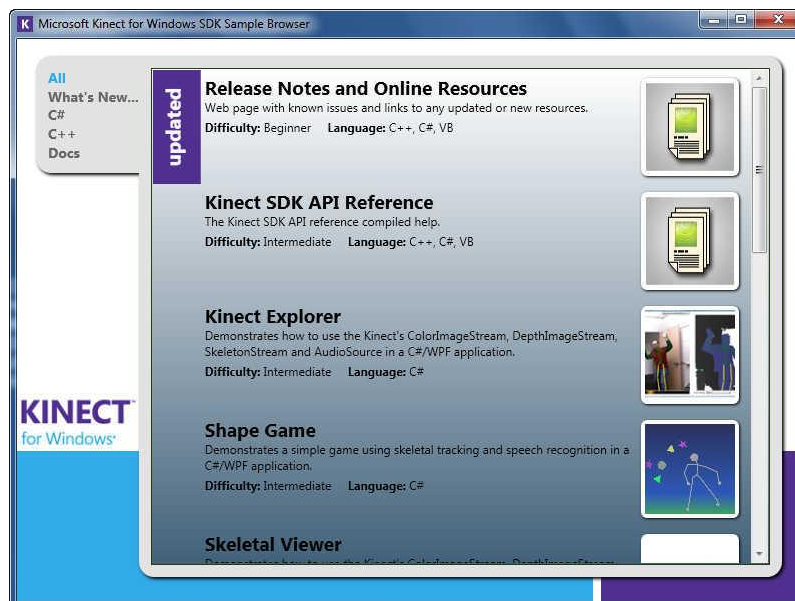


Gambar 3.1. Software Kinect SDK

3.2. Microsoft Kinect SDK (Kinect for Windows SDK)

Driver paling resmi dan *native* untuk perangkat Kinect tentu saja dikembangkan oleh Microsoft, yang dikenal dengan nama Kinect for Windows SDK. Driver resmi ini dirilis secara gratis oleh Microsoft untuk menjembatani pengembangan aplikasi berbasis Kinect pada sistem operasi Windows. Keunggulan utama dari SDK resmi ini adalah stabilitasnya yang tinggi dan akses penuh terhadap fitur-fitur *built-in* yang canggih, terutama algoritma *skeleton tracking* (pelacakan kerangka tubuh) yang sudah teroptimasi dengan baik di tingkat *hardware/software* tingkat rendah. SDK ini juga mendukung *beamforming* pada array mikrofon Kinect untuk isolasi suara.

Meskipun canggih, penggunaan Kinect SDK di dunia robotika memiliki keterbatasan yang signifikan. Pertama, SDK ini secara eksklusif hanya berjalan di atas sistem operasi Windows, sementara sebagian besar ekosistem robotika otonom dan *middleware* (seperti ROS) berjalan di atas sistem operasi berbasis Linux (seperti Ubuntu). Kedua, SDK resmi ini cenderung bersifat *black box*; algoritma *skeleton tracking* dijalankan secara tertutup, sehingga peneliti yang ingin memodifikasi algoritma pengenalan gestur tingkat rendah akan kesulitan melakukannya. Oleh karena itu, meskipun Kinect SDK sangat populer di kalangan pengembang aplikasi *desktop* atau *kiosk* interaktif, para insinyur robotika sering kali menghindarinya demi mencari alternatif yang lebih fleksibel dan lintas-platform (*cross-platform*). Berikut ini merupakan tampilan Microsoft Kinect SDK yang ditunjukkan pada Gambar 3.2.



Gambar 3.2. Microsoft Kinect SDK

3.3. OpenNI (Open Natural Interaction)

Kebutuhan akan *framework* yang independen dari sistem operasi dan bersifat *open-source* dijawab oleh perusahaan PrimeSense melalui peluncuran OpenNI (Open Natural Interaction). OpenNI bukan sekadar driver, melainkan sebuah *framework* aplikasi tingkat tinggi yang memungkinkan komunikasi natural antara manusia dan perangkat (seperti gestur dan suara). PrimeSense merilis OpenNI beserta *driver* perangkat keras (disebut *Sensor Module*) secara *open-source* agar komunitas pengembang dapat menggunakannya tanpa biaya lisensi.

Dalam konteks robotika, OpenNI menjadi sangat berpengaruh karena mendukung sistem operasi Linux, Windows, dan macOS. Framework ini memungkinkan akses ke aliran data RGB, Depth, dan IR secara terpisah maupun sinkron. Yang paling penting, OpenNI memungkinkan komunitas pihak ketiga untuk memasang algoritma *middleware* pilihan mereka di atas lapisan driver. Contoh paling sukses dari *middleware* ini adalah NITE, yang menyediakan algoritma deteksi gestur tangan dan *skeleton tracking* yang bersaing dengan SDK resmi Microsoft. Sayangnya, setelah PrimeSense diakuisisi oleh Apple pada tahun 2013, pengembangan OpenNI secara resmi dihentikan. Namun, warisan kode OpenNI versi 2 (OpenNI2) masih sangat relevan dan banyak digunakan untuk sensor-sensor turunannya, seperti Asus Xtion dan Orbbec Astra. Berikut ini merupakan tampilan OpenNI pada Gambar 3.3.



Gambar 3.3. OpenNI

3.4. Libfreenect (Proyek OpenKinect)

Ketika Kinect pertama kali dirilis pada tahun 2010, Microsoft belum merilis SDK untuk Windows, dan sama sekali tidak ada dukungan untuk sistem non-Windows. Merespons hal ini, komunitas *hacker* dan akademisi di seluruh dunia bergabung dalam proyek bernama OpenKinect. Tujuan utama proyek ini adalah melakukan *reverse-engineering* terhadap protokol USB Kinect V1 untuk menciptakan driver *open-source* yang sepenuhnya independen. Hasil dari proyek monumental ini adalah pustaka (library)

bernama libfreenect.

Libfreenect adalah driver minimalis dan bersih yang berfokus pada satu hal: mengekstrak data mentah (*raw data*) dari sensor. Berbeda dengan SDK Microsoft atau OpenNI+NITE yang menyertakan algoritma *skeleton tracking* yang rumit, libfreenect awalnya hanya menyediakan akses ke aliran data RGB dan Depth. Filosofi "hanya menyediakan data" ini justru disukai oleh banyak peneliti robotika tingkat lanjut, karena mereka dapat membangun algoritma visi komputer mereka sendiri (misalnya menggunakan OpenCV) dari nol tanpa terbebani oleh proses *black box*. Libfreenect mendukung berbagai bahasa pemrograman melalui *wrapper*, seperti Python (python-libfreenect), C, dan Java. Keberhasilan libfreenect juga mendorong lahirnya libfreenect2 beberapa tahun kemudian, yang dirancang khusus untuk mendukung Kinect V2 yang menggunakan teknologi *Time-of-Flight* (ToF) dengan protokol USB 3.0 yang jauh lebih kompleks. Berikut ini merupakan tampilan libfreenect yang di tampilkan pada Gambar 3.4.

OpenKinect/ libfreenect



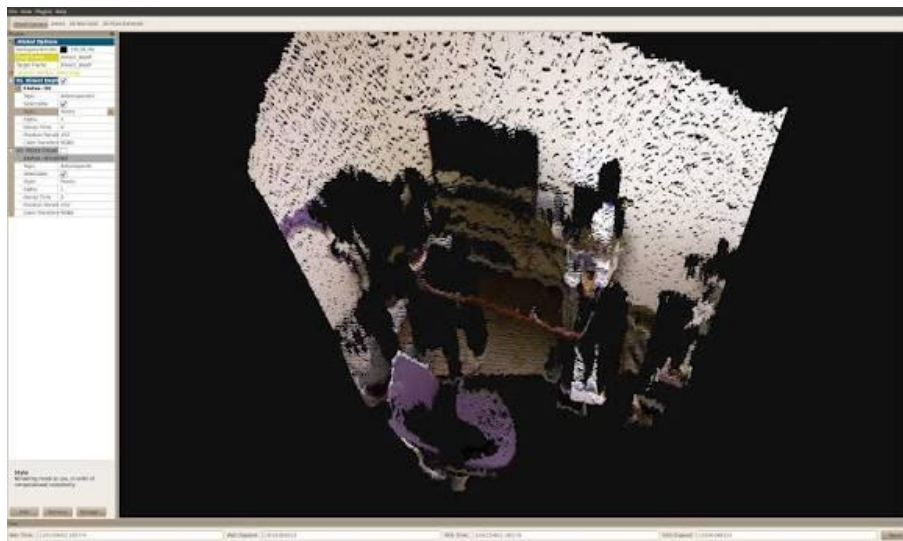
Gambar 3.4. Libfreenect

3.5. Integrasi Driver pada ROS (Robot Operating System)

Bagi sebagian besar praktisi robotika modern, driver sensor tidak dianggap lengkap jika tidak terintegrasi dengan ROS (Robot Operating System). ROS adalah *middleware* paling populer di dunia riset robotika yang menyediakan arsitektur *hardware abstraction* dan komunikasi antarproses yang terdistribusi. Untuk sensor Kinect, komunitas ROS telah menyediakan *wrapper* (pembungkus) yang sangat andal yang mengonversi data dari libfreenect atau OpenNI menjadi *topic* ROS yang dapat di-*subscribe* oleh berbagai node robotika.

Paket driver Kinect di ROS dikenal dengan nama `freenect_stack` (untuk Kinect V1) dan `iai_kinect2` (untuk Kinect V2). Ketika driver ini dijalankan, ia akan secara otomatis membaca data RGB dan Depth dari libfreenect, lalu mempublikasikannya ke

dalam topik ROS standar, yaitu “`cd:yourfolderincomputer/camera/rgb/image_color`” dan “`/camera/depth/image_rect.`” Keuntungan utama menggunakan ROS *wrapper* ini adalah fitur sinkronisasi *timestamp* antara kamera RGB dan Depth yang akurat, serta kalibrasi intrinsik kamera yang otomatis ter-load melalui file *camera_info*. Dengan demikian, sensor Kinect dapat langsung digunakan untuk algoritma SLAM (Simultaneous Localization and Mapping), deteksi objek 3D, atau visual servoing tanpa harus memikirkan detail teknis pengambilan data di tingkat *hardware*. Berikut merupakan tampilan ROS pada Kinect yang ditunjukkan pada Gambar 3.5.



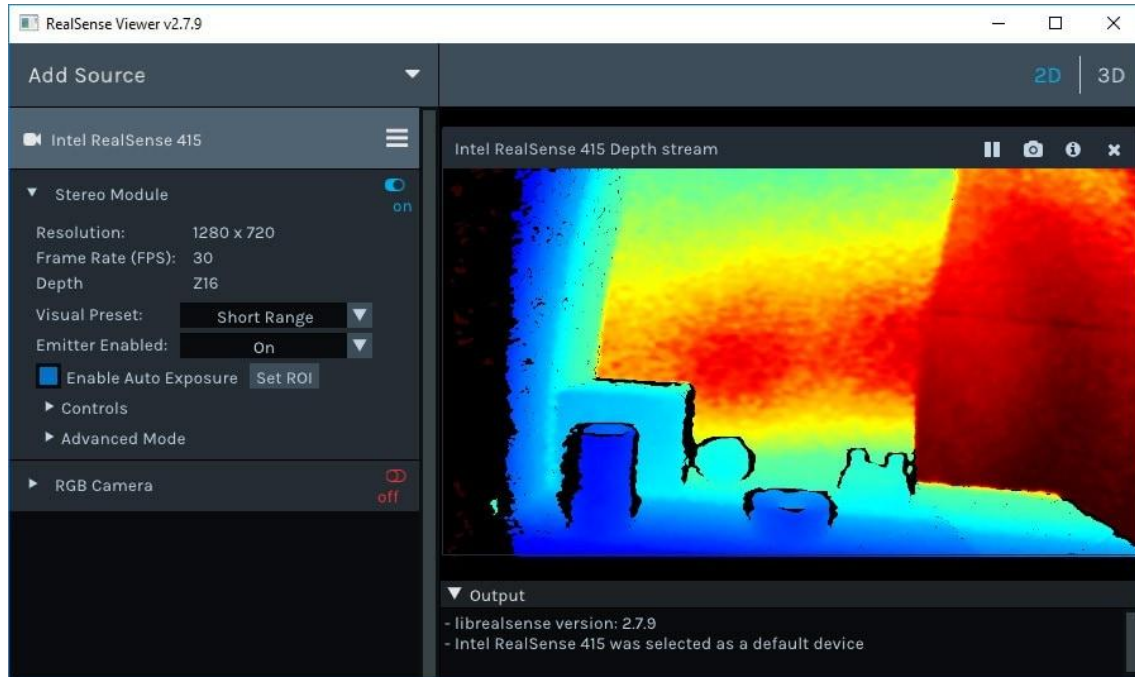
Gambar 3.5. ROS Kinect

3.6. Alternatif Driver Modern (Intel RealSense SDK dan Orbbec SDK)

Seiring bergesernya penggunaan dari Kinect menuju sensor generasi baru seperti Intel RealSense dan Orbbec, ekosistem driver juga mengalami evolusi. Untuk sensor Intel RealSense, driver resminya dikenal sebagai Intel RealSense SDK 2.0 (dengan library C++ bernama *librealsense2*). SDK ini sangat komprehensif, mendukung lintas platform (Linux, Windows, macOS, Android), dan terintegrasi sangat baik dengan ROS melalui paket *realsense2_camera*. Driver ini secara otomatis mengkalibrasi *stereo depth* dan menyediakan filter *post-processing* (seperti *spatial filter* dan *temporal filter*) langsung di tingkat *hardware*, sehingga mengurangi beban komputasi pada CPU robot.

Sementara itu, untuk sensor Orbbec, komunitas masih sangat bergantung pada warisan arsitektur OpenNI2. Orbbec menyediakan Orbbec SDK yang pada dasarnya adalah modifikasi dari OpenNI2 untuk mendukung perangkat keras mereka yang baru. Di

ekosistem ROS, driver Orbbec Astra dilayani oleh paket **astra_camera**. Walaupun teknologinya diwariskan dari masa lalu, adaptasi ini membuktikan bahwa arsitektur perangkat lunak *open-source* yang baik (seperti OpenNI) mampu bertahan hidup dan beradaptasi melintasi beberapa dekade perubahan perangkat keras sensor robotika. Tampilan RealSense SDK pada Gambar 3.6.



Gambar 3.6. RealSense SDK

3.7. Tantangan Bandwidth USB dan Isu Komputasi Edge

Salah satu tantangan teknis yang sering dihadapi saat menggunakan driver-driver tersebut di sistem robotika adalah manajemen *bandwidth* USB. Sensor 3D seperti Kinect V2, RealSense, atau ZED menghasilkan data citra dengan ukuran yang sangat masif. Data RGB beresolusi 1080p pada 30 fps, dikombinasikan dengan data *Depth Map* 16-bit pada resolusi yang sama, membutuhkan *throughput* yang melebihi kapasitas standar USB 2.0. Oleh karena itu, sensor modern mengharuskan penggunaan port USB 3.0 atau di atasnya. Jika *bandwidth* USB tidak mencukupi (misalnya karena *hub* USB berbagi jalur dengan perangkat lain), driver akan mengalami *frame drop* (hilangnya *frame* data) yang berakibat fatal pada algoritma navigasi robot yang membutuhkan data *real-time*.

Selain isu *bandwidth*, pemilihan driver juga harus mempertimbangkan kapasitas komputasi *edge* di robot. Jika robot menggunakan *Single Board Computer* seperti

Raspberry Pi, menjalankan algoritma *skeleton tracking* secara *real-time* melalui OpenNI atau SDK resmi akan membebani prosesor ARM hingga 100%, menyebabkan robot lag (lambat merespons). Solusi alternatifnya adalah menggunakan driver minimalis (seperti libfreenect) dan mengimplementasikan algoritma pelacakan objek yang lebih ringan (seperti *Motion Detection* atau *Color Tracking* menggunakan OpenCV), atau melakukan *offloading* komputasi dengan mengirimkan data *stream* ke komputer *host* melalui jaringan nirkabel. Pemahaman atas karakteristik *software* dan keterbatasan *hardware* inilah yang menjadi kunci keberhasilan dalam mengintegrasikan sistem visi 3D pada robotika.

BAB IV

DATA YANG DIHASILKAN: RGB, DEPTH, SKELETON

4.1. Arsitektur Multistream Sensor Kinect

Salah satu keunggulan utama sensor Kinect dibandingkan kamera konvensional adalah kemampuannya untuk menghasilkan beberapa aliran data (*data stream*) secara simultan dan sinkron dalam satu waktu. Dalam satu periode penangkapan (*frame*), Kinect tidak hanya mengambil satu jenis informasi, melainkan tiga hingga empat modalitas data yang berbeda namun saling melengkapi. Modalitas-modalitas ini mencakup citra warna (RGB), citra kedalaman (Depth), dan data kerangka tubuh (Skeleton), yang kadang disertai dengan citra inframerah mentah (IR).

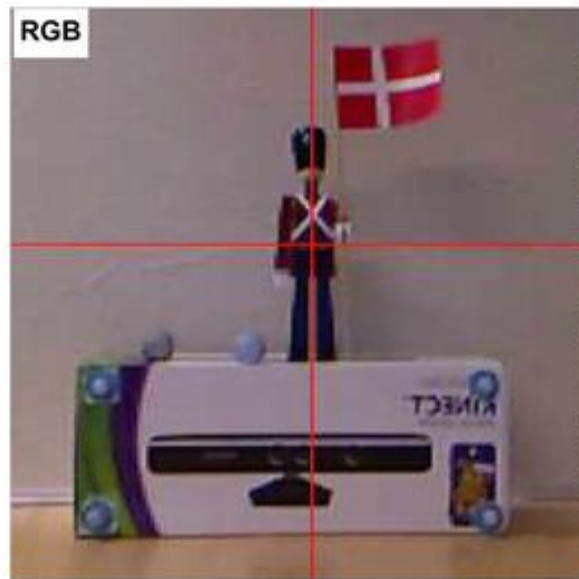
Kemampuan *multistream* ini memungkinkan robot untuk memperoleh persepsi multimodal terhadap lingkungannya. Sama seperti sistem visual manusia yang tidak hanya melihat warna, tetapi juga mampu merasakan jarak secara instingtif, robot dengan Kinect dapat mengenali warna rambu lalu lintas sekaligus mengetahui seberapa jauh rambu tersebut berada darinya. Namun, dari perspektif komputasi dan arsitektur perangkat lunak, menangani multi-aliran data ini membutuhkan pemahaman yang mendalam mengenai bagaimana data tersebut direpresentasikan secara matematis dan komputasional, serta bagaimana sinkronisasi waktu (*timestamping*) antar aliran dijaga agar tidak terjadi *mismatch* (ketidakcocokan) data saat diproses oleh algoritma robotika.

4.2. Data Citra RGB (Red, Green, Blue)

Data pertama dan paling dasar yang dihasilkan oleh Kinect adalah citra warna RGB. Secara esensial, kamera RGB pada Kinect tidak berbeda secara fundamental dengan kamera digital pada umumnya. Citra ini direpresentasikan sebagai matriks dua dimensi (atau tensor tiga dimensi jika memperhitungkan kanal warna) yang berisi nilai intensitas piksel. Untuk Kinect V1, resolusi standar yang sering digunakan adalah 640x480 piksel pada 30 *frame per second* (fps), sedangkan Kinect V2 mampu menghasilkan citra RGB dengan resolusi *Full HD* 1920x1080 piksel.

Dalam komputasi memori, citra RGB disimpan dalam format 8-bit *unsigned integer* per kanal. Artinya, setiap piksel terdiri dari tiga nilai yang mewakili intensitas merah (R), hijau (G), dan biru (B), di mana setiap nilai memiliki rentang antara 0 hingga

255. Dalam konteks robotika, data RGB digunakan untuk tugas-tugas yang membutuhkan informasi tekstur dan warna, seperti klasifikasi objek (misalnya membedakan antara bola merah dan bola biru), deteksi rambu jalan, hingga pengenalan wajah. Kelemahan data RGB adalah ketidakmampuannya untuk memberikan informasi spasial 3D secara langsung; sebuah bola kecil yang dekat dengan kamera akan terlihat sama besar dengan bola besar yang jauh dari kamera (fenomena perspektif).

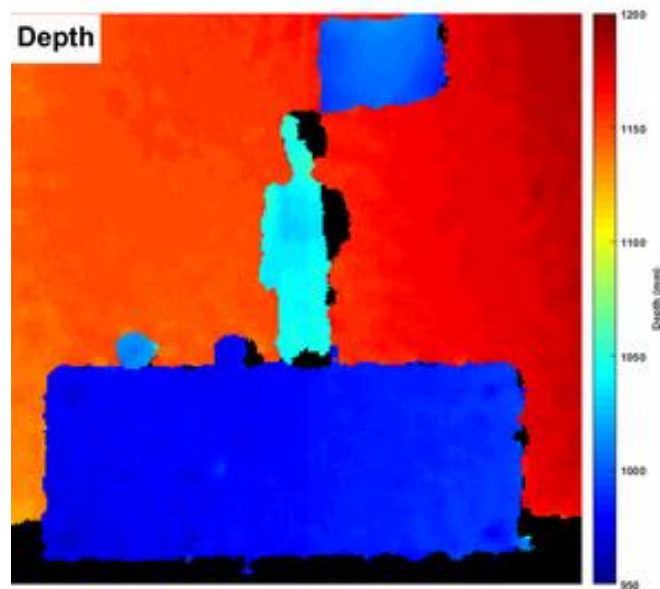


Gambar 4.1. Kinect Output RGB

4.3. Data Citra Kedalaman (Depth Image)

Data kedalaman atau *Depth Image* adalah inovasi inti yang membuat Kinect begitu berharga dalam robotika. Berbeda dengan citra RGB yang nilainya merepresentasikan warna, nilai piksel pada *Depth Image* merepresentasikan jarak fisik antara sensor dan objek yang memantulkan cahaya. Secara matematis, *Depth Map* dapat didefinisikan sebagai fungsi diskret $D(u, v)$, di mana u dan v adalah koordinat piksel pada bidang gambar, dan nilai D adalah jarak (umumnya dalam milimeter) dari bidang *image plane* kamera ke objek. Dari segi format data, *Depth Image* biasanya direpresentasikan dalam matriks 16-bit *unsigned integer* (*CV_16UC1* dalam terminologi OpenCV). Penggunaan 16-bit (yang memiliki rentang nilai 0 hingga 65,535) diperlukan karena jarak yang diukur bisa mencapai beberapa meter (misalnya 8000 milimeter atau 8 meter), yang tidak mungkin ditampung oleh format 8-bit standar (maksimal 255). Nilai 0 pada matriks kedalaman biasanya memiliki makna khusus, yaitu "*don't care*" atau "*no depth data*". Hal

ini terjadi ketika sinar inframerah tidak berhasil dipantulkan kembali ke sensor (misalnya pada bahan kaca bening atau objek yang menyerap cahaya secara sempurna). Secara visual, jika *Depth Image* ditampilkan di layar, objek yang dekat dengan sensor akan tampak lebih gelap (atau lebih terang, tergantung peta warna/*colormap* yang digunakan), sementara objek yang jauh akan memiliki intensitas warna yang berbeda.



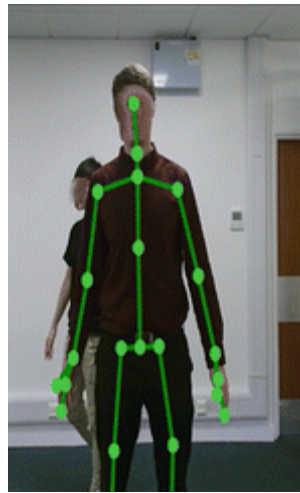
Gambar 4.2. Depth Output Kinect

4.4. Data Kerangka Tubuh (Skeleton Tracking)

Data *Skeleton* (kerangka) adalah bentuk data abstrak tingkat tinggi yang dihasilkan dari pemrosesan lanjutan terhadap *Depth Image*. Alih-alih berupa matriks piksel, data skeleton berupa kumpulan koordinat 3D yang merepresentasikan sendi-sendi (*joints*) utama tubuh manusia. Berkat algoritma *machine learning* yang tertanam di dalam *driver* (seperti NITE pada OpenNI atau Kinect SDK), sensor mampu mengidentifikasi pola manusia dari sekumpulan titik kedalaman, lalu mengekstrak posisi anatomi seperti kepala, bahu, siku, pergelangan tangan, pinggul, dan lutut. Kinect V1 mampu melacak hingga 2 orang secara simultan dengan 20 titik sendi per orang, sementara Kinect V2 mampu melacak hingga 6 orang dengan 25 titik sendi.

Secara struktur data, setiap titik sendi (*joint*) direpresentasikan dalam vektor tiga dimensi $\mathbf{J}=(x,y,z)$. Penting untuk dicatat bahwa koordinat ini biasanya berada dalam ruang koordinat kamera (*camera space*), di mana sumbu *Z* merepresentasikan jarak lurus ke

depan (kedalaman), sumbu X merepresentasikan posisi horizontal (kiri-kanan), dan sumbu Y merepresentasikan posisi vertikal (atas-bawah). Selain koordinat 3D ini, algoritma *skeleton tracking* juga biasanya menyediakan status pelacakan untuk setiap sendi, yang menunjukkan apakah sendi tersebut "terlacak dengan baik" (*tracked*), "diperkirakan" (*inferred*), atau "tidak terlacak" (*untracked*). Dalam konteks robotika, data ini mengubah interaksi manusia-robot (HRI); robot tidak lagi memerlukan *joystick*, melainkan dapat diprogram untuk merespons perintah suara atau gerakan tangan manusia yang berdiri di depannya.



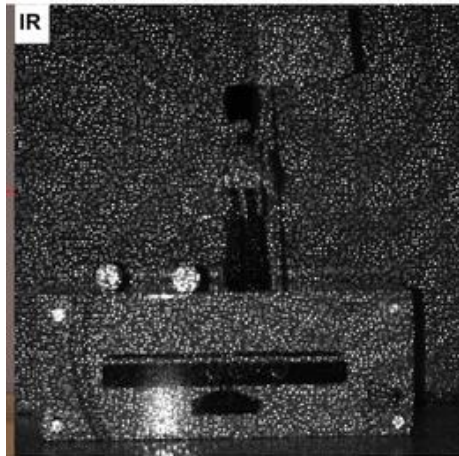
Gambar 4.3. Skeleton Tracking Output Kinect

4.5. Data Inframerah (IR Stream)

Selain RGB dan Depth, Kinect juga menyediakan akses langsung ke aliran data inframerah (IR). Data ini pada dasarnya adalah citra grayscale mentah yang ditangkap oleh kamera IR sebelum diolah oleh algoritma perhitungan kedalaman. Pada Kinect V1, citra IR menampilkan pola bintik-bintik (*speckle pattern*) yang diproyeksikan oleh emiter IR ke ruangan. Pada Kinect V2 yang menggunakan ToF, citra IR menampilkan intensitas pantulan pulsa cahaya inframerah.

Meskipun jarang digunakan sebagai satu-satunya sumber data dalam navigasi robot, aliran IR memiliki aplikasi khusus yang sangat berguna. Karena kamera IR tidak terpengaruh oleh pencahayaan ruangan yang gelap (karena ia membaca cahaya inframerahnya sendiri), data ini sangat ideal untuk sistem keamanan robot di malam hari, deteksi gerakan di ruang gelap, atau penggunaan di lingkungan indoors yang minim cahaya. Selain itu, peneliti sering menggunakan citra IR untuk mempelajari perilaku

refleksi material dari berbagai permukaan objek secara mikroskopis.



Gambar 4.4. Infrared Output Kinect

4.6. Sinkronisasi dan Registrasi Antar-Modalitas

Mengingat Kinect menghasilkan tiga aliran data yang berbeda secara simultan, muncul sebuah tantangan teknis: bagaimana memastikan bahwa piksel (u,v) pada citra RGB merujuk pada titik fisik yang sama persis dengan piksel (u,v) Karena lensa kamera RGB dan sensor IR (Depth) terpisah secara fisik dengan jarak beberapa sentimeter (*baseline*), terdapat pergeseran parallax antara kedua citra tersebut. Sebuah objek yang berada di ujung kiri bingkai RGB mungkin akan sedikit bergeser ke kanan pada bingkai Depth. Untuk mengatasi hal ini, driver Kinect menyediakan fitur *Registration* atau *Alignment*. Proses ini memetakan koordinat dari ruang kamera Depth ke ruang kamera RGB menggunakan matriks kalibrasi intrinsik dan ekstrinsik yang telah difabrikasi pabrik. Selain sinkronisasi spasial, sistem juga harus memastikan sinkronisasi temporal (*timestamping*). Ketika sebuah robot bergerak cepat, *frame* RGB dan Depth harus diambil pada millidetik yang sama. Jika sinkronisasi waktu gagal, objek yang bergerak akan tampak terpisah antara citra warna dan citra kedalamannya, menyebabkan algoritma *fusion 3D* (seperti *colored point cloud*) menjadi kacau dan menghasilkan *ghosting effect*. Oleh karena itu, manajemen antrean data (*data queueing*) dan pemilihan *frame* yang memiliki *timestamp* identik adalah langkah krusial dalam pemrograman robotika berbasis Kinect.

BAB V

PEMROSESAN CITRA RGB DAN DEPTH MENGGUNAKAN OPENCV

5.1. Mengakses Data RGB Image Stream

Kinect adalah menghubungkan perangkat Kinect ke komputer menggunakan adaptor daya dan kabel USB, kemudian memastikan driver OpenNI2 telah berjalan dengan baik. Setelah koneksi berhasil dilakukan, program Python dijalankan dengan mengimpor pustaka yang diperlukan, seperti OpenNI2, NumPy, dan OpenCV. Selanjutnya dilakukan proses inisialisasi menggunakan fungsi `openni2.initialize()` untuk mengaktifkan komunikasi antara sistem operasi dan sensor Kinect. Setelah perangkat berhasil dikenali melalui fungsi `Device.open_any()`, program membuat RGB Image Stream menggunakan fungsi `create_color_stream()` kemudian mengaktifkannya melalui `start()`. Tahap ini memungkinkan kamera RGB mulai mengirimkan data citra berwarna secara kontinu dari sensor menuju komputer. Alur proses inisialisasi dan pembentukan RGB stream

Setelah RGB stream aktif, program melakukan pembacaan frame secara berulang menggunakan fungsi `read_frame()`. Data citra yang diperoleh masih berupa buffer sehingga perlu dikonversi menjadi array NumPy agar dapat diproses lebih lanjut. Karena format keluaran Kinect menggunakan ruang warna RGB, sedangkan OpenCV menggunakan format BGR, maka dilakukan konversi warna menggunakan fungsi `cv2.cvtColor()` sebelum citra ditampilkan. Selanjutnya, hasil konversi divisualisasikan secara real-time menggunakan fungsi `cv2.imshow()` di dalam sebuah perulangan (*loop*) sehingga pengguna dapat memantau keluaran kamera secara langsung dengan kecepatan sekitar 30 frame per detik. Proses pengambilan citra dihentikan ketika pengguna menekan tombol Q, kemudian program menutup RGB stream, melepaskan koneksi Kinect, dan menutup seluruh jendela OpenCV.

Dalam sistem visi komputer dan robotika, citra RGB (Red, Green, Blue) yang ditangkap oleh sensor Kinect bukanlah sekadar gambar statis yang terlihat oleh mata manusia, melainkan merupakan sebuah struktur data matematis yang merepresentasikan intensitas cahaya pada bidang diskret. Secara fundamental, citra digital adalah fungsi diskret dua dimensi yang memiliki nilai pada setiap koordinat spasial. Karena sensor Kinect menangkap tiga kanal warna primer—Merah (R), Hijau (G), dan Biru (B)—citra

ini direpresentasikan sebagai tensor tiga dimensi (3D array). Secara matematis, tensor citra ini dinotasikan sebagai:

$$IRGB \in ZH \times W \times 3$$

Di mana:

H = (Height) adalah resolusi vertikal dalam piksel,

W = (Width) adalah resolusi horizontal

3 = merepresentasikan jumlah kanal warna.

Untuk mengekspresikan nilai piksel pada koordinat tertentu, kita menggunakan notasi indeks $IRGB(u, v, c)$ di mana

$$u \in [0, W - 1], v \in [0, H - 1], c \in \{0, 1, 2\}$$

Pada notasi tersebut:

U = merepresentasikan sumbu horizontal (kolom),

V = merepresentasikan sumbu vertikal (baris), dan

C = adalah indeks kanal. Karena kedalaman warna (bit depth) yang digunakan oleh Kinect adalah 8-bit *unsigned integer*, rentang nilai intensitas cahaya pada setiap piksel dan kanal dibatasi secara ketat pada:

$$IRGB(u, v, c) \in [0, 255]$$

Nilai 0 menandakan intensitas cahaya minimum (hitam absolut/tidak ada cahaya pada kanal tersebut), sedangkan 255 menandakan intensitas maksimum (saturasi warna penuh). Pemahaman mengenai struktur tensor ini sangat krusial bagi insinyur robotika, karena menentukan bagaimana algoritma konvolusi, filtering, atau deteksi objek akan mengakses memori secara efisien.

Proses penangkapan citra dimulai dari lensa kamera yang memfokuskan cahaya ke sensor CMOS. Sensor ini menggunakan susunan filter warna mozaik yang dikenal sebagai Bayer Filter. Filter ini memastikan setiap piksel fisik pada sensor hanya menangkap satu frekuensi cahaya (R, G, atau B). Data mentah dari sensor ini kemudian dilewatkan ke prosesor internal Kinect yang menjalankan algoritma *demosaiicing* untuk menginterpolasi nilai dua kanal yang hilang pada setiap piksel. Secara matematis, interpolasi linear *demosaiicing* untuk sebuah piksel hijau (G) pada baris merah (R) dapat direpresentasikan secara sederhana sebagai rata-rata dari piksel:

$$Ginterpolated(u, v) = 4G(u - 1, v) + G(u + 1, v) + G(u, v - 1) + G(u, v + 1)$$

Hasil akhir dari proses ini adalah tensor *IRGB* yang lengkap, yang dikirimkan ke komputer melalui bus USB dengan kecepatan 30 *frame per second* (fps). Besar memori yang dialokasikan untuk menampung satu frame citra secara matematis dihitung melalui persamaan kapasitas memori:

$$M_{byte} = 8W \times H \times C \times b_{bit}$$

Di mana

$C = 3$ (jumlah kanal)

$B_{bit} = 8$ (kedalaman bit per kanal). Untuk resolusi standar Kinect V1 ($W=640, H=480$), diperoleh:

$$M_{byte} = 8640 \times 480 \times 3 \times 8 = 921.600 \text{ byte} \approx 0.92 \text{ MB per frame}$$

Data sebesar ini yang datang 30 kali per detik membutuhkan *throughput* bandwidth USB sebesar $\approx 27.6\text{MB/s}$, yang masih aman berada di bawah batas teoritis USB 2.0 (60 MB/s).

5.2. Inisialisasi dan Konfigurasi Perangkat Melalui OpenNI

Sebelum tensor *IRGB* dapat diakses oleh program Python, kita harus membangun jalur komunikasi *hardware-software* yang stabil. Dalam ekosistem robotika, pustaka *openni2* (Open Natural Interaction versi 2) merupakan *middleware* standar yang menangani abstraksi protokol USB Kinect. Langkah pertama adalah memanggil fungsi **openni2.initialize()**, yang memuat *dynamic-link library* (DLL) driver ke dalam memori komputer. Tanpa inisialisasi ini, sistem operasi akan menolak permintaan *read/write* ke perangkat Kinect karena tidak ada *interpreter* protokol yang tersedia.

Setelah driver dimuat, perangkat fisik harus dibuka menggunakan **Device.open_any()**. Fungsi ini melakukan *scanning* pada bus USB host dan mengambil alih kontrol eksklusif atas sensor Kinect pertama yang valid ditemukan. Penting untuk dicatat bahwa pada sistem operasi Linux, izin *root* (*sudo*) atau aturan *udev* yang tepat seringkali diperlukan agar fungsi ini tidak mengembalikan *exception* *Permission Denied*. Selanjutnya, kita mendefinisikan *stream* data melalui **create_color_stream()**. Objek *stream* ini pada awalnya berada dalam kondisi *idle* (berhenti) dan harus dikonfigurasi secara eksplisit menggunakan **set_video_mode()**.

Konfigurasi *video mode* mencakup penetapan parameter dimensional

$$V = \{W, H, fps, format\}$$

. Pemilihan parameter ini adalah sebuah *trade-off* (kompromi) dalam desain robotika. Resolusi tinggi (misal $W = 1280, H = 960$) menghasilkan kepadatan informasi spasial yang lebih besar, sangat bermanfaat untuk mendeteksi objek kecil dari jarak jauh. Namun, konsekuensi matematisnya adalah peningkatan dramatis pada beban komputasi. Ketika W dan H dinaikkan, jumlah operasi matematis pada algoritma *convolution* (misalnya deteksi tepi Canny) akan meningkat secara kuadratik:

$$O(\text{convolution}) \propto W \times H \times K_w \times K_h$$

Di mana:

K_w dan K_h

Adalah dimensi kernel filter. Oleh karena itu, untuk menjaga keseimbangan antara detail visual dan kebutuhan komputasi *real-time* pada prosesor robot (seperti Raspberry Pi), resolusi 640×480 pada 30 fps menjadi pilihan yang paling umum dan optimal. Setelah konfigurasi ditetapkan, metode **start()** dipanggil, memerintahkan *firmware* Kinect untuk memulai *polling* data CMOS dan mengirimkan tensor *IRGB* secara kontinu.

Berikut adalah implementasi kode untuk proses inisialisasi dan konfigurasi:

```
import numpy as np
import cv2
from openni import openni2

# Path tempat driver OpenNI2 (Redist) berada
OPENNI2_PATH = "C:/Program Files/OpenNI2/Redist"

# 1. Inisialisasi Driver Middleware
openni2.initialize(OPENNI2_PATH)
dev = openni2.Device.open_any()

if dev is None:
    raise Exception("Gagal mendeteksi sensor Kinect! Periksa koneksi USB.")
```

```

# 2. Membuat Objek Stream RGB
rgb_stream = dev.create_color_stream()

# 3. Mengkonfigurasi Parameter Dimensi V = {640, 480, 30, RGB888}
video_mode =
openni2.VideoMode(pixelFormat=openni2.PIXEL_FORMAT_RGB888,
                    resolutionX=640,
                    resolutionY=480,
                    fps=30)
rgb_stream.set_video_mode(video_mode)

# 4. Mengaktifkan Transmisi Data
rgb_stream.start()
print("RGB Stream aktif: Tensor I_RGB(480, 640, 3) siap
dibaca.")

```

5.3. Pembacaan Buffer dan Konversi Ruang Warna

Ketika *stream* aktif, data tensor *IRGB* tidak secara magis muncul sebagai variabel array; ia tersimpan dalam *memory buffer* raw (mentah) di ruang kernel sistem operasi yang harus diekstraksi secara eksplisit oleh program. Dalam *main loop* pemrosesan, fungsi **read_frame()** dipanggil untuk melakukan *memory mapping*, mengambil satu frame dari *queue* antrean FIFO (First-In-First-Out) sensor. Objek *frame* yang dikembalikan memiliki *buffer* berupa kumpulan byte *unsigned integer* 8-bit (1 byte-per piksel per kanal). Karena pustaka NumPy dan OpenCV membutuhkan representasi matriks terstruktur, *buffer* 1D ini harus dikonversi menggunakan **np.frombuffer()** dan kemudian di-*reshape* (dibentuk ulang) dari array 1D menjadi tensor 3D sesuai dimensi $H \times W \times 3$. Tahapan kritis yang sering menjadi sumber *bug* bagi programmer pemula adalah konversi ruang warna (*color space transformation*). Secara *default*, protokol internal Kinect mengurutkan byte data dalam urutan kanal Red-Green-Blue (RGB). Namun, standar internal pustaka OpenCV—yang berakar pada sejarah format file BMP pada era Windows 90-an—mengadopsi urutan kanal Blue-Green-Red (BGR). Jika tensor *IRGB*

Kinect ditampilkan langsung menggunakan `cv2.imshow()` tanpa transformasi, akan terjadi distorsi warna yang fundamental: objek merah akan terlihat biru, dan objek biru akan terlihat merah, sementara kanal hijau tetap pada posisinya.

Secara matematis, konversi dari RGB ke BGR adalah operasi permutasi matriks pada sumbu kanal ketiga (*c*). Didefinisikan vektor warna piksel pada ruang RGB sebagai:

$$\mathbf{v}_{RGB} = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Maka vektor pada ruang BGR, $\mathbf{v}_{BGR}$ diperoleh melalui perkalian dengan matriks permutasi $\mathbf{P}_{RGB} \rightarrow BGR$:

$$\mathbf{P}_{RGB} \rightarrow BGR = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Operasi perkalian matriks ini menghasilkan sistem persamaan linear yang memetakan setiap indeks kanal:

$IBGR(u,v,0) = IRGB(u,v,2)$ (Kanal Biru memperoleh data Red)

$IBGR(u,v,1) = IRGB(u,v,1)$ (Kanal Hijau tetap pada posisi tengah)

$IBGR(u,v,2) = IRGB(u,v,0)$ (Kanal Merah memperoleh data Blue)

OpenCV menyediakan fungsi `cv2.cvtColor()` yang mengeksekusi perkalian matriks $\mathbf{P}$ ini di tingkat bahasa C yang teroptimasi (menggunakan instruksi SIMD prosesor) sehingga sangat cepat dan tidak membebani prosesor Python. Berikut adalah implementasi kode untuk pembacaan, *reshaping*, dan transformasi ruang warna:

```
while True:
    # 1. Ekstraksi Frame dari Queue Buffer
    frame = rgb_stream.read_frame()

    # 2. Konversi Buffer Memori Mentah ke Tensor NumPy 3D
    # Buffer berisi 921.600 byte (640*480*3)
    rgb_data_raw = np.frombuffer(frame.get_buffer_as_uint8(),
    dtype=np.uint8)

    # Reshape array 1D menjadi Tensor 3D: I_RGB(v, u, c)
    rgb_array = rgb_data_raw.reshape((480, 640, 3))
```

```
# 3. Transformasi Matematis Ruang Warna (Permutasi Kanal)
# Mengalikan vektor piksel dengan matriks P_RGB->BGR
bgr_array = cv2.cvtColor(rgb_array, cv2.COLOR_RGB2BGR)

# Catatan: Jika langkah 3 dilewatkan, warna display akan
terdistorsi!
```

5.4. Visualisasi Real-Time dan Manajemen Sumber Daya

Setelah tensor *IBGR* yang valid diperoleh, tahap selanjutnya adalah merender data numerik tersebut ke dalam antarmuka grafis (GUI) menggunakan **cv2.imshow('Window_Name', bgr_array)**. Fungsi ini membaca nilai setiap elemen tensor *IBGR*(*u,v,c*) dan memetakan intensitas [0,255] ke dalam skala kecerahan piksel pada monitor. Agar jendela GUI ini terus diperbarui secara *real-time* mengikuti perubahan tensor dari sensor (sebanyak 30 kali per detik), fungsi **cv2.waitKey(delay)** harus dieksekusi di dalam *loop*. Parameter **delay** dalam milidetik menentukan berapa lama OpenCV menunggu input keyboard sebelum melanjutkan iterasi. Untuk keperluan *streaming*, nilai 1 ms digunakan agar loop berjalan secepat mungkin, memaksimalkan *frame rate* tampilan sesuai kapasitas komputasi CPU.

Mekanisme penghentian program (*program termination*) adalah aspek kritis dalam arsitektur sistem robotika. Dalam komunikasi USB, sensor Kinect terus-menerus melakukan *polling* dan mengirimkan paket data ke *host controller* komputer, yang mengalokasikan memori *ring buffer* untuk menampungnya. Jika program Python dihentikan secara paksa (misalnya dengan mengklik tombol 'X' pada terminal, atau *kill process*) tanpa mengeksekusi rutin *cleanup*, antrian *buffer* ini tidak akan dikosongkan oleh sistem operasi. Akibatnya, *thread* driver USB akan mengalami *deadlock* (macet), sensor Kinect akan *hang* (status *busy* tanpa respons), dan *memory leak* akan terjadi di RAM komputer. Untuk memulihkan kondisi ini, programmer harus melakukan *hard reset* fisik dengan mencabut dan mencolokkan ulang kabel USB Kinect, yang sangat tidak praktis pada robot otonom yang sedang beroperasi.

Oleh karena itu, perulangan *streaming* harus dilengkapi dengan kondisi *exit* yang elegan, misalnya menangkap sinyal keyboard *ASCII* untuk tombol 'q'. Ketika *break*

condition terpenuhi, program harus secara berurutan memanggil tiga fungsi pembersihan: **rgb_stream.stop()** untuk mengirimkan sinyal terminasi ke *firmware* Kinect agar menghentikan transmisi CMOS; **openni2.unload()** untuk membebaskan memori DLL driver dari prosesor; dan **cv2.destroyAllWindows()** untuk me-release resource windowing system (X11 atau Win32) yang dialokasikan oleh OpenCV. Dengan rutin pembersihan ini, status USB kembali ke *idle*, memori dikosongkan, dan program dapat dijalankan kembali tanpa perlu melakukan *hardware reset*.

Berikut adalah kode lengkap untuk visualisasi dan penutupan sumber daya yang aman:

```
# Melanjutkan kode di dalam while True:

# 4. Rendering Tensor ke GUI Monitor
cv2.imshow('Kinect RGB Real-Time Visualization', bgr_array)

# 5. Mekanisme Exit & Polling Keyboard
# waitKey(1) menahan loop 1ms, & 0xFF masking 8-bit untuk
kompatibilitas OS
key_press = cv2.waitKey(1) & 0xFF

# Jika input ASCII == 'q' (113), eksekusi break loop
if key_press == ord('q'):
    print("Menerima sinyal exit (q), menghentikan
stream...")
    break

# == RUTIN CLEANUP SUMBER DAYA (WAJIB DIEKSEKUSI) ==
# 1. Menghentikan transmisi data dari Firmware Kinect
rgb_stream.stop()

# 2. Membebaskan memori DLL Driver OpenNI dari OS
openni2.unload()
```



```
# 3. Me-release resource Windowing System OpenCV
cv2.destroyAllWindows()

print("Sistem dihentikan dengan aman. Tidak ada memory leak atau
USB deadlock.")
```

Berikut ini merupakan hasil program pada Bab 5 sebagai berikut yang ditujukan pada Gambar 5.1



Gambar 5.1. Output Program Bab 5

BAB VI

MENGAKSES DEPTH IMAGE STREAM

6.1. Arsitektur dan Representasi Matematis Citra Kedalaman

Transisi dari citra RGB menuju citra kedalaman (Depth Image) merupakan lompatan konseptual yang paling signifikan dalam persepsi robotika modern. Jika citra RGB, *IRGB* merepresentasikan *texture* dan warna objek, citra Depth *ID* merepresentasikan topologi spasial dan geometri lingkungan. Alih-alih menangkap pantulan cahaya tampak, sensor inframerah Kinect mengukur waktu tempuh atau distorsi pola cahaya untuk menghitung jarak absolut antara bidang sensor dan permukaan objek di dunia nyata. Secara matematis, citra kedalaman adalah fungsi diskret yang memetakan setiap koordinat piksel (u, v) pada bidang citra dua dimensi ke dalam jarak fisik tiga dimensi Z (kedalaman) pada ruang kamera:

$$ID: Z^2 \rightarrow R, (u, v) \mapsto Z(u, v)$$

Karena sifatnya yang merepresentasikan jarak fisik dalam milimeter, representasi data Depth sangat berbeda dari RGB. Kinect V1 memiliki jangkauan operasional efektif (*sweet spot*) dari 800 mm hingga 4000 mm. Untuk menampung rentang nilai numerik yang sebesar ini, format 8-bit unsigned integer (maks 255) yang digunakan oleh RGB jelas tidak memadai. Oleh karena itu, standar industri untuk Depth Image menggunakan 16-bit unsigned integer (*CV_16UC1* dalam terminologi OpenCV). Secara matematis, tensor depth didefinisikan sebagai matriks 2 dimensi:

$$ID \in ZH \times W, \text{ di mana } ID(u, v) \in [0, 2^{16} - 1] = [0, 65535]$$

Konfigurasi standar OpenNI (*PIXEL_FORMAT_DEPTH_1_MM*), setiap nilai piksel $ID(u, v)$ memiliki korespondensi linear langsung dengan jarak fisik dalam milimeter, sehingga konversi ke satuan dunia nyata sangat trivial:

$$Z_{mm}(u, v) = ID(u, v) \times 1 \text{ mm}$$

Namun, terdapat sebuah nilai khusus yang sangat penting dalam logika pemrograman robotika: nilai 0. Piksel dengan $ID(u, v) = 0$ bukan berarti objek berada pada jarak nol milimeter di depan lensa, melainkan merepresentasikan kondisi "No Depth Data" atau *invalid pixel*. Kondisi ini terjadi ketika sinar inframerah tidak berhasil memantul kembali ke sensor, yang umumnya diakibatkan oleh material yang menyerap IR (infrared), permukaan transparan (kaca yang meneruskan IR), atau objek yang berada

di luar batas jangkauan sensor (di bawah 800 mm atau di atas 4000 mm). Dalam desain algoritma navigasi, piksel dengan nilai 0 ini *harus* dieksklusi secara matematis dari kalkulasi rata-rata atau filtering untuk mencegah anomali data yang berpotensi membuat robot menabrak objek yang sebenarnya ada namun "buta" oleh sensor.

6.2. Inisialisasi, Konfigurasi, dan Registrasi Spasial

Inisialisasi Depth Stream mengikuti pola arsitektur yang serupa dengan RGB Stream, namun dengan perbedaan parameter konfigurasi yang esensial. Objek stream dibuat menggunakan `dev.create_depth_stream()`, dan mode video diatur ke format `PIXEL_FORMAT_DEPTH_1_MM` dengan resolusi umumnya 640×480 pada 30 fps. Meskipun resolusi spasialnya tampak sama dengan RGB, informasi yang dikandung oleh setiap piksel jauh lebih padat (16-bit vs 8-bit per kanal pada RGB). Hal ini menyebabkan bandwidth data Depth mencapai ≈ 18.4 MB/s, yang membebani bus USB secara signifikan. Tahapan kritis yang tidak boleh dilewatkan pada konfigurasi depth adalah Image Registration (Registrasi Citra). Secara fisik, sensor inframerah (IR) yang menghasilkan Depth Map dan sensor CMOS yang menghasilkan RGB Image terpisah oleh jarak horizontal beberapa sentimeter, yang disebut sebagai *baseline* (b).

Akibat perbedaan posisi fisik ini, sudut pandang (*perspective*) kamera IR dan kamera RGB tidak identik; sebuah objek di dunia nyata akan diproyeksikan pada koordinat piksel yang berbeda pada kedua sensor. Fenomena ini dikenal sebagai Parallax Effect. Jika kedua citra ditumpuk tanpa koreksi, akan terjadi *misalignment* (pergeseran spasial) yang membuat tepi objek pada Depth Map tidak cocok dengan tepi objek pada RGB Image, menghancurkan validitas data fusi 3D. Untuk mengatasi parallax, OpenNI menyediakan mode **IMAGE_REGISTRATION_DEPTH_TO_COLOR**. Ketika mode ini diaktifkan, firmware Kinect secara otomatis melakukan transformasi geometris pada matriks ID di tingkat hardware. Secara matematis, proses ini mengaplikasikan matriks transformasi ekstrinsik $T_{IR \rightarrow RGB}$ yang mengkompensasi baseline b dan perbedaan parameter intrinsik kamera (f_x, f_y, c_x, c_y). Proyeksi piksel depth dari ruang kamera IR, p_{IR} ke ruang piksel kamera RGB, p_{RGB} dihitung melalui persamaan perspektif 3D:

$$P_{3D, RGB} = T_{IR \rightarrow RGB} \cdot P_{3D, IR}$$

$$p_{RGB} = \begin{bmatrix} f_x, RGB + \frac{x_{RGB}}{z_{RGB}} c_x, RGB \\ f_y, RGB + \frac{y_{RGB}}{z_{RGB}} c_y, RGB \end{bmatrix}$$

Aktivasi registrasi memaksa Depth Image yang di-output oleh stream telah "di-warping" sehingga koordinat (u,v) pada ID tepat merepresentasikan titik fisik yang sama dengan koordinat (u,v) pada *IRGB* . Berikut adalah implementasi kode untuk inisialisasi Depth Stream dengan Registrasi:

```
# Melanjutkan dari inisialisasi OpenNI pada Bab V
# dev = openni2.Device.open_any()

# 1. Membuat Objek Depth Stream
depth_stream = dev.create_depth_stream()

# 2. Konfigurasi Mode Video: Format 1mm, Resolusi 640x480, 30fps
depth_video_mode =
openni2.VideoMode(pixelFormat=openni2.PIXEL_FORMAT_DEPTH_1_MM,
                    resolutionX=640,
                    resolutionY=480,
                    fps=30)
depth_stream.set_video_mode(depth_video_mode)

# 3. Aktifkan Registrasi Spasial (Mengkompensasi Parallax)
# Meng-align Depth Map ke perspektif RGB Camera
dev.set_image_registration_mode(openni2.IMAGE_REGISTRATION_DEPTH_TO_COLOR)

# 4. Memulai Transmisi Data
depth_stream.start()
print("Depth Stream aktif: Matriks I_D(480, 640) 16-bit siap dibaca.")
```

6.3. Pembacaan Buffer dan Normalisasi Visualisasi

Seperti pada RGB stream, data depth diekstraksi dari *ring buffer* menggunakan metode `read_frame()`. Namun, karena kedalaman bitnya adalah 16-bit, fungsi *getter*

buffer yang digunakan adalah `get_buffer_as_uint16()`, bukan `uint8()`. Konversi ke array NumPy menghasilkan matriks 2D bertipe data `np.uint16`. Kesalahan umum programmer adalah menggunakan `uint8` pada `depth`; ini akan memotong (*truncate*) 8-bit tertinggi, mengubah nilai jarak $> 255\text{mm}$ menjadi *overflow* yang menghasilkan data noise yang tidak bermakna. Tantangan utama dalam Depth Stream adalah visualisasi untuk verifikasi manusia. Jika matriks *ID* 16-bit ditampilkan langsung menggunakan `cv2.imshow()`, fungsi internal OpenCV akan mencoba memetakan rentang $[0, 65535]$ ke skala kecerahan monitor $[0, 255]$. Karena jangkauan operasional Kinect hanya pada rentang $[800, 4000]$, peta warna yang dihasilkan akan sangat gelap dan kontrasnya minim (objek di 800mm dan 4000mm akan terlihat sama hitamnya bagi mata). Untuk mengatasi ini, kita harus menerapkan Normalisasi Linear terhadap rentang jarak operasional yang valid. Secara matematis, normalisasi linear memetakan subset rentang dinamis $[D_{min}, D_{max}]$ ke rentang tampilan penuh $[0, 255]$. Persamaan normalisasi piksel demi piksel adalah:

$$I_{norm}(u, v) = 255 \times \frac{I_d(u, v) - D_{min}}{D_{max} - D_{min}}$$

Di mana D_{min} diatur ke 800 mm (jarak terdekat) dan D_{max} diatur ke 4000 mm (jarak terjauh). Dengan persamaan ini, objek yang berada di 800mm akan memiliki kecerahan 255 (putih bersih), dan objek di 4000mm akan memiliki kecerahan 0 (hitam pekat), memaksimalkan distribusi histogram kontras visual. OpenCV mengimplementasikan persamaan ini secara teroptimasi melalui fungsi `cv2.normalize(src, dst, alpha, beta, norm_type)`, di mana **alpha=0**, **beta=255**, dan **NORM_MINMAX** menjadi parameter konversinya. Perlu dicatat bahwa piksel *invalid* ($ID=0$) harus ditangani secara khusus agar tidak dihitung dalam statistik normalisasi, yang mana `cv2.normalize` secara cerdas akan mengabaikan nilai 0 jika kita masking matriksnya. Setelah normalisasi menghasilkan matriks 8-bit I_{norm} , matriks grayscale ini masih sulit diinterpretasikan secara cepat oleh operator robot. Untuk meningkatkan persepsi visual, kita mengaplikasikan Pseudocoloring melalui fungsi `cv2.applyColorMap()`. Pseudocoloring memetakan nilai intensitas grayscale tunggal ke dalam vektor warna 3-kanal (RGB) menggunakan tabel lookup (LUT) yang terdefinisi. Peta warna **COLORMAP_JET**, misalnya, memetakan jarak dekat ke warna merah (panas), menengah ke hijau/kuning, dan jarak jauh ke biru (dingin), menghasilkan visualisasi *thermal-like* yang sangat intuitif untuk mendeteksi rintangan.

```

while True:
    # 1. Pembacaan Frame dari Buffer
    depth_frame = depth_stream.read_frame()

    # 2. Ekstraksi Buffer 16-bit ke NumPy Matrix (Uint16)
    depth_data_raw = np.frombuffer(depth_frame.get_buffer_as_uint16(),
    dtype=np.uint16)

    # Reshape ke Tensor 2D: I_D(v, u)
    depth_array = depth_data_raw.reshape((480, 640))

    # 3. Normalisasi Linear Math:  $I_{norm} = 255 * (I_D - 800) / (4000 - 800)$ 
    # cv2.normalize menghitung ini secara otomatis
    depth_norm = cv2.normalize(depth_array, None, 0, 255,
    cv2.NORM_MINMAX, dtype=cv2.CV_8U)

    # 4. Pseudocoloring (JET Colormap untuk representasi
    thermal)
    depth_colormap = cv2.applyColorMap(depth_norm,
    cv2.COLORMAP_JET)

    # 5. Visualisasi Real-Time
    cv2.imshow('Depth Map Visualization (JET)', depth_colormap)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

depth_stream.stop()

```

6.4. Pemrosesan Lanjutan: Filtering Noise pada Depth Map

Meskipun revolusioner, data mentah ID dari sensor struktur cahaya (Structured Light) Kinect V1 sangat rentan terhadap artefak noise, terutama di sekitar tepi objek yang memiliki perbedaan kedalaman drastis, serta pada permukaan yang reflektansinya rendah. Noise yang paling merusak algoritma navigasi robot adalah Flying Pixels (piksel terbang) dan Hole/Zero Pixels (lubang data). *Flying pixels* terjadi ketika proyeksi pola bintang IR jatuh di perbatasan antara objek dekat dan latar belakang jauh. Pada batas ini, sensor IR membaca distorsi dari dua kedalaman yang berbeda secara simultan pada satu piksel, menghasilkan nilai jarak $ID(u,v)$ yang merupakan rata-rata tak bermakna di antara kedua objek, menciptakan "bayangan kabut" di tepi siluet objek. Untuk mengeliminasi *flying pixels*, filter Median secara matematis merupakan pendekatan yang paling efektif dibandingkan filter Gaussian (Mean). Filter Median mengganti nilai piksel sentral dengan nilai median (nilai tengah) dari himpunan piksel tetangga dalam jendela kernel $K_w \times K_h$. Secara formal, operasi median untuk jendela berukuran 3×3 (9 elemen) diformulasikan sebagai:

$$ID, median(u, v) = median\{ID(u + i, v + j) \mid (i, j) \in \{-1, 0, 1\}^2\}$$

Karena median mengambil nilai *tengah* dan bukan *rata-rata*, operasi ini secara matematis mempertahankan ketajaman tepi (edge preservation) dan menghapus nilai outlier ekstrem (yang merupakan flying pixels) tanpa mengaburkan geometri objek secara global. OpenCV menyediakan implementasi filter Median yang sangat cepat melalui **cv2.medianBlur(src, ksize)**, di mana **ksize** harus berupa bilangan ganjil (3, 5, atau 7).

Sementara itu, *Hole pixels* (piksel bernilai 0) merupakan masalah yang membutuhkan pendekatan interpolasi. Menghapus piksel 0 dengan filter median dapat menyebarkan nilai 0 ke tetangganya jika lubang tersebut cukup besar. Strategi *hole-filling* yang umum adalah melakukan Inpainting atau mengganti nilai 0 dengan nilai kedalaman terdekat yang valid (*nearest neighbor interpolation*). Pendekatan programmatic sederhana adalah membuat mask untuk piksel invalid, lalu mengganti nilai 0 dengan rata-rata dari piksel non-zero di sekitarnya. Pustaka OpenCV menyediakan fungsi *inpainting* tingkat lanjut (**cv2.inpaint**) yang menggunakan algoritma Navier-Stokes atau Fast Marching Method (FFM) untuk mengisi lubang secara matematis dengan mempertahankan kontur gradient permukaan kedalaman, sangat ideal untuk membangun *Point Cloud* yang rapat dan mulus.

```

# --- Lanjutan di dalam While Loop sebelum visualisasi ---
    # 1. Filtering Noise: Median Blur untuk menghilangkan Flying
    Pixels

    # Kernel size 5x5 cukup efektif menghapus outlier tanpa
    merusak geometri besar
    depth_filtered = cv2.medianBlur(depth_array, ksize=5)
    # 2. Hole Filling (Inpainting) untuk piksel Invalid (I_D ==
    0)
    # Membuat mask untuk lokasi piksel yang bernilai 0 (No Data)
    holes_mask = np.where(depth_filtered == 0, 255,
    0).astype(np.uint8)

    # Menggunakan Inpainting Navier-Stokes untuk mengisi lubang
    secara kontinu
    # radius=3 menentukan area tetangga untuk interpolasi
    gradient
    depth_inpainted = cv2.inpaint(depth_filtered, holes_mask,
    inpaintRadius=3,
                                flags=cv2.INPAINT_NS)

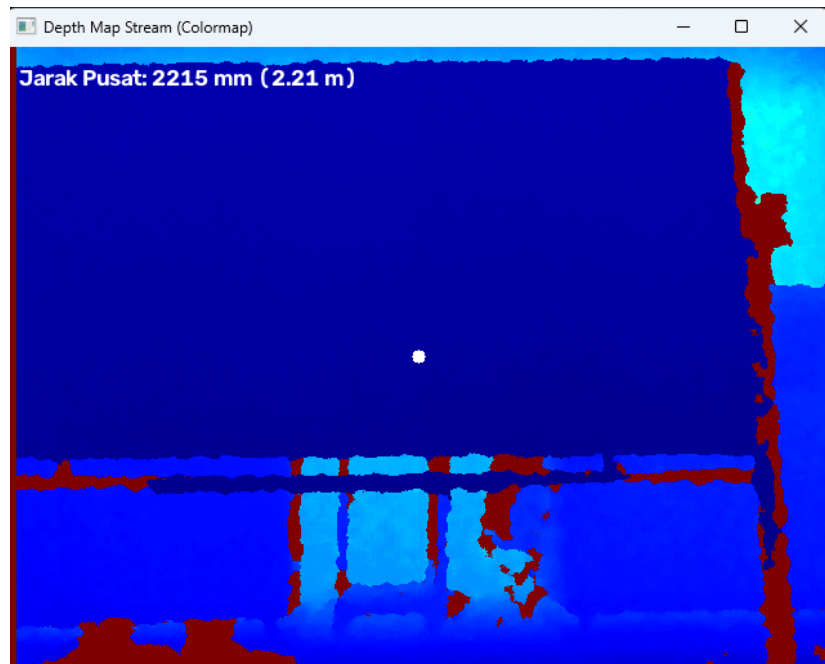
    # Setelah noise di-filter dan lubang di-inpaint, lakukan
    Normalisasi & Colormap
    depth_norm = cv2.normalize(depth_inpainted, None, 0, 255,
    cv2.NORM_MINMAX, dtype=cv2.CV_8U)

```



```
depth_colormap = cv2.applyColorMap(depth_norm,  
cv2.COLORMAP_JET)  
cv2.imshow('Filtered Depth Map', depth_colormap)
```

Berikut ini merupakan percobaan pada bab 6 sebagai berikut ini yang ditunjukkan pada Gambar 6.1.



Gambar 6.1. Output Program Bab 6

BAB VII

MENGAKSES SKELETON IMAGE

7.1. Arsitektur dan Representasi Matematis Data Skeleton

Evolusi paling revolusioner yang ditawarkan oleh sensor Kinect bukanlah sekadar kemampuan melihat kedalaman lingkungan, tetapi kemampuan untuk menginterpretasikan data kedalaman tersebut menjadi pemahaman anatomi manusia melalui *Skeleton Tracking*. Berbeda fundamental dengan citra RGB IRGB yang merupakan tensor padat (*dense tensor*) 3 dimensi, atau Depth ID yang merupakan matriks padat 2 dimensi, data Skeleton S adalah kumpulan titik sparse (jarang) yang merepresentasikan sendi-sendi (*joints*) utama tubuh manusia dalam ruang koordinat 3D. Secara matematis, sendi ke- j dari kerangka manusia direpresentasikan sebagai sebuah vektor posisi 3D dalam ruang koordinat kamera (*Camera Space*):

$$J_j = \begin{bmatrix} x_j \\ y_j \\ z_j \end{bmatrix} \in R^3$$

Di mana sumbu X merepresentasikan posisi horizontal (positif ke kanan sensor), sumbu Y merepresentasikan posisi vertikal (positif ke atas, berlawanan dengan arah gravitasi), dan sumbu Z merepresentasikan jarak kedalaman (positif menjauhi sensor). Keseluruhan kerangka tubuh satu orang pengguna direpresentasikan sebagai himpunan terurut dari N sendi, membentuk struktur data Kinematic Tree (Pohon Kinematika):

$$S = \{J_1, J_2, \dots, J_N\} \text{ di mana } N \in \{15, 20, 25\}$$

Nilai N bergantung pada versi perangkat dan *middleware* yang digunakan; Kinect V1 dengan NITE mendukung hingga 15 sendi, sedangkan Kinect V2 mendukung 25 sendi. Pada pohon kinematika, sendi tidak berdiri sendiri secara independen, melainkan terhubung oleh *bones* (tulang virtual) yang membentuk hierarki parent-child. Misalnya, sendi siku kiri (JL_Elbow) secara kinematika adalah *child* dari sendi bahu kiri ($JL_Shoulder$). Pemahaman hierarki ini sangat esensial dalam robotika untuk menghitung rotasi relatif (*local rotation*) antar sendi, bukan sekadar posisi absolut global. Selain vektor posisi, setiap sendi J_j memiliki atribut tambahan yang disebut Tracking State, dinotasikan sebagai σ_j . Atribut $\sigma_j \in \{\text{TRACKED}, \text{INFERRED}, \text{UNTRACKED}\}$

menandakan tingkat kepastian algoritma terhadap posisi sendi tersebut. Status **TRACKED** menandakan posisi J_j dihitung langsung dari data piksel depth yang terdeteksi jelas (probabilitas akurat tinggi). Status **INFERRED** menandakan sendi terhalang objek (*occlusion*, misal tangan di belakang punggung), sehingga posisinya diestimasi menggunakan model kinematika *forward kinematics* dari sendi *parent*-nya (probabilitas akurat sedang). Status **UNTRACKED** menandakan data sendi tidak tersedia. Dalam pemrograman kontrol robot, logika *decision-making* harus secara ketat memfilter perintah gerak berdasarkan status σ_j ; mengandalkan sendi dengan status **INFERRED** untuk kendali presisi dapat mengakibatkan robot bergerak erratic (tersentak-sentak) akibat data *noise*.

7.2. Inisialisasi Pelacakan Kerangka melalui NITE Middleware

Pemrosesan data Depth ID menjadi data Skeleton S tidak dilakukan oleh sensor fisik maupun driver dasar OpenNI, melainkan oleh *middleware* analitik yang berjalan di atasnya, yang dikenal sebagai NITE (Natural Interaction Technology for End-User). NITE mengimplementasikan algoritma *machine learning* (khususnya *decision tree* dan *randomized forest*) yang memindai matriks ID untuk menemukan pola siluet yang probability-nya mendekati bentuk anatomi bipedal (manusia). Oleh karena itu, sebelum *streaming* dimulai, NITE harus diinisialisasi secara eksplisit di dalam kode program melalui fungsi **nite2.initialize()**. Proses pelacakan kerangka terbagi menjadi dua tahap: User Detection dan Skeleton Calibration. Pada tahap *detection*, NITE mengidentifikasi massa piksel depth yang bergerak dan membentuk *bounding box* 3D di sekitar siluet tersebut, memberikan status **NEW**. Namun, koordinat sendi J_j belum diekstraksi. Untuk mengaktifkan ekstraksi S, pengguna harus melakukan Calibration Pose (Pose Kalibrasi). Pada Kinect V1 dengan NITE, pose standar yang wajib dilakukan adalah "Psi Pose"—di mana pengguna berdiri tegak dan mengangkat kedua tangan ke samping sejajar bahu dengan siku ditekuk 90 derajat.

Secara matematis, pose kalibrasi memberikan *initial constraint* yang memaksa model probabilistik NITE untuk mengidentifikasi sendi bahu, siku, dan tangan sebagai titik anchor (**Janchor**). Dari titik anchor ini, algoritma *gradient descent* kemudian menghitung letak sendi torsop (kepala, leher, pinggul) secara rekursif. Tanpa pose kalibrasi ini, ruang pencarian (*search space*) model kinematika terlalu luas, menyebabkan

computational explosion dan algoritma gagal menemukan solusi sendi yang valid. Setelah kalibrasi berhasil, status pengguna berubah menjadi **TRACKED**, dan sensor mulai mengirimkan vektor $\mathbf{J_j}$ secara *real-time* meskipun pengguna kemudian bergerak bebas keluar dari pose Psi. Dengan dirilisnya Kinect V2 dan SDK modern, *calibration pose* ini dihilangkan karena model *randomized forest* generasi baru mampu menginferensi anatomi dari pose apapun (*zero-calibration*), namun pada ekosistem OpenNI/NITE V1, kalibrasi tetap menjadi prosedur protokol yang tak terhindarkan. Berikut adalah implementasi kode untuk inisialisasi NITE dan *User Tracker*:

```
import numpy as np
import cv2
from openni import openni2
from openni import nite2

# Inisialisasi OpenNI dan NITE Middleware
openni2.initialize()
nite2.initialize()

dev = openni2.Device.open_any()
user_tracker = nite2.UserTracker(dev)

# Membuat Depth Stream (NITE membutuhkan Depth sebagai input)
depth_stream = dev.create_depth_stream()
depth_stream.start()

print("NITE UserTracker aktif. Silakan berdiri di depan sensor
untuk deteksi.")
```

7.3. Pemetaan Koordinat 3D ke Bidang Citra 2D (Perspektif Proyeksi)

Setelah data S berhasil diekstraksi oleh NITE, kita mendapatkan kumpulan vektor

$Jj \in R^3$ dalam satuan milimeter. Meskipun data koordinat 3D ini sangat sempurna untuk kalkulasi matematika robotika (seperti menghitung jarak lengungan tangan atau sudut rotasi sendi), kita tidak dapat langsung menggambar titik-titik tersebut di atas citra RGB *IRGB* atau Depth *ID*. Layar monitor dan matriks piksel beroperasi dalam ruang 2D $\mathbf{pj} = (uj, vj)$. Oleh karena itu, kita harus melakukan Proyeksi Perspektif (*Perspective Projection*) yang memetakan titik dari dunia 3D ke bidang image plane 2D.

Secara matematis, pemetaan ini menggunakan model kamera *pinhole*, yang mensimulasikan bagaimana cahaya melewati lensa kamera dan terproyeksi ke sensor CMOS/film. Transformasi ini secara fundamental bergantung pada parameter intrinsik kamera (*Intrinsic Parameters*): focal length (f_x, f_y) dalam satuan piksel, dan optical center (c_x, c_y). Persamaan proyeksi perspektif standar untuk memetakan vektor $Jj = (Xj, Yj, Zj)$ ke piksel $\mathbf{pj} = (uj, vj)$ adalah:

$$u_j = f_x \left(\frac{x_j}{z_j} \right) + c_x$$

$$v_j = f_y \left(\frac{y_j}{z_j} \right) + c_y$$

Perhatikan bahwa operasi ini memerlukan pembagian dengan Zj (kedalaman). Ini adalah sifat fundamental dari proyeksi perspektif: objek yang jauh (Z besar) akan memiliki rasio ZX yang kecil, sehingga tampak kecil di layar (perspektif linier), sedangkan objek dekat akan diproyeksikan lebih besar. Dalam terminologi geometri proyektif, transformasi ini direpresentasikan secara elegan menggunakan matriks homogen 3×4 :

$$\begin{bmatrix} w & U_j \\ w & V_j \\ w & w \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_j \\ Y_j \\ Z_j \\ 1 \end{bmatrix}$$

Di mana $w = Zj$. Koordinat piksel final diturunkan dengan operasi

dehomogenization (pembagian dengan w): $uj = \frac{w \cdot u_j}{w}$. Meskipun persamaan ini dapat kita implementasikan secara manual menggunakan NumPy jika kita mengetahui nilai f_x, c_x dari hasil kalibrasi, pustaka OpenNI menyediakan fungsi C++ tingkat rendah yang sangat teroptimasi untuk melakukan kalkulasi perspektif ini secara akurat.

7.4. Visualisasi dan Penggambaran Hierarki Pohon Kinematika

Visualisasi kerangka di atas citra bukan sekadar menetapkan titik-titik acak, melainkan merekonstruksi visualisasi anatomi yang terhubung. Secara matematis, kita menggambar Graf Terhubung (*Connected Graph*) berdasarkan definisi *Kinematic Tree* $T=(V,E)$. Himpunan verteks (titik) V merepresentasikan sendi $\mathbf{p}_j$, dan himpunan edges (garis) E merepresentasikan tulang penghubung antar sendi *parent* dan *child*, misalnya $E_k = (\mathbf{p}_{Head}, \mathbf{p}_{Neck})$. Dalam implementasi OpenCV, verteks dirender menggunakan **cv2.circle(img, center, radius, color, thickness)**, yang secara matematis menggambar lingkaran $C(\mathbf{p}_j, r)$ pada pusat piksel $\mathbf{p}_j$ dengan radius r . Sementara edges dirender menggunakan **cv2.line(img, pt1, pt2, color, thickness)**, yang menggambar ruas garis linear $L(\mathbf{p}_a, \mathbf{p}_b)$ di antara dua verteks. Pemilihan warna (*color mapping*) untuk setiap sendi dapat diatur secara diferensial; misalnya, warna merah (0,0,255) untuk sendi *tracked* ($\sigma=\text{TRACKED}$) dan warna kuning (0,255,255) untuk sendi *inferred* ($\sigma=\text{INFERRED}$), memberikan umpan balik visual langsung kepada operator manusia mengenai tingkat kepercayaan sensor. Berikut adalah implementasi kode lengkap untuk deteksi pengguna, kalibrasi, konversi proyeksi 3D-2D, dan visualisasi graf kinematika:

```
# Konstanta Warna BGR untuk Visualisasi
COLOR_TRACKED = (0, 0, 255)    # Merah untuk sendi TRACKED
COLOR_INFERRED = (0, 255, 255) # Kuning untuk sendi INFERRED
COLOR_BONE = (255, 255, 0)     # Cyan untuk garis tulang

# Definisi Hierarki Kinematika (Kinematic Tree Edges)
# Format: (Parent_Joint, Child_Joint)
KINEMATIC_TREE_EDGES = [
    (nite2.JointType.HEAD, nite2.JointType.NECK),
```

```

        (nite2.JointType.NECK, nite2.JointType.LEFT_SHOULDER),
        (nite2.JointType.LEFT_SHOULDER,
nite2.JointType.LEFT_ELBOW),
        (nite2.JointType.LEFT_ELBOW, nite2.JointType.LEFT_HAND),
        # ... (Tambahkan edge untuk sisi kanan dan kaki bawah sesuai
kebutuhan)
    ]

while True:
    # 1. Baca Depth Frame (untuk visualisasi background)
    depth_frame = depth_stream.read_frame()
    depth_data =
np.frombuffer(depth_frame.get_buffer_as_uint16(),
dtype=np.uint16)
    depth_array = depth_data.reshape((480, 640))

    # Normalisasi Depth untuk layar
    depth_norm = cv2.normalize(depth_array, None, 0, 255,
cv2.NORM_MINMAX, dtype=cv2.CV_8U)
    depth_colormap = cv2.applyColorMap(depth_norm,
cv2.COLORMAP_JET)

    # 2. Baca User Tracker Frame (untuk data Skeleton)
    user_frame = user_tracker.read_frame()

    # 3. Iterasi Setiap Pengguna yang Terdeteksi
    for user in user_frame.users:
        # Tahap Deteksi & Kalibrasi
        if user.is_new():
            print(f"Pengguna {user.id} terdeteksi (NEW).
Meminta Calibration Pose (Psi)...")

```

```

        user_tracker.start_skeleton_tracking(user.id)
        # Tahap Ekstraksi Skeleton (Hanya jika status TRACKED)
        elif user.skeleton.state ==
nite2.SkeletonState.TRACKED:
            skeleton = user.skeleton
            # Dictionary untuk menyimpan piksel 2D proyeksi
            joint_pixels_2d = {}
            # Ekstraksi Seluruh Sendi yang Terdefinisi
            for joint_type in nite2.JointType:
                joint = skeleton.joints[joint_type]

                # Filter: Hanya proses sendi yang posisiZ > 0
(valid)

                if joint.position.z > 0:
                    # Konversi Proyeksi Perspektif 3D -> 2D
                    # Math:  $u = f_x \cdot (X/Z) + c_x$  ;  $v =$ 
f_y*(Y/Z)+c_y
                    pixel_2d =
user_tracker.convert_joint_coordinates_to_depth(
                        joint.position.x, joint.position.y,
joint.position.z
                    )
                    joint_pixels_2d[joint_type] = (pixel_2d.x,
pixel_2d.y)

                    # Render Verteks (Lingkaran) dengan Warna
berdasarkan Status
                    color = COLOR_TRACKED if
joint.tracking_state == nite2.JointTrackingState.TRACKED else
COLOR_INFERRED
                    cv2.circle(depth_colormap, (pixel_2d.x,
pixel_2d.y), 8, color, -1)

```

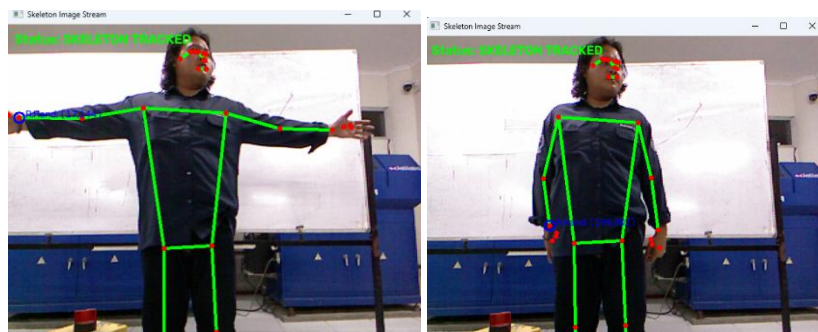


```

# Render Edges (Garis Tulang Kinematika)
for parent_joint, child_joint in
KINEMATIC_TREE_EDGES:
    if parent_joint in joint_pixels_2d and
child_joint in joint_pixels_2d:
        pt1 = joint_pixels_2d[parent_joint]
        pt2 = joint_pixels_2d[child_joint]
        # Menggambar ruas garis linear L(pt1, pt2)
        cv2.line(depth_colormap, pt1, pt2,
COLOR_BONE, 3)
# 4. Visualisasi Real-Time
cv2.imshow('Skeleton Tracking Visualization',
depth_colormap)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
# Cleanup Sumber Daya
depth_stream.stop()
openni2.unload()
nite2.unload()
cv2.destroyAllWindows()

```

Berikut Ini merupakan output pada program Bab 7 yang ditujukan pada Gambar 7.1 sebagai berikut



Gambar 7.1. Output Bab 7

BAB VIII

MENGAKSES INFRARED IMAGE

8.1. Arsitektur Fisis dan Representasi Matematis Citra Inframerah

Di balik kemampuan Kinect dalam menghasilkan peta kedalaman ID yang presisi, terdapat sebuah fenomena fisis fundamental yang sering diabaikan: proyeksi cahaya terstruktur (Structured Light). Citra inframerah (IR) merupakan artefak visual langsung dari proses ini. Berbeda dengan citra RGB yang menangkap pantulan foton dari spektrum tampak (380–700 nm), dan citra Depth yang merupakan hasil kalkulasi algoritma kompleks, citra IR IIR adalah rekaman mentah (raw data) intensitas cahaya inframerah dekat (Near-Infrared / NIR, sekitar 830 nm) yang dipantulkan oleh lingkungan ke sensor monokrom Kinect. Pada arsitektur Kinect V1, proyektor IR internal memancarkan pola bintik (speckle pattern) yang deterministik P_{ref} ke ruangan. Ketika pola ini mengenai permukaan objek 3D, geometri kontur permukaan akan mendistorsi pola bintik tersebut, menghasilkan pola observasi $P_{obs}(u,v)$. Kamera IR bertugas menangkap P_{obs} ini sebelum distorsi dipetakan oleh prosesor PrimeSense menjadi Disparity Map I_{disp} . Oleh karena itu, secara matematis, citra IR direpresentasikan sebagai matriks 2 dimensi grayscale yang merekam *irradiance* (kepadatan fluks energi foton NIR) pada setiap piksel:

$$IIR \in ZH \times W, \text{ di mana } IIR(u, v) \in [0, 2^k - 1]$$

Parameter k merepresentasikan bit-depth sensor IR. Karena sensor CMOS untuk IR pada Kinect dirancang untuk sensitivitas tinggi di kondisi redup, data mentah ini seringkali dikodekan dalam 10-bit atau 16-bit (packed into 16-bit unsigned integer buffer) untuk menangkap rentang dinamis (*dynamic range*) pantulan cahaya yang luas, dari bahan yang sangat reflektif hingga absorptif. Intensitas piksel $IIR(u,v)$ secara fisis berbanding lurus dengan energi pantulan yang mencapai sensor, yang secara umum mengikuti hukum penurunan intensitas cahaya invers-kuadrat dan properti reflektansi material ρ :

$$I_{IR}(u, v) \propto \frac{\rho(u, v) \cdot P_{emit}}{z(u, v)^2}$$

Di mana P_{emit} adalah daya pancar proyektor IR, $\rho(u,v)$ adalah *albedo* (koefisien reflektansi) material objek pada koordinat dunia 3D yang terproyeksi ke piksel (u,v) , dan

$Z(u,v)$ adalah jarak objek ke sensor. Persamaan ini menjelaskan mengapa objek jauh (Z besar) memiliki titik IR yang redup, dan bahan hitam ($\rho \approx 0$) menghasilkan *void* (piksel gelap nilai 0) karena cahaya NIR diserap dan tidak dipantulkan kembali. Visualisasi pola *Pobs* ini memberikan wawasan eksklusif bagi insinyur robotika untuk menganalisis mengapa sensor Depth gagal membaca objek transparan atau permukaan *specular* (seperti cermin yang memantulkan pola IR ke luar bidang sensor).

8.2. Inisialisasi IR Stream dan Konstrain Bandwidth Hardware

Mengakses IR Stream dalam ekosistem OpenNI memerlukan pemahaman mendalam tentang arsitektur *hardware abstraction* Kinect V1. Sensor CMOS IR dan prosesor internal yang menghasilkan Depth Map berbagi jalur *data bus* dan *frame grabber* yang sama. Akibatnya, terjadi Hardware Mutual Exclusion (Eksklusi Mutual Perangkat Keras); sensor tidak dapat secara simultan memproduksi citra Depth ID 16-bit dan citra IR mentah *IIR* 16-bit pada *frame rate* penuh 30 fps tanpa mengorbankan performa.

Apabila kedua stream (Depth dan IR) diaktifkan secara paralel, *throughput* data mentah yang harus melewati bus USB 2.0 akan melebihi kapasitas alokasi *isochronous transfer* (sekitar 24 MB/s efektif per endpoint). Konsekuensi fisis dari *bandwidth starvation* ini adalah Frame Drop (pemilihan frame yang terputus) pada salah satu atau kedua stream, atau penurunan *frame rate* drastis menjadi 10-15 fps, yang menghancurkan validitas waktu (*timestamping*) pada algoritma SLAM atau visual servoing. Oleh karena itu, protokol praktik terbaik (*best practice*) dalam pemrograman robotika Kinect V1 adalah: jika tujuan utama adalah memvisualisasikan pola mentah untuk analisis reflektansi, Depth Stream harus dihentikan (*stop*) terlebih dahulu sebelum IR Stream di-*start*. Inisialisasi dimulai dengan **dev.create_ir_stream()**. Konfigurasi *video mode* pada IR stream mengatur resolusi spasial 640×480 dan format piksel **PIXEL_FORMAT_GRAY16** (atau **PIXEL_FORMAT_SHIFT_9_3** pada beberapa firmware V1, yang merupakan format kompresi 10-bit ke dalam ruang 16-bit). Karena IR stream tidak memerlukan koreksi perspektif dari kamera RGB (seperti *registration* pada Depth), tidak ada metode **set_image_registration_mode** yang dipanggil.

Berikut adalah implementasi kode untuk transisi eksklusif dari Depth Stream ke IR Stream:

```

import numpy as np
import cv2
from openni import openni2

openni2.initialize()
dev = openni2.Device.open_any()

# Asumsikan Depth Stream sebelumnya sedang berjalan untuk
navigasi
depth_stream = dev.create_depth_stream()
depth_stream.start()
print("Depth Stream aktif...")

# --- TRANSISI KE IR STREAM ---
# 1. Hentikan Depth Stream (Hardware Mutual Exclusion)
print("Menghentikan Depth Stream untuk mengalokasikan bandwidth
IR...")
depth_stream.stop() # Hanya stop, tidak destroy object agar bisa
di-restart

# 2. Membuat dan Mengkonfigurasi IR Stream
ir_stream = dev.create_ir_stream()

# Mengatur mode: 16-bit Grayscale, 640x480, 30fps
ir_video_mode =
openni2.VideoMode(pixelFormat=openni2.PIXEL_FORMAT_GRAY16,
                    resolutionX=640,
                    resolutionY=480,
                    fps=30)
ir_stream.set_video_mode(ir_video_mode)

```

```
# 3. Memulai Transmisi Data IR
ir_stream.start()
print("IR Stream aktif: Matriks I_IR(480, 640) 16-bit siap
dibaca.")
```

8.3. Pembacaan Buffer dan Peningkatan Kontras melalui Histogram Equalization

Pembacaan frame IR mengikuti pola yang sama dengan Depth, menggunakan `read_frame()` dan mengekstraksi buffer `get_buffer_as_uint16()`, yang kemudian di-*reshape* ke matriks NumPy 2D (480,640). Namun, tantangan utama dalam visualisasi IR bukanlah rentang nilai jarak seperti pada Depth, melainkan Skewed Dynamic Range. Pola bintik (speckle pattern) proyektor IR *Pobs* memiliki distribusi histogram yang sangat tidak merata. Area antara bintik (inter-dot area) memiliki intensitas pantulan NIR yang sangat rendah, mendekati 0 (background gelap), sedangkan titik bintik sendiri memiliki intensitas yang sangat tinggi terkonsentrasi pada nilai-nilai atas. Apabila kita menerapkan normalisasi linear standar $Inorm = 255 \times \frac{max - min}{IIR - min}$, kontras pada area antara-bintik (yang seringkali menyimpan informasi tekstur halus material) akan hilang sepenuhnya, menghasilkan citra yang tampak seperti "bintik-bintik putih di atas kanvas hitam mutlak" tanpa detail kontur objek. Solusi matematis untuk redistribusi dinamis ini adalah Histogram Equalization (Ekualisasi Histogram). Tujuan dari ekualisasi adalah mengtransformasi distribusi probabilitas intensitas piksel $pI(i)$ yang terkonsentrasi (skewed) menjadi distribusi uniform $peq(j)$ yang merata sepanjang rentang [0,255]

Sehingga memaksimalkan entropy informasi visual citra. Secara matematis, proses ini menghitung Cumulative Distribution Function (CDF) dari histogram citra. Didefinisikan $h(i)$ sebagai frekuensi piksel dengan intensitas i , dan N sebagai total piksel. Probabilitas kemunculan intensitas adalah:

$$pI(i) = \frac{h(i)}{N} \text{ di mana } i \in [0, Lmax]$$

CDF dinotasikan sebagai $c(i)$, merupakan integral (sum kumulatif) dari probabilitas:

$$c(i) = \sum_{j=0}^i pI(j)$$

Fungsi transformasi ekualisasi $T(i)$ memetakan intensitas input I ke intensitas output j baru berdasarkan normalisasi CDF terhadap rentang bit-depth target $Lmax=255$

$j=T(i)=\text{round}(c(i)\times 255)$ Operasi $T(IIR(u,v))$

Diterapkan ke seluruh piksel. Hasil transformasi ini memaksa piksel background gelap yang awalnya terkompresi di rentang $[0,30]$ untuk "diregangkan" ke rentang yang lebih luas, dan menekan piksel bintik terang agar tidak saturasi penuh, mengungkapkan detail tekstur mikro lingkungan. OpenCV mengimplementasikan algoritma CDF ini secara teroptimasi dalam fungsi **cv2.equalizeHist()**. Perlu dicatat bahwa **equalizeHist** hanya menerima input 8-bit (**CV_8UC1**), sehingga matriks 16-bit *IIR* harus dinormalisasi linear ke 8-bit terlebih dahulu sebelum di-ekualisasi. Berikut adalah implementasi kode untuk pembacaan, normalisasi, ekualisasi, dan visualisasi IR:

```
while True:
    # 1. Pembacaan Frame IR
    ir_frame = ir_stream.read_frame()

    # 2. Ekstraksi Buffer 16-bit
    ir_data_raw = np.frombuffer(ir_frame.get_buffer_as_uint16(),
                                dtype=np.uint16)
    ir_array = ir_data_raw.reshape((480, 640))

    # 3. Normalisasi Linear Preliminary: Konversi 16-bit ke 8-bit
    # Rentang data IR V1 umumnya 0 - 8000, kita normalisasi ke [0, 255]
    ir_norm = cv2.normalize(ir_array, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)

    # 4. Histogram Equalization Math:  $j = \text{round}(c(i) * 255)$ 
    # Meratakan distribusi probabilitas  $p_I(i)$  yang skewed
    ir_eq = cv2.equalizeHist(ir_norm)

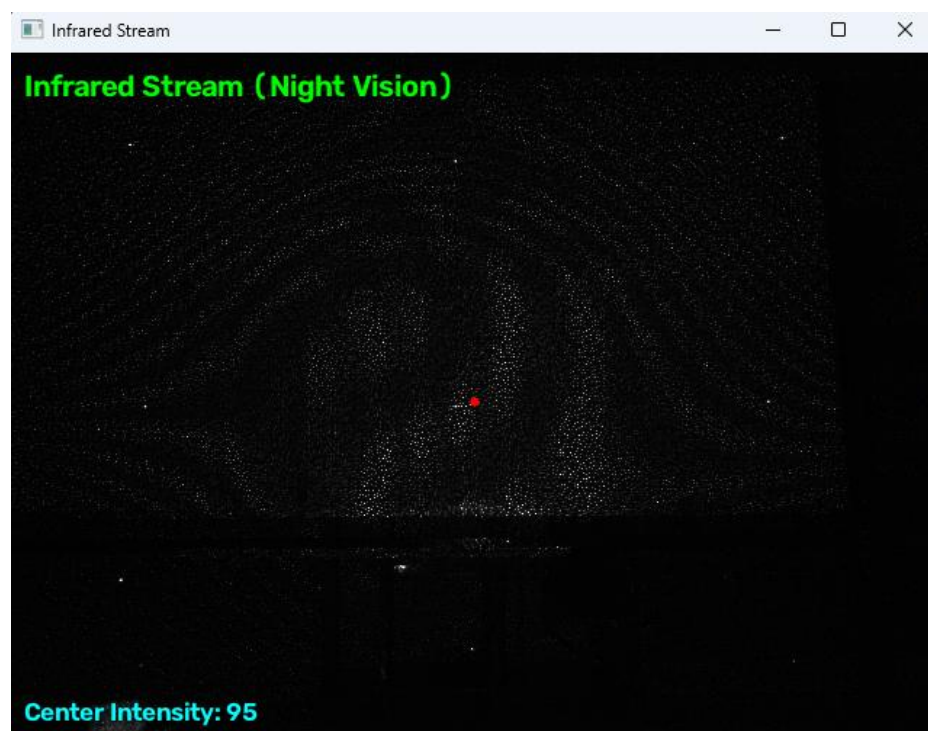
    # 5. Peningkatan Visual Tambahan: Gaussian Blur untuk mengurangi noise sensor
```

```

# Kernel 3x3 sangat efektif merapikan pola speckle noise
pada sensor CMOS IR
ir_smooth = cv2.GaussianBlur(ir_eq, (3, 3), 0)
# 6. Visualisasi Real-Time
cv2.imshow('Raw IR Normalized', ir_norm)      # Tanpa
Equalization (Detail hilang)
cv2.imshow('IR Equalized & Smoothed', ir_smooth) # Dengan
Equalization (Detail muncul)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
# Cleanup Sumber Daya
ir_stream.stop()
openni2.unload()
cv2.destroyAllWindows()

```

Berikut dibawah ini merupakan output percobaan yang ditujukan pada Gambar 8.1.



Gambar 8.1. Output Bab 8

BAB IX

MENGESTIMASI JARAK DENGAN KINECT

9.1. Prinsip Pengukuran Jarak Berdasarkan Agregasi Statistik Depth Data

Tujuan utama dari integrasi sensor 3D pada platform robot otonom adalah kemampuan untuk memahami geometri ruang kerja (*workspace geometry*) untuk menghindari tabrakan (*collision avoidance*) atau mengikuti target (*target following*). Meskipun sensor Kinect menyediakan matriks kedalaman ID yang berisi informasi jarak absolut $Z(u,v)$ dalam milimeter pada setiap piksel, mengestimasi "jarak objek" D_{obj} ke robot bukanlah proses trivial yang hanya membaca satu piksel tunggal. Karena objek di dunia nyata memiliki luas permukaan fisik yang menutupi banyak piksel pada bidang citra, dan data depth mentah sarat dengan artefak noise (seperti *flying pixels* dan *hole pixels* yang dibahas pada Bab VI), pembacaan piksel tunggal $Z(u,v)$ akan menghasilkan error yang tinggi dan tidak representatif. Oleh karena itu, estimasi jarak harus dilakukan melalui Agregasi Statistik di atas sebuah Region of Interest (ROI), dinotasikan sebagai himpunan piksel R . ROI R adalah subset dari matriks ID yang secara spasial berada di dalam kotak pembatas (*bounding box*) objek yang telah berhasil diidentifikasi pada citra RGB. Secara matematis, ROI didefinisikan sebagai:

$$R = \{(u, v) \mid u_{min} \leq u \leq u_{max}, v_{min} \leq v \leq v_{max}\}$$

Di mana (u_{min}, v_{min}) adalah koordinat piksel sudut atas-kiri, dan (u_{max}, v_{max}) adalah sudut bawah-kanan *bounding box*. Karena piksel *invalid* (lubang data) memiliki nilai 0, kita harus mendefinisikan subset valid R_{valid} yang hanya berisi piksel dengan data kedalaman yang tercatat akurat:

$$R_{valid} = \{(u, v) \in R \mid D(u, v) > 0\}$$

Pendekatan agregasi yang paling umum digunakan adalah Mean Distance (Jarak Rata-rata), dihitung menggunakan operasi summation rata-rata aritmatika:

$$D_{mean} = \frac{1}{|R_{valid}|} \sum_{(u,v) \in R_{valid}} D(u,v)$$

Namun, D_{mean} sangat rentan terhadap distorsi dari *outlier* (piksel terbang yang memiliki nilai jarak ekstrem di luar kontur objek). Untuk aplikasi robotika di mana kesalahan estimasi jarak dapat berakibat fatal, pendekatan yang lebih robust secara statistik adalah Median Distance D_{median} . Median mengambil nilai tengah dari

himpunan data yang telah diurutkan, secara fundamental mengeliminasi pengaruh *outlier* ekstrem tanpa memerlukan filtering pre-processing yang kompleks. Secara algoritmik, OpenCV/NumPy menghitung median ini dengan sangat efisien menggunakan fungsi `np.median()`.

9.2. Implementasi Deteksi Objek Berbasis Warna untuk ROI Extraction

Sebelum kita dapat menghitung jarak pada ROI R, kita harus terlebih dahulu menemukan koordinat spasial objek di dalam bidang citra 2D. Dalam pemrosesan visi komputer tingkat dasar, metode yang paling *cost-effective* secara komputasi adalah Color-based Object Tracking. Deteksi dilakukan pada citra RGB *IRGB*, namun terdapat anomali matematis: representasi warna dalam ruang RGB sangat korelatif dengan intensitas cahaya (*luminance*). Perubahan pencahayaan ambient akan menggeser nilai piksel objek secara drastis, membuat *thresholding* (pemotongan batas) pada ruang RGB tidak stabil. Solusi matematisnya adalah melakukan transformasi ruang warna (*Color Space Transformation*) dari RGB ke HSV (Hue, Saturation, Value). Secara matematis, HSV memisahkan informasi chroma (warna dasar, diwakili oleh *Hue H*) dari luma (kecerahan, diwakili oleh *Value V*). *Hue* direpresentasikan sebagai sudut pada roda warna silinder $[0^\circ, 360^\circ]$, yang diskretkan ke $[0, 180]$ pada OpenCV 8-bit. Dengan pemisahan ini, objek merah akan selalu memiliki nilai *Hue* sekitar 160–180 atau 0–10, terlepas apakah objek tersebut terkena cahaya terang ($V=255$) atau redup ($V=50$).

Operasi segmentasi dilakukan dengan Thresholding Multidimensional, mengaplikasikan batas interval pada ketiga kanal HSV secara simultan. Piksel

(u, v) diklasifikasikan sebagai objek target O jika memenuhi kondisi logika konjungsi ($\wedge$):

$$(u, v) \in O \Leftrightarrow H(u, v) \in [Hmin, Hmax] \wedge S(u, v) \in [Smin, Smax] \wedge V(u, v) \in [Vmin, Vmax]$$

OpenCV mengimplementasikan operasi logika ini sebagai *bitwise operation* pada tensor, menghasilkan Binary Mask $M \in \{0, 255\}$ melalui fungsi `cv2.inRange()`. Setelah *mask* dibersihkan dari *noise* morfologis menggunakan operasi *Opening* (Erosi diikuti Dilasi) via `cv2.morphologyEx()`, kontur objek C diekstraksi menggunakan `cv2.findContours()`. Kontur terbesar (berdasarkan luas area `cv2.contourArea()`) diasumsikan sebagai target, dan *bounding box* (x, y, w, h) dihitung menggunakan `cv2.boundingRect()`. PENTING: Agar ROI R pada Depth Map *ID* merepresentasikan

titik fisik yang sama persis dengan *bounding box* pada RGB *IRGB* , Image Registration (Bab VI) *harus* telah diaktifkan sebelumnya. Berikut adalah implementasi kode untuk ekstraksi ROI berbasis warna:

```
import numpy as np
import cv2
from openni import openni2

# Inisialisasi (Registration ON!)
openni2.initialize()
dev = openni2.Device.open_any()
dev.set_image_registration_mode(openni2.IMAGE_REGISTRATION_DEPTH_TO_COLOR)

rgb_stream = dev.create_color_stream()
rgb_stream.start()
depth_stream = dev.create_depth_stream()
depth_stream.start()

# Definisi Batas HSV untuk Objek Berwarna Merah
# Hue Merah terbagi di dua ujung spectrum [0,10] dan [160,180]
lower_red_1 = np.array([0, 100, 100])
upper_red_1 = np.array([10, 255, 255])
lower_red_2 = np.array([160, 100, 100])
upper_red_2 = np.array([180, 255, 255])

while True:
    # 1. Pembacaan Frame Sinkron
    rgb_frame = rgb_stream.read_frame()
    depth_frame = depth_stream.read_frame()
```

```

    rgb_data = np.frombuffer(rgb_frame.get_buffer_as_uint8(),
dtype=np.uint8).reshape((480, 640, 3))
    bgr_array = cv2.cvtColor(rgb_data, cv2.COLOR_RGB2BGR)

    depth_data =
np.frombuffer(depth_frame.get_buffer_as_uint16(),
dtype=np.uint16).reshape((480, 640))

# 2. Transformasi Ruang Warna (RGB -> HSV)
hsv_array = cv2.cvtColor(bgr_array, cv2.COLOR_BGR2HSV)

# 3. Thresholding Multidimensional (Segmentasi Warna)
mask_1 = cv2.inRange(hsv_array, lower_red_1, upper_red_1)
mask_2 = cv2.inRange(hsv_array, lower_red_2, upper_red_2)
mask = cv2.add(mask_1, mask_2) # Union dari dua rentang Hue
merah

# 4. Morphological Opening (Menghapus noise kecil)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,
np.ones((5,5), np.uint8))

# 5. Extract Contours & Bounding Box
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

if len(contours) > 0:
    # Ambil kontur dengan area terbesar
    c = max(contours, key=cv2.contourArea)
    x, y, w, h = cv2.boundingRect(c)

```

```
# Gambar Bounding Box di RGB Image untuk Verifikasi Visual
cv2.rectangle(bgr_array, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

9.3. Ekstraksi Jarak Objek dari ROI pada Depth Map

Setelah koordinat *bounding box* (x, y, w, h) berhasil diekstraksi pada citra RGB, langkah selanjutnya adalah melakukan *slicing* (pemotongan) matriks pada citra Depth ID menggunakan koordinat yang sama. Sub-matriks hasil *slicing* ini merepresentasikan subset R yang berisi seluruh informasi kedalaman objek target. Proses *slicing* pada NumPy dilakukan dengan sintaks **depth_array[y:y+h, x:x+w]**, yang sangat efisien karena NumPy tidak menyalin data fisik ke memori baru (*view mechanism*), melainkan hanya membuat referensi memori ke sub-blok tensor asli. Setelah sub-matriks R diperoleh, langkah kritis adalah melakukan filtering terhadap *invalid pixels* ($D(u,v)=0$).

Jika piksel bernilai 0 disertakan dalam kalkulasi agregasi statistik, nilai rata-rata D_{mean} akan terkompresi secara artifisial ke bawah (objek terlihat lebih dekat dari yang seharusnya), atau jika median D_{median} dihitung dan lebih dari 50% piksel adalah 0, median akan mengembalikan nilai 0 mm, yang secara logika robotika akan memicu respon *emergency stop* yang palsu (ghost obstacle).

Filtering dilakukan menggunakan operasi *boolean masking* NumPy: **roi_depth[roi_depth > 0]**. Operasi ini menghasilkan array 1D flat yang hanya berisi nilai kedalaman fisik objek yang valid. Selanjutnya, kita menghitung Median Distance menggunakan **np.median()**. Median dipilih karena kemampuannya menahan *outlier*; jika suatu piksel pada tepi objek terdistorsi menjadi *flying pixel* dengan nilai 5000 mm (padahal objek berada di 1000 mm), median tidak akan terpengaruh, sedangkan rata-rata akan terdorong naik secara drastis, menyebabkan robot gagal menghindari rintangan yang sebenarnya sudah sangat dekat.

```
# 6. Slicing Depth Map berdasarkan Bounding Box ROI
roi_depth = depth_array[y:y+h, x:x+w]

# 7. Filtering Invalid Pixels (Z = 0)
```

```

        valid_depths = roi_depth[roi_depth > 0]

        # 8. Kalkulasi Jarak Objek menggunakan Agregasi
        Statistik
        if len(valid_depths) > 0:
            # D_median = median dari { D(u,v) | D(u,v) > 0 }
            distance_mm = np.median(valid_depths)

            # Konversi ke Meter untuk Logika Kontrol Robotika
            distance_m = distance_mm / 1000.0

            # Tampilkan estimasi jarak di layar Visualisasi
            text_display = f"Distance: {distance_m:.2f} m"
            cv2.putText(bgr_array, text_display, (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
255), 2)

            print(f"Objek terdeteksi di jarak: {distance_m:.2f}
meter")
        else:
            print("Objek terdeteksi, namun Depth Data invalid
(Z=0).")

```

9.4. Implementasi Estimasi Jarak untuk Navigasi: Kontrol Proporsional (P-Controller)

Estimasi numerik jarak *Dobj* (dalam meter) yang diperoleh dari proses agregasi median bukanlah data akhir yang hanya ditampilkan di layar, melainkan parameter masukan (*input variable*) untuk sistem kendali kinematika (*motion control system*) robot. Dalam desain navigasi otonom, skenario paling dasar adalah Obstacle Stopping

(Penghentian Otomatis) dan Target Following (Pengikutan Target). Pada skenario *Obstacle Stopping*, robot diprogram untuk berhenti maju jika jarak rintangan D_{obj} kurang dari batas aman operasional D_{safe} (misal 1.0 m). Logika *decision-making* sederhana menggunakan fungsi *step function* (kontrol *On-Off*):

$$v_x = \begin{cases} v_{target} & \text{jika } D_{obj} \geq D_{safe} \\ 0 & \text{jika } D_{obj} \leq D_{safe} \end{cases}$$

Namun, kontrol *On-Off* ini menyebabkan gerakan robot yang kasar dan tersentak (*jerkiness*). Saat jarak tepat berada di batas D_{safe} , robot akan bergantian maju dan berhenti secara cepat (*chattering*), memicu keausan mekanis pada motor roda dan ketidakstabilan inertial. Untuk menghasilkan gerakan yang mulus (*smooth deceleration*), insinyur kontrol mengimplementasikan Proportional Controller (P-Controller). P-Controller mengatur kecepatan linear v_x secara kontinu berdasarkan besar *error* $e(t)$ antara jarak aktual dengan jarak target:

$$e(t) = D_{obj}(t) - D_{target}(t)$$

Kecepatan output dihitung sebagai:

$$v_x(t) = K_p \cdot e(t) = K_p \cdot (D_{obj}(t) - D_{target})$$

Di mana K_p adalah konstanta proporsional (gain). Jika $D_{target}=1.0$ m dan $K_p=0.5$, maka:

- Saat objek sangat dekat ($D_{obj}=0.5$ m): error $e=-0.5$. Kecepatan $v_x = 0.5 \times (-0.5) = -0.25$ m/s (Robot mundur perlahan).
- Saat objek di jarak aman ($D_{obj}=2.0$ m): error $e=1.0$. Kecepatan $v_x = 0.5 \times 1.0 = 0.5$ m/s (Robot maju dengan kecepatan proporsional).
- Saat objek tepat di target ($D_{obj}=1.0$ m): error $e=0$. Kecepatan $v_x=0$ (Robot diam stabil).

Agar robot tidak melaju terlalu cepat jika objek sangat jauh, atau mundur terlalu cepat jika sangat dekat, output P-Controller harus dibatasi oleh Saturation Limits (batas kecepatan fisik motor):

$$v_x(t) = \max(-v_{max}, \min(v_{max}, K_p \cdot e(t)))$$

Implementasi logika P-Controller ini di dalam loop pemrosesan Kinect menghubungkan persepsi sensor matematika langsung ke aktuator kinematika, membentuk *closed-loop feedback system* yang menjadi ciri khas sistem robotika otonom modern.

```

D_target = 1.0      # Target jarak (meter)
Kp = 0.5            # Gain Proporsional
V_max = 0.5         # Batas kecepatan maju maksimum (m/s)
V_min = -0.2        # Batas kecepatan mundur maksimum
(m/s)

if len(valid_depths) > 0:
    # 9. Kalkulasi Error  $e(t) = D_{obj}(t) - D_{target}$ 
    error = distance_m - D_target

    # 10. P-Controller:  $v_x = Kp * e(t)$ 
    v_x_raw = Kp * error

    # 11. Saturation Limits (Clamping)
    v_x = max(V_min, min(V_max, v_x_raw))

    # Output perintah kecepatan ke motor/ROS
    print(f"Control Command: v_x = {v_x:.2f} m/s")

    # Visualisasi status kontrol pada citra
    if v_x > 0:
        status = "FORWARD"
    elif v_x < 0:
        status = "REVERSE"
    else:
        status = "STOP"

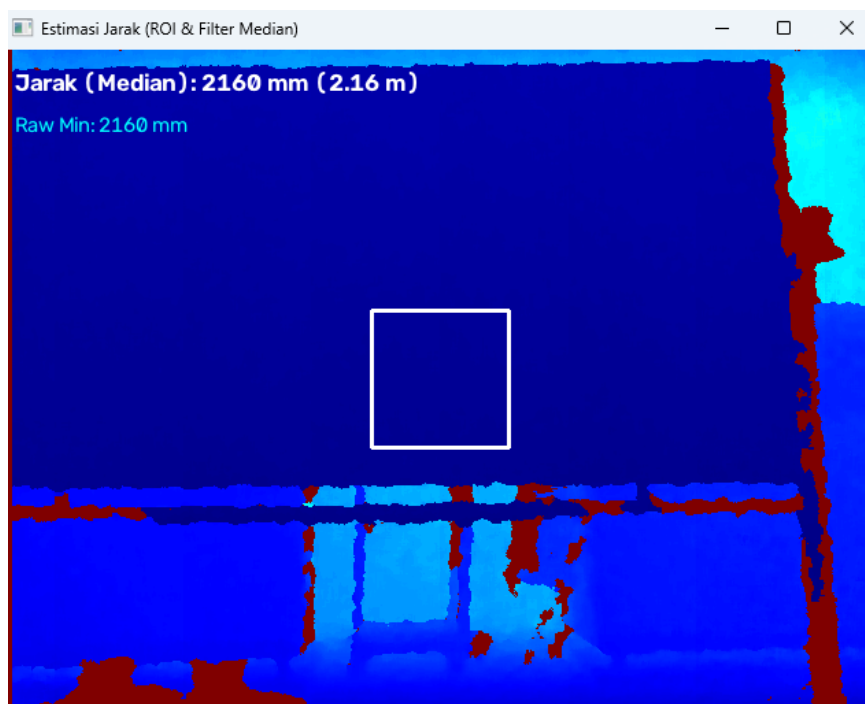
    cv2.putText(bgr_array, f"Status: {status}", (10,
30),
                                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
0), 2)

```

```
# Visualisasi Frame Gabungan
cv2.imshow('RGB + Distance Control', bgr_array)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Cleanup
rgb_stream.stop()
depth_stream.stop()
openni2.unload()
cv2.destroyAllWindows()
```

Berikut ini merupakan output percobaan yang dilakukan pada Bab 9 sebagai berikut pada Gambar 9.1.



Gambar 9.1. Output Bab 9

BAB X

MENGESTIMASI ARAH DENGAN KINECT

10.1. Prinsip Estimasi Sudut Arah (Bearing) pada Bidang Citra

Pada Bab IX, kita telah berhasil mengestimasi jarak absolut objek D_{obj} terhadap sensor pada sumbu Z (kedalaman), memungkinkan robot untuk mengatur kecepatan linear v_x (maju-mundur). Namun, navigasi otonom di ruang 2D/3D yang penuh rintangan tidak hanya membutuhkan pemahaman *distance* (seberapa jauh), tetapi juga bearing (di arah mana). Estimasi arah memungkinkan robot untuk memutar tubuhnya (*yaw rotation*) menghadapi target atau menghindari hambatan, yang dikendalikan melalui kecepatan angular ω_z . Secara geometris, sensor Kinect dapat dimodelkan sebagai kamera *pinhole* (lubang jarum). Pada model ini, bidang citra 2D I adalah proyeksi perspektif dari ruang dunia nyata 3D R^3 . Titik tengah horizontal bidang citra, yang direpresentasikan oleh koordinat piksel *optical center* c_x (umumnya ≈ 320 untuk resolusi 640), merepresentasikan sumbu longitudinal robot—yaitu arah lurus ke depan (0°). Objek yang terdeteksi pada koordinat piksel horizontal u_{obj} di bidang citra mengindikasikan bahwa objek tersebut berada di sisi kanan atau kiri robot. Deviasi piksel $\Delta u = u_{obj} - c_x$ dari titik tengah ini secara matematis berkorelasi langsung dengan sudut azimuth θ (sudut arah horizontal) objek relatif terhadap frame referensi robot. Untuk mengkonversi deviasi piksel Δu menjadi sudut fisis θ (dalam radian atau derajat), kita harus melakukan invers dari persamaan proyeksi perspektif pinhole. Proyeksi standar memetakan koordinat dunia 3D (X, Z) ke piksel u menggunakan parameter intrinsik kamera *focal length* f_x (dalam satuan piksel):

$$u = f_x \left(\frac{Z}{X} \right) + c_x$$

Dari geometri trigonometri 3D, sudut azimuth θ antara vektor arah robot (sumbu Z) dan vektor yang menghubungkan sensor ke objek (X, Z) didefinisikan sebagai:

$$\tan(\theta) = \frac{X_{OBJ}}{Z_{OBJ}}$$

Substitusikan rasio Z/X dari invers proyeksi pinhole $\left(\frac{u_{OBJ} - c_x}{f_x} \right)$ ke dalam persamaan tangensial, kita mendapatkan Persamaan Estimasi Bearing yang fundamental:

$$\tan(\theta) = \left(\frac{u_{obj} - c_x}{f_x} \right)$$

Dengan mengaplikasikan fungsi trigonometri invers (*arctangent*), sudut arah objek θ dihitung secara absolut: $\theta = \arctan(f_x u_{obj} - c_x)$

Di mana:

- θ : Sudut azimuth target (radian). $\theta > 0$ berarti objek di kanan, $\theta < 0$ berarti di kiri.
- u_{obj} : Koordinat horizontal centroid objek pada bidang citra (piksel).
- c_x : Titik pusat optikal kamera (piksel). Untuk Kinect V1, $c_x \approx 319.5$
- f_x : Focal length horizontal kamera (piksel). Untuk Kinect V1, $f_x \approx 525$.

Persamaan ini memberikan keuntungan monumental: ia membebaskan estimasi arah dari ketergantungan pada jarak objek Z_{obj} . Apakah objek berada di 1 meter atau 5 meter dari sensor, selama objek berada pada piksel u_{obj} yang sama, sudut arah θ yang dihasilkan akan identik. Hal ini memungkinkan robot untuk memutar arah (*steering*) menuju target jauh sebelum sensor mampu mengukur jarak kedalaman secara akurat.

Sebagai alternatif komputasi yang lebih ringan (sering digunakan pada microcontroller yang tidak memiliki FPU untuk operasi **arctan**), sudut dapat diestimasi menggunakan pendekatan Linear Field-of-View (FoV) Approximation. Asumsi linear menganggap bahwa distribusi piksel pada bidang citra proporsional langsung dengan sudut FoV total αFoV . Koefisien konversi derajat-per-piksel $Kdpp$ dihitung sebagai:

$$Kdpp = W \alpha FoV$$

Untuk Kinect V1 dengan $\alpha FoV = 57^\circ$ dan $W = 640$ piksel, $Kdpp \approx 0.089^\circ/\text{piksel}$. Estimasi sudut linear menjadi:

$$\theta_{linear} \approx (u_{obj} - c_x) \times Kdpp$$

Meskipun θ_{linear} sangat cepat dihitung, ia mengandung error distorsi barrel (*radial distortion*) yang meningkat drastis di perbatasan lensa (piksel u mendekati 0 atau 640). Oleh karena itu, untuk kendali robotika presisi, derivasi **arctan** (θ) tetap menjadi standar emas matematis yang harus diimplementasikan.

10.2. Ekstraksi Posisi Piksel Centroid Objek

Untuk menghitung θ , kita membutuhkan parameter input u_{obj} , yaitu koordinat horizontal *centroid* (titik massa geometris) objek di bidang citra 2D. Pada Bab IX, kita

mengeksktraksi *bounding box* (x,y,w,h) dari kontur objek C. Pendekatan paling trivial untuk menentukan *uobj* adalah menghitung titik tengah kotak pembatas:

$$u_{obj, box} = x + \frac{w}{2}$$

Meskipun efisien secara komputasi, *uobj,box* memiliki kelemahan signifikan: ia mengasumsikan massa objek tersebar simetris di dalam *bounding box*. Jika objek memiliki bentuk asimetris (misalnya, lengan manusia yang menjauhi tubuh), atau jika segmentasi warna hanya menangkap sebagian objek (*partial occlusion*), centroid kotak akan bergeser jauh dari massa geometris visual objek yang sesungguhnya, menyebabkan robot mengarahkan putaran yaw ω_z ke lokasi kosong di samping objek.

Solusi matematis yang lebih robust adalah menghitung Image Moment (Momen Citra). Momen citra adalah konsep dari fisika statistik yang diadopsi ke visi komputer untuk mengukur properti geometris distribusi piksel. Untuk binary mask $M(u,v) \in \{0,255\}$ yang dihasilkan oleh segmentasi warna, *Spatial Moment* orde pertama m_{10} dan *Zeroth Moment* (total massa/luas) m_{00} didefinisikan sebagai:

$$m_{10} = \sum_{u=0}^{w-1} \sum_{v=0}^{H-1} M(u,v) \cdot u \quad m_{00} = \sum_{u=0}^{w-1} \sum_{v=0}^{H-1} M(u,v)$$

Koordinat horizontal centroid geometris objek *uobj* kemudian diekstraksi menggunakan rasio momen:

$$u_{obj} = \frac{m_{10}}{m_{00}}$$

OpenCV menyediakan fungsi `cv2.moments(contour)` yang menghitung seluruh himpunan momen (m_{00}, m_{10}, m_{01} , hingga momen orde tinggi untuk orientasi) secara teroptimasi dalam satu kali eksekusi C++. Keuntungan menggunakan *uobj* berbasis momen adalah akurasi sub-piksel; hasil divisi $m_{00}m_{10}$ menghasilkan nilai *float* (misal 320.5), memberikan granularitas navigasi yang lebih halus dibanding koordinat integer *bounding box*. Berikut adalah implementasi kode integrasi ekstraksi centroid dan perhitungan sudut bearing θ :

```
import numpy as np
import cv2
import math
from openni import openni2
```

```

# ... (Inisialisasi OpenNI, Stream RGB & Depth, Segmentasi Warna Merah seperti Bab IX)

# Parameter Intrinsik Kinect V1
CX_OPTICAL = 319.5
FX_FOCAL = 525.0

while True:
    # ... (Pembacaan frame, Konversi HSV, Thresholding, FindContours)

    if len(contours) > 0:
        c = max(contours, key=cv2.contourArea)

        # --- METODE CENTROID MOMEN CITRA ---
        M = cv2.moments(c)

        # Validasi: m00 tidak boleh 0 (objek tidak boleh kosong)
        if M["m00"] != 0:
            # Kalkulasi Centroid Geometris:  $u_{obj} = m_{10} / m_{00}$ 
            u_obj = int(M["m10"] / M["m00"])
            v_obj = int(M["m01"] / M["m00"]) #  $v_{obj} = m_{01} / m_{00}$  (untuk sumbu Y)

            # --- KALKULASI SUDUT BEARING MATEMATIS ---
            #  $\theta = \arctan((u_{obj} - c_x) / f_x)$ 
            delta_u = u_obj - CX_OPTICAL
            theta_rad = math.atan2(delta_u, FX_FOCAL) # atan2 lebih aman dari atan(y/x)
            theta_deg = math.degrees(theta_rad)

            # Visualisasi centroid dan arah bearing pada RGB Image
            cv2.circle(bgr_array, (u_obj, v_obj), 5, (0, 255, 0), -1)

```

```

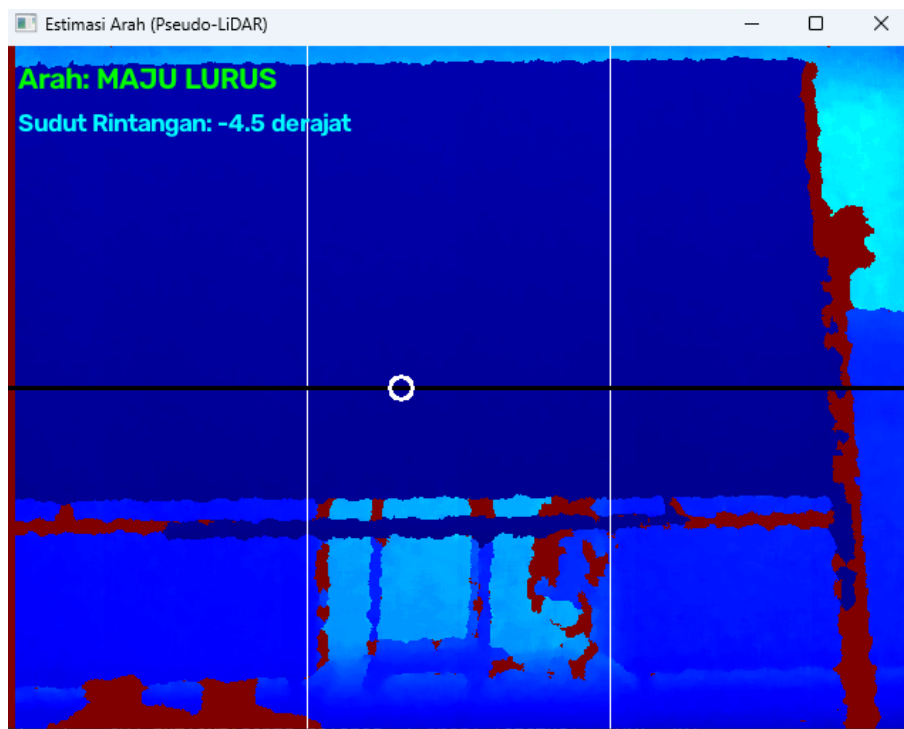
cv2.line(bgr_array, (int(CX_OPTICAL), 240), (u_obj, v_obj), (255, 0, 0), 2)

bearing_text = f"Bearing: {theta_deg:.1f} deg"
cv2.putText(bgr_array, bearing_text, (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)

print(f'Objek terdeteksi pada piksel u={u_obj}, Sudut Arah
theta={theta_deg:.1f} derajat")

```

Berikut dibawah ini merupakan output yang di cobakan sebagai berikut ini pada Bab 10 yang ditunjukkan Gambar 10.1



Gambar 10.1. Output Bab 10

BAB XI

MENGGUNAKAN POSE ESTIMATION PADA RGB IMAGE

11.1. Konsep Pose Estimation 2D (Keypoints Detection)

Pose Estimation 2D adalah proses komputasional untuk mendeteksi dan melokalisir koordinat anatomis tubuh manusia, yang dikenal sebagai *keypoints* (titik kunci), secara langsung dari citra dua dimensi (2D). Dalam konteks robotika, meskipun sensor Kinect memiliki kemampuan *skeleton tracking* 3D bawaan yang memanfaatkan sensor inframerah dan kedalaman, seringkali kita hanya membutuhkan input dari kamera RGB murni. Alasannya adalah untuk menghemat bandwidth USB, mengurangi beban komputasi prosesor *edge*, atau mengadaptasi algoritma agar nantinya bisa berjalan di kamera webcam murah yang tidak punya sensor depth. Dengan hanya memanfaatkan matriks piksel warna, algoritma kecerdasan buatan dilatih untuk menyimpulkan postur tubuh manusia dari pola visual bayangan, tekstur, dan kontur pakaian. Secara matematis, tujuan dari *2D Pose Estimation* adalah memetakan citra input *IRGB* berukuran $H \times W \times 3$ menjadi sekumpulan koordinat 2D.

Output yang dihasilkan adalah sekumpulan titik $K = \{p_1, p_2, \dots, p_N\}$, di mana N adalah jumlah *keypoints* (misalnya 17 titik untuk kerangka MOVE_NET atau 33 titik untuk MediaPipe). Setiap titik p_i direpresentasikan sebagai vektor $p_i = (x_i, y_i, c_i)$, di mana (x_i, y_i) adalah koordinat piksel pada sumbu horizontal dan vertikal, serta $c_i \in [0, 1]$ adalah nilai *confidence score* (tingkat keyakinan model) bahwa titik tersebut benar-benar terdeteksi. Proses deteksi ini tidak bergantung pada pengukuran jarak fisik (kedalaman), melainkan murni pengenalan pola visual dari data latih berupa jutaan gambar manusia.

11.2. Algoritma Pelacakan Pose (MediaPipe)

Untuk menjalankan *Pose Estimation* pada stream RGB Kinect, kita menggunakan framework MediaPipe yang dikembangkan oleh Google. MediaPipe menggunakan arsitektur jaringan saraf tiruan (CNN) yang sangat ringan dan dioptimalkan, yang dikenal dengan nama model *BlazePose*. Keunggulan utama BlazePose adalah kemampuannya berjalan secara *real-time* (di atas 30 FPS) pada prosesor CPU standar atau *Single Board Computer* (SBC) seperti Raspberry Pi atau Jetson Nano, jauh lebih cepat dan ringan dibandingkan algoritma *skeleton tracking* Depth IR bawaan Kinect yang membutuhkan

library OpenNI/NITE berat.

Algoritma *BlazePose* bekerja melalui mekanisme dua tahap. Tahap pertama adalah *Detector* yang memindai seluruh citra RGB untuk mendeteksi keberadaan manusia dan membuat *Bounding Box* (kotak pembatas) di sekitar tubuh. Tahap kedua adalah *Tracker/Regressor* yang menerima *region of interest* (ROI) dari kotak tersebut dan memprediksi lokasi 33 titik *keypoints* secara presisi. Sebelum masuk ke jaringan saraf, citra RGB dari Kinect harus melalui proses *pre-processing* vektorisasi. Nilai piksel 8-bit (0-255) dinormalisasi ke rentang $[-1, 1]$ menggunakan persamaan:

$$I_{norm}(x, y) = \frac{I_{RGB}(x, y) - 127.5}{127.5}$$

Normalisasi tersebut memastikan gradien jaringan saraf stabil saat melakukan proses *backpropagation* dan inferensi. Karena MediaPipe dirancang untuk inferensi gambar warna, data matriks 16-bit dari sensor Depth Kinect sama sekali tidak digunakan dalam algoritma.

11.3. Ekstraksi Koordinat Pixel Sendi Tubuh

Output mentah dari model MediaPipe bukanlah koordinat piksel absolut, melainkan koordinat yang dinormalisasi terhadap resolusi citra input. Hal ini dilakukan agar model bersifat *resolution-invariant* (tidak terpengaruh ukuran gambar). Jika stream RGB Kinect memiliki resolusi $W=640$ piksel (lebar) dan $H=480$ piksel (tinggi), maka MediaPipe akan memberikan output $x_{norm} \in [0,1]$ dan $y_{norm} \in [0,1]$. Untuk mendapatkan koordinat piksel absolut (u, v) di atas layar monitor, kita harus melakukan denormalisasi menggunakan persamaan transformasi skalar linier berikut:

$$u_i = x_{norm, i} \times W$$

$$v_i = y_{norm, i} \times H$$

Selain itu, sistem harus mengevaluasi *confidence score* c_i untuk setiap sendi. Jika c_i berada di bawah ambang batas tertentu (misalnya $\tau=0.5$), sendi tersebut dianggap tidak terlacak (occluded/terhalang) dan sistem harus mengabaikannya agar robot tidak salah membaca gestur. Ekstraksi koordinat ini adalah jembatan krusial antara output *Deep Learning* abstrak dan logika kontrol kinematika robot yang membutuhkan angka pasti. Berikut ini merupakan program yang digunakan

```
def extract_2d_joints(nite_tracker, user):  
    """
```

```

    Mengekstrak koordinat piksel 2D (u, v) dari sendi bahu,
siku,
    dan pergelangan tangan kanan menggunakan OpenNI.
    """
    skeleton = user.skeleton

    # Mengambil joint spesifik dari NITE
    j_shoulder =
skeleton.get_joint(nite2.JointType.RIGHT_SHOULDER)
    j_elbow = skeleton.get_joint(nite2.JointType.RIGHT_ELBOW)
    j_wrist = skeleton.get_joint(nite2.JointType.RIGHT_HAND)

    # Cek kepercayaan posisi (confidence > 0.5 berarti terlacak
baik)
    if (j_shoulder.positionConfidence > 0.5 and
        j_elbow.positionConfidence > 0.5 and
        j_wrist.positionConfidence > 0.5):

        # Konversi World 3D (X,Y,Z) ke Depth/Pixel 2D (u,v)
        # Fungsi ini memproyeksikan titik 3D ke bidang piksel
        p_shoulder =
nite_tracker.convert_joint_coordinates_to_depth(
            j_shoulder.position.x,      j_shoulder.position.y,
j_shoulder.position.z)
        p_elbow =
nite_tracker.convert_joint_coordinates_to_depth(
            j_elbow.position.x,          j_elbow.position.y,
j_elbow.position.z)
        p_wrist =
nite_tracker.convert_joint_coordinates_to_depth(

```



```

        j_wrist.position.x,          j_wrist.position.y,
j_wrist.position.z)

    # Mengembalikan dalam format tuple integer (u, v)
    return (int(p_shoulder.x), int(p_shoulder.y)), \
           (int(p_elbow.x), int(p_elbow.y)), \
           (int(p_wrist.x), int(p_wrist.y))

return None, None, None

```

11.4. Perhitungan Sudut Sendi (Angle Estimation)

Setelah koordinat piksel 2D (u, v) dari sendi-sendi tubuh berhasil diproyeksikan ke atas citra RGB, langkah selanjutnya adalah menerjemahkan pola posisi titik-titik tersebut menjadi data numerik yang dapat diproses robot, yaitu berupa nilai sudut antar-tulang (*joint angle*). Misalnya, untuk mengetahui apakah operator sedang mengangkat tangan ke samping atau menekuk siku di depan dada, yang dibatasi oleh tiga titik geometris: bahu ($P1$), siku ($P2$), dan pergelangan tangan ($P3$).

Perhitungan sudut dilakukan menggunakan prinsip geometri vektor 2D. Pertama, kita membentuk dua vektor garis, vektor A yang membentang dari siku ke bahu, dan vektor B yang membentang dari siku ke pergelangan tangan. Secara matematis:

$$A = P1 - P2 = (u1 - u2, v1 - v2)$$

$$B = P3 - P2 = (u3 - u2, v3 - v2)$$

Sudut θ di sendi siku ($P2$) dapat dihitung menggunakan operasi *dot product* (perkalian titik) dari kedua vektor tersebut. Rumus *dot product* menyatakan bahwa $A \cdot B = |A| |B| \cos(\theta)$. Dengan menurunkan rumus ini, kita mendapatkan persamaan untuk mencari sudut θ :

$$\theta = \arccos(|A| \cdot |B| / A \cdot B) \text{ Di mana } A \cdot B = (u1 - u2)(u3 - u2) + (v1 - v2)(v3 - v2), \text{ dan } |A| \text{ serta } |B|$$

Magnitudo (panjang) masing-masing vektor yang dihitung menggunakan teorema Pythagoras. Penerapan sudut ini sangat efektif untuk *state machine* gestur. Jika sudut siku θ terhitung sekitar 160° – 180° , sistem menyimpulkan lengan lurus. Jika $\theta < 60^\circ$, siku tertekuk.

```

import numpy as np

def calculate_2d_angle(p1, p2, p3):
    """Menghitung sudut (derajat) di titik p2 (siku)."""
    a = np.array(p1) # Bahu
    b = np.array(p2) # Siku (titik pusat)
    c = np.array(p3) # Pergelangan tangan

    vec_a = a - b
    vec_b = c - b

    dot_product = np.dot(vec_a, vec_b)
    mag_a = np.linalg.norm(vec_a)
    mag_b = np.linalg.norm(vec_b)

    if mag_a * mag_b == 0:
        return 0.0

    cosine_angle = np.clip(dot_product / (mag_a * mag_b), -1.0,
1.0)
    return np.degrees(np.arccos(cosine_angle))

def main_pose_estimation():
    while True:
        img = get_rgb_image()
        # Membaca frame Depth untuk User Tracker
        depth_frame = depth_stream.read_frame()
        depth_data = depth_frame.get_buffer_as_uint16()
        depth_array = np.frombuffer(depth_data,
dtype=np.uint16)

```

```

        depth_array = depth_array.reshape((depth_frame.height,
depth_frame.width))

# Melacak user di OpenNI NITE
user_frame = nite_tracker.read_frame()

for user in user_frame.users:
    if user.is_new():
        nite_tracker.start_skeleton_tracking(user.id)
    elif user.state == nite2.UserState.USER_TRACKED:
        # Ekstrak 3 titik sendi dalam piksel 2D
        p_shoulder,      p_elbow,      p_wrist      =
extract_2d_joints(nite_tracker, user)

        if p_shoulder is not None:
            # Hitung sudut siku
            angle      =      calculate_2d_angle(p_shoulder,
p_elbow, p_wrist)

            # Visualisasi di Citra RGB
            cv2.circle(img, p_shoulder, 7, (0, 255, 0),
-1)

            cv2.circle(img, p_elbow, 7, (0, 255, 0), -
1)

            cv2.circle(img, p_wrist, 7, (0, 255, 0), -
1)

            cv2.line(img, p_shoulder, p_elbow, (255, 0,
0), 3)

            cv2.line(img, p_elbow, p_wrist, (255, 0,
0), 3)

```

```

        # Tampilkan nilai sudut di dekat sendi siku
        cv2.putText(img, f"Angle: {angle:.1f} deg",
                    (p_elbow[0] + 10, p_elbow[1]),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0, 0, 255), 2)

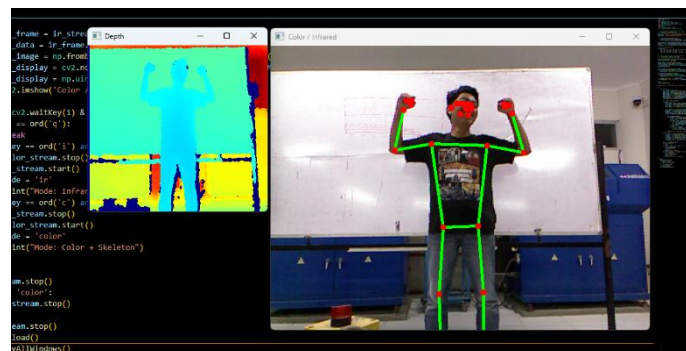
        cv2.imshow('OpenNI Pose Estimation on RGB', img)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    rgb_stream.stop()
    depth_stream.stop()
    openni2.unload()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main_pose_estimation()

```

Berikut ini merupakan output pada penelitian bab 11 yang ditujukan pada Gambar 11.1



Gambar 11.1. Output Bab 11

BAB XII

MEMBANGUN POINT CLOUD DENGAN DEPTH IMAGE

12.1. Pengantar Point Cloud 3D

Point Cloud (awan titik) adalah representasi data spasial tiga dimensi (3D) yang paling murni dari suatu lingkungan fisik. Alih-alih menyajikan data dalam bentuk matriks piksel 2D seperti halnya citra RGB atau Depth Map, Point Cloud menyajikan sekumpulan titik-titik koordinat kartesius $\mathbf{P} = (X, Y, Z)$ di ruang angkasa. Setiap titik dalam awan ini merepresentasikan satu sampel permukaan objek yang dipantulkan kembali ke sensor. Dalam konteks robotika otonom, Point Cloud adalah representasi utama yang digunakan oleh robot untuk benar-benar "memahami" geometri struktur ruangan, mengenali bentuk objek 3D, hingga memetakan topologi lingkungan yang kompleks untuk misi pencarian dan penyelamatan.

Transformasi data dari matriks kedalaman Kinect menjadi Point Cloud adalah langkah esensial karena matriks 2D memiliki keterbatasan dalam menggambarkan struktur volumetrik. Sebagai contoh, dua objek dengan ukuran fisik yang berbeda (satu dekat dan kecil, satu jauh dan besar) dapat menghasilkan proyeksi luas piksel yang sama pada Depth Map. Namun, dalam bentuk Point Cloud, kedua objek tersebut akan memiliki sebaran titik (X, Y, Z) yang sepenuhnya berbeda, memungkinkan robot menghitung volume fisik, jarak Euclidean antar titik, dan vektor normal permukaan secara akurat sesuai dengan skala dunia nyata (dalam satuan meter atau milimeter).

12.2. Persamaan Transformasi Koordinat (u, v, z) ke (X, Y, Z)

Proses pembangkitan Point Cloud secara matematis adalah kebalikan dari proses proyeksi kamera (back-projection). Sensor Kinect memberikan data kedalaman Z pada setiap koordinat piksel (u, v) . Untuk mengembalikan nilai piksel 2D ini ke koordinat ruang kamera 3D (X, Y, Z) , kita memerlukan parameter intrinsik kamera yang diperoleh dari hasil kalibrasi pabrik, yaitu *focal length* (f_x, f_y) yang merepresentasikan jarak fokus lensa dalam satuan piksel, serta *optical center* (c_x, c_y) yang merupakan titik tengah bidang citra. Persamaan transformasi yang memetakan titik piksel $D(u, v)$ beserta nilai kedalamannya ke ruang kamera 3D didefinisikan sebagai berikut:

$$X = \frac{(u - c_x) \times Z}{f_x}, Y = \frac{(v - c_y) \times Z}{f_y}, Z = D(u, v)$$

Proses ini dapat dituliskan secara kompak dalam bentuk perkalian matriks homogen menggunakan invers matriks parameter intrinsik kamera (K^{-1}):

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = Z \cdot K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Di mana K adalah matriks 3×3 yang berisi parameter intrinsik. Penting untuk dicatat bahwa koordinat hasil transformasi ini berpusat pada lensa kamera (Camera Space), di mana sumbu Z mengarah ke depan (jarak ke objek), sumbu X mengarah ke kanan, dan sumbu Y mengarah ke bawah sesuai dengan standar koordinat citra komputer.

12.3. Implementasi Pembangkitan Point Cloud di OpenCV/PCL

Menerjemakan persamaan matematis di atas ke dalam kode program membutuhkan strategi komputasi yang efisien. Jika kita menggunakan perulangan **for** tradisional bersarang (nested loop) untuk menghitung persamaan tersebut pada setiap piksel di citra beresolusi 640×480 (sekitar 307.200 titik), eksekusinya akan memakan waktu berabad-abad dalam skala milidetik, menyebabkan sistem robotika berjalan di bawah 5 frame per second (FPS). Untuk mengatasi hal ini, di lingkungan Python, kita menggunakan operasi vektorisasi massal melalui pustaka NumPy atau memanfaatkan fungsi bawaan OpenCV `cv2.reprojectImageTo3D()`.

Fungsi ini memproses seluruh matriks secara paralel di level C++ di latar belakang. Selain membangun geometri, sebuah Point Cloud yang informatif biasanya memerlukan data warna. Proses ini dikenal sebagai fusi sensor RGB-D. Karena stream RGB dan Depth telah diselaraskan (registrasi), kita dapat mengambil nilai warna (R, G, B) dari piksel yang sama dan melampirkannya pada titik 3D. Hasilnya adalah *Colored Point Cloud*, model 3D yang fotorealistik. Implementasi di bawah ini menunjukkan bagaimana menggabungkan data Depth dan RGB dari Kinect menggunakan operasi vektorisasi NumPy untuk menghasilkan larik Point Cloud berstruktur $(N, 6)$ yang berisi X, Y, Z, R, G, B

12.4. Visualisasi 3D Point Cloud

Membangkitkan larik numerik Point Cloud hingga ratusan ribu titik tidak akan ada gunanya jika pengembang robotika tidak dapat melihatnya secara visual. Visualisasi 3D adalah langkah krusial untuk *debugging* algoritma, verifikasi kalibrasi kamera, dan analisis struktur spasial. Pustaka standar industri untuk ini adalah Point Cloud Library

(PCL) dalam ekosistem C++, atau dalam Python, kita menggunakan pustaka Open3D yang sangat ringan dan efisien. Pustaka ini mampu merender ratusan ribu titik secara interaktif menggunakan akselerasi GPU.

Dalam visualisasi, pengguna dapat memutar, *zoom*, dan menggeser pandangan kamera virtual untuk melihat model 3D dari berbagai sudut yang tidak mungkin dilihat oleh sensor fisik secara langsung. Hal ini membantu insinyur memastikan bahwa geometri objek seperti dinding lurus tidak melengkung (indikasi kalibrasi intrinsik yang salah) dan bahwa warna RGB benar-benar sejajar dengan kontur Depth (indikasi kegagalan registrasi). Kode di bawah ini memanfaatkan objek **open3d.geometry.PointCloud()** untuk merender hasil algoritma di subbab 12.3 ke dalam jendela grafis 3D. Berikut merupakan program yang digunakan

```
import cv2
import numpy as np
import open3d as o3d
import freenect

# Parameter intrinsik Kinect V1
fx, fy = 525.0, 525.0
cx, cy = 319.5, 239.5

def get_kinect_streams():
    # Mengambil data RGB dan Depth secara sinkron
    rgb, _ = freenect.sync_get_video()
    depth, _ = freenect.sync_get_depth()
    rgb = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
    return rgb, depth

def main():
    # Inisialisasi jendela visualisasi Open3D
    vis = o3d.visualization.Visualizer()
    vis.create_window(window_name="Real-Time Kinect Point
Cloud", width=800, height=600)
```

```

# Buat objek Point Cloud kosong
pcd = o3d.geometry.PointCloud()
vis.add_geometry(pcd)
# Atur sudut pandang kamera virtual agar pas
ctr = vis.get_view_control()
ctr.set_front([0, 0, -1])
ctr.set_up([0, -1, 0])
ctr.set_zoom(0.4)
print("Memulai visualisasi Real-Time. Tekan 'Q' di jendela
Open3D untuk keluar.")
while True:
    rgb, depth = get_kinect_streams()
    # 1. Vektorisasi Transformasi Koordinat (u,v,z) ke
(X,Y,Z)
    h, w = depth.shape
    u, v = np.meshgrid(np.arange(w), np.arange(h))

    # Filter noise dan batasi jarak maksimal 4 meter
(4000mm)
    mask = (depth > 0) & (depth < 4000)
    z = depth[mask] / 1000.0
    x = ((u[mask] - cx) * z) / fx
    y = ((v[mask] - cy) * z) / fy
    colors = rgb[mask] / 255.0
    points = np.vstack((x, y, z)).T
    # 2. Update geometri Point Cloud
    pcd.points = o3d.utility.Vector3dVector(points)
    pcd.colors = o3d.utility.Vector3dVector(colors)
    # 3. (Opsional) Voxel Down-sampling untuk performa
grafis yang mulus
    # Menggunakan ukuran voxel 0.01 meter (1 cm)

```



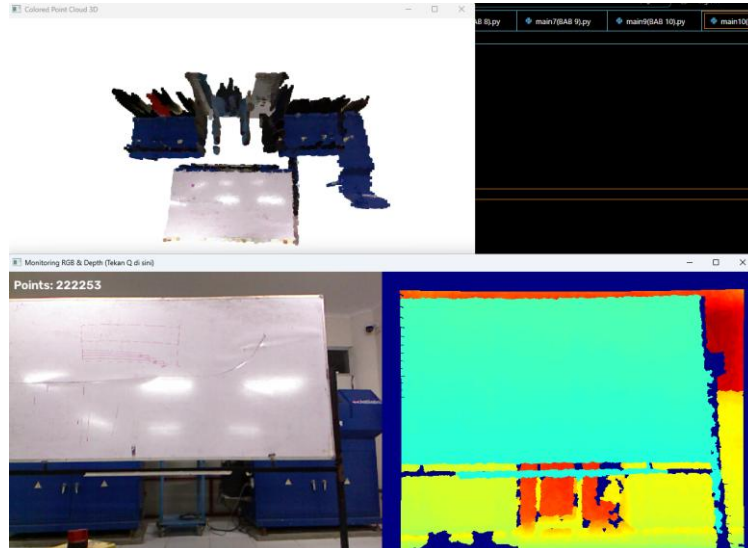
```

pcd_down = pcd.voxel_down_sample(voxel_size=0.01)
# 4. Update render jendela
vis.remove_geometry(pcd)
vis.add_geometry(pcd_down)
vis.poll_events()
vis.update_renderer()

# Loop breaker (jika jendela ditutup atau tombol 'q'
ditekan di terminal)
if not vis.poll_events():
    break
vis.destroy_window()
if __name__ == "__main__":
    main()

```

Berikut ini merupakan hasil output pada Bab 12 yang ditujukan berikut ini pada Gambar 12.1



Gambar 12.1. Output Bab 12

BAB XIII

KONTROL GERAK SEDERHANA DENGAN KINECT DENGAN SIMULASI ROBOT

13.1. Arsitektur Teleoperation Berbasis PyBullet

Teleoperation (kendali jarak jauh) adalah metode pengendalian robot di mana manusia memberikan perintah gerak tanpa harus bersentuhan langsung dengan *joystick* fisik, melainkan melalui gestur tubuh yang ditangkap oleh sensor. Dalam konteks buku ajar ini, sensor Kinect membaca gerakan operator, lalu perangkat lunak menerjemahkan posisi sendi (joints) manusia menjadi perintah kecepatan motor. Untuk menguji algoritma ini sebelum dipasang di robot fisik, kita menggunakan lingkungan simulasi. Berbeda dengan ekosistem ROS (Robot Operating System) yang membutuhkan arsitektur *middleware* jaringan yang kompleks, kita menggunakan PyBullet, mesin fisika 3D berbasis Python yang ringan dan berjalan secara monolitik (satu *script* utuh).

Pemilihan arsitektur PyBullet sangat menguntungkan bagi para peneliti dan mahasiswa karena seluruh *pipeline* pemrosesan data—mulai dari akuisisi citra Kinect, pembacaan kerangka tubuh (skeleton) via OpenNI, perhitungan kinematika, hingga eksekusi putaran roda virtual—dapat ditulis dalam satu file Python tanpa overhead komunikasi antar-*node*. Arsitektur ini menghasilkan latensi yang sangat rendah karena pertukaran data terjadi secara *in-memory* di dalam satu proses utama. Saat operator menggerakkan tangannya, perhitungan matematis segera dieksekusi dan perintah **pybullet.setJointMotorControl2()** langsung dipanggil, membuat robot virtual di layar merespons gerakan dalam hitungan milidetik, menciptakan pengalaman *teleoperation* yang mulus dan *real-time*.

13.2. Setup Lingkungan Simulasi PyBullet

Untuk memulai simulasi, pengguna perlu menginstal pustaka PyBullet melalui terminal dengan perintah **pip install pybullet**. Inisialisasi simulasi dimulai dengan menghubungkan mesin fisika ke antarmuka grafis (GUI) menggunakan **pybullet.connect(pybullet.GUI)**. Setelah koneksi terbuka, pengguna memuat objek-objek dasar ke dalam ruang virtual. Pertama, bidang datar (ground plane) dimuat untuk memberikan arena gerak bagi robot. Kedua, model robot beroda differensial (differential drive) dalam format file URDF (*Unified Robot Description*

Format) dimuat ke dalam ruang simulasi. Model URDF ini mendefinisikan sifat fisik robot seperti massa, dimensi chassis, serta *joint* (sendi) roda kiri dan roda kanan.

PyBullet bekerja dengan cara mensimulasikan gaya, torsi, dan gesekan berdasarkan *time step* (langkah waktu) yang ditentukan. Dalam loop simulasi, fungsi **pybullet.stepSimulation()** harus dipanggil secara terus-menerus untuk memajukan waktu virtual. Saat robot virtual diberi perintah kecepatan sudut pada roda-rodanya, mesin fisika PyBullet akan menghitung interaksi antar roda dengan permukaan tanah, menghasilkan pergerakan linier dan rotasi pada chassis robot secara akurat. Kode di bawah ini menunjukkan bagaimana menginisialisasi PyBullet dan memuat model robot yang akan digunakan untuk eksperimen *teleoperation*.

13.3. Pemetaan Gestur Skeleton ke Kecepatan Motor (Kinematic Mapping)

Agar robot virtual dapat bergerak berdasarkan gerakan tubuh operator, kita harus mendefinisikan aturan pemetaan kinematik (*kinematic mapping*). Pada pendekatan ini, tangan kanan operator dijadikan sebagai "joystick" nirkabel. Posisi tangan kanan relatif terhadap bahu kanan akan menentukan arah dan kecepatan robot. Secara matematis, kita menghitung vektor selisih posisi antara koordinat tangan dan bahu pada sumbu Z (kedalaman) untuk gerakan maju/mundur, dan sumbu X (horizontal) untuk gerakan belok kiri/kanan:

$$\Delta Z = Z_{tangan} - Z_{bahu}$$

$$\Delta X = X_{tangan} - X_{bahu}$$

Jika operator mendorong tangannya ke depan ($\Delta Z > 0$), robot harus maju. Jika tangan digeser ke kanan ($\Delta X > 0$), robot harus belok kanan. Selisih ini kemudian dikalikan dengan konstanta penguatan gain (K_v untuk kecepatan linier dan K_ω untuk kecepatan sudut), lalu dibatasi (*clamped*) agar tidak melampaui batas kecepatan maksimal motor virtual:

$$v = clip(K_v \cdot \Delta Z, -v_{max}, v_{max}) \quad \omega = clip(K_\omega \cdot \Delta X, -\omega_{max}, \omega_{max})$$

Nilai target v (kecepatan linier) dan ω (kecepatan sudut) tersebut merupakan perintah level tinggi. Karena robot di PyBullet menggunakan dua roda aktif (differential drive), perintah ini harus dikonversi menjadi kecepatan sudut roda kiri ($\dot{\phi}_L$) dan roda kanan ($\dot{\phi}_R$) menggunakan persamaan kinematika roda differensial:

$$\dot{\phi}_L = \frac{v - \omega \cdot \frac{L}{2}}{r} \quad \dot{\phi}_R = \frac{v + \omega \cdot \frac{L}{2}}{r}$$

Di mana L adalah jarak antara kedua roda (*wheelbase*), dan r adalah jari-jari roda. Hasil perhitungan $\dot{\phi}L$ dan $\dot{\phi}R$ (dalam satuan radian per detik) inilah yang akan dikirimkan ke *joint motor* robot virtual di PyBullet.

13.4. Implementasi Skrip Python untuk Kendali

Pada tahap implementasi, seluruh komponen—OpenNI untuk pembacaan skeleton, persamaan kinematika, dan API PyBullet—digabungkan dalam satu skrip Python monolitik. Saat program berjalan, OpenNI secara kontinu memantau keberadaan operator. Jika operator terdeteksi, koordinat 3D sendi bahu dan tangan kanan diekstrak. Selisih koordinat ΔX dan ΔZ diumpankan ke dalam persamaan kinematika untuk menghasilkan kecepatan roda kiri dan kanan. Nilai ini kemudian diterapkan ke motor robot menggunakan mode kontrol kecepatan **pybullet.VELOCITY_CONTROL**.

Aspek krusial dalam skrip ini adalah implementasi mekanisme *failsafe* (keselamatan). Pada PyBullet, jika operator keluar dari jangkauan kamera Kinect atau terhalang oleh objek, algoritma OpenNI akan kehilangan jejak skeleton dan mengembalikan status *untracked*. Skrip harus mampu mendeteksi kondisi ini secara otomatis. Jika data skeleton hilang, skrip akan memaksa nilai $\dot{\phi}L=0$ dan $\dot{\phi}R=0$, lalu mengirimkannya ke PyBullet. Hal ini mencegah robot virtual terus melaju tanpa kendali (*runaway robot*) saat operator tidak dilihat oleh sensor, mensimulasikan tombol *emergency stop* virtual. Kode di bawah ini merepresentasikan integrasi penuh dari konsep tersebut.

```
import pybullet as p
import pybullet_data
import time
import numpy as np
from primesense import openni2, nite2

# === Parameter Fisik Robot & Kendali ===
L = 0.35          # Wheelbase (jarak antar roda) dalam meter
r = 0.05          # Jari-jari roda dalam meter
Kv = 1.5          # Gain kecepatan linier
Kw = 1.5          # Gain kecepatan sudut
```

```

V_MAX = 2.0      # Batas kecepatan sudut roda (rad/s)
ALPHA = 0.2      # Koefisien Smoothing EMA (0 = halus/lambat, 1
= no filter)
MOTOR_FORCE = 5 # Torsi maksimum motor virtual (Nm)

def init_kinect_nite():
    # Inisialisasi OpenNI dan NITE (sesuaikan path Redist)
    openni2.initialize("/path/to/OpenNI2/Redist")
    nite2.initialize("/path/to/OpenNI2/Redist")
    dev = openni2.Device.open_any()
    tracker = nite2.UserTracker(dev)
    return tracker

def get_skeleton_delta(nite_tracker):
    user_frame = nite_tracker.read_frame()
    for user in user_frame.users:
        if user.state == nite2.UserState.USER_TRACKED:
            skeleton = user.skeleton
            j_shoulder =
skeleton.get_joint(nite2.JointType.RIGHT_SHOULDER)
            j_hand =
skeleton.get_joint(nite2.JointType.RIGHT_HAND)

            # Hanya proses jika kepercayaan pelacakan tinggi (>
0.5)

            if (j_shoulder.positionConfidence > 0.5 and
                j_hand.positionConfidence > 0.5):

                delta_x = j_hand.position.x -
j_shoulder.position.x

```

```

        delta_z = j_hand.position.z -
j_shoulder.position.z

        return delta_x, delta_z

    # Failsafe: Kembalikan None jika tidak ada user/sendir hilang
    return None, None

def main():
    # === 1. Inisialisasi PyBullet ===
    p.connect(p.GUI)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.81)
    p.loadURDF("plane.urdf")
    # Asumsi robot.urdf memiliki roda kiri (index 0) dan roda
kanan (index 1)
    robot = p.loadURDF("robot.urdf", [0, 0, 0.1])

    # === 2. Inisialisasi Kinect ===
    nite_tracker = init_kinect_nite()
    print("Sistem siap. Berdirilah di depan Kinect dan dorong
tangan kanan ke depan.")

    # Variabel state untuk menyimpan nilai smoothing sebelumnya
    v_smoothed = 0.0
    w_smoothed = 0.0

    while True:
        # === 3. Baca Gestur Skeleton ===
        delta_x, delta_z = get_skeleton_delta(nite_tracker)

        if delta_x is not None:
            # Hitung nilai target mentah

```

```

        v_target = np.clip(Kv * delta_z, -V_MAX * r, V_MAX
* r)

        w_target = np.clip(Kw * delta_x, -V_MAX, V_MAX)
    else:
        # Failsafe: Jika hilang, target di-set 0 agar
berhenti perlahan
        v_target = 0.0
        w_target = 0.0

    # === 4. Terapkan Exponential Moving Average (Smoothing)
===

    v_smoothed = ALPHA * v_target + (1 - ALPHA) * v_smoothed
    w_smoothed = ALPHA * w_target + (1 - ALPHA) * w_smoothed

    # === 5. Kinematika Differensial ===
    # Konversi v dan w ke kecepatan sudut roda (phi dot)
    phi_left = (v_smoothed - w_smoothed * (L / 2.0)) / r
    phi_right = (v_smoothed + w_smoothed * (L / 2.0)) / r

    # === 6. Eksekusi Motor di PyBullet ===
    # Mode VELOCITY_CONTROL membutuhkan targetVelocity
(rad/s) dan force (Nm)
    p.setJointMotorControl2(robot, 0, p.VELOCITY_CONTROL,
                             targetVelocity=phi_left,
force=MOTOR_FORCE)
    p.setJointMotorControl2(robot, 1, p.VELOCITY_CONTROL,
                             targetVelocity=phi_right,
force=MOTOR_FORCE)

    # Majukan simulasi fisika PyBullet
    p.stepSimulation()

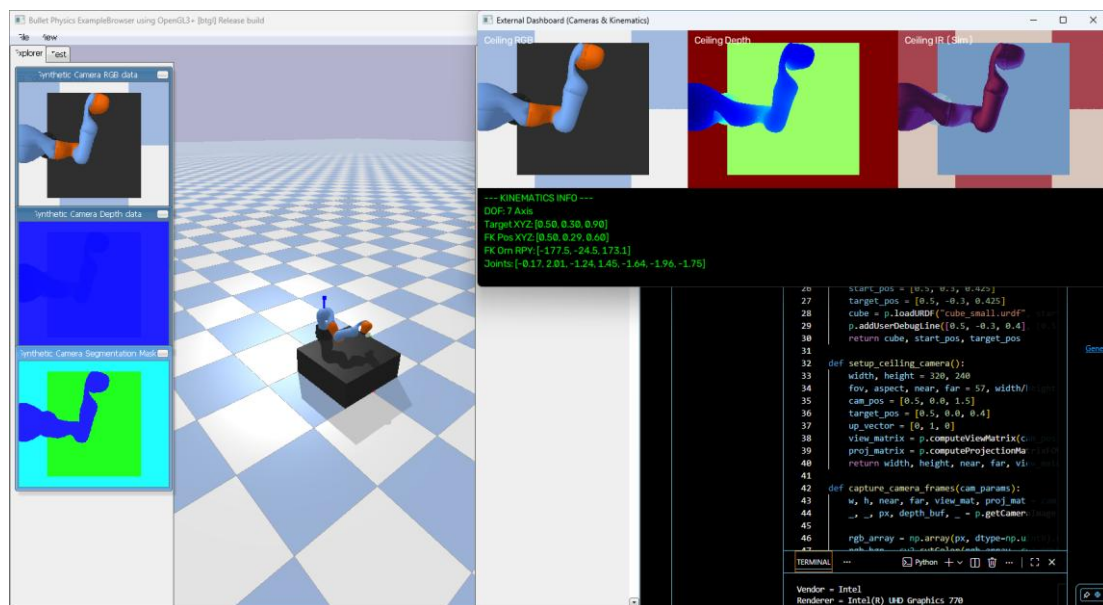
```

```
# Jaga agar simulasi berjalan secara real-time (240 Hz
physics loop)

time.sleep(1./240.)

if __name__ == "__main__":
    main()
```

Berikut ini merupakan Percobaan pada bab 13 sebagai berikut ini yang ditujukan pada Gambar 13.1.



Gambar 13.1. Output Bab 13

Berikut ini merupakan repository yang dilakukan pada percobaan pada bab 13 sebagai berikut:

https://youtu.be/j1ZueIetGmU?si=MjA_AXUwa1IZDF9Q

<https://github.com/devanys/Kuka-LBR-iiwa-Robotic-Arm-Pick-And-Place>

BAB XIV

KONTROL GERAK SEDERHANA UNTUK ROBOT BERODA DI DUNIA NYATA

14.1. Arsitektur Robot Beroda (TurtleBot URDF) di PyBullet

Dalam ekosistem simulasi robotika, "dunia nyata" direpresentasikan melalui model fisika yang akurat. Untuk menggantikan *hardware* fisik, kita menggunakan model TurtleBot URDF (*Unified Robot Description Format*) di dalam mesin fisika PyBullet. TurtleBot adalah platform robot beroda differensial standar yang memiliki dua roda aktif (kiri dan kanan) dan satu roda pasif (caster) untuk keseimbangan. File URDF mendefinisikan tidak hanya bentuk visual robot, tetapi juga properti fisik krusial seperti massa chassis, jari-jari roda (r), jarak antar roda (*wheelbase*, L), koefisien gesekan ban dengan lantai, serta *inertia* (kelembaban) setiap *link* robot.

Arsitektur simulasi ini dirancang untuk mereplika keterbatasan *hardware* nyata. Mesin PyBullet akan mensimulasikan efek gravitasi, torsi motor, dan gesekan statis. Jika robot virtual diperintahkan bergerak menanjak atau menabrak dinding virtual, perhitungan fisika akan menghentikan pergerakan robot secara realistis layaknya roda terjepit. Hal ini memungkinkan kita menguji algoritma *obstacle avoidance* dan kendali kinematika tanpa risiko merusak komponen elektronik. Kode di bawah ini menunjukkan bagaimana memuatkan model TurtleBot URDF dan mengonfigurasi lingkungan simulasi yang mensimulasikan properti dunia nyata.

14.2. Kinematika dan Kalibrasi Odometri Motor DC

Pada robot beroda differensial, pergerakan dikendalikan oleh perbedaan kecepatan rotasi roda kiri dan roda kanan. Untuk menggerakkan robot virtual, perintah kecepatan linier (v) dan kecepatan sudut (ω) dari algoritma navigasi harus dikonversi menjadi kecepatan sudut roda target ($\dot{\phi}L$ dan $\dot{\phi}R$) menggunakan persamaan kinematika roda differensial:

$$\dot{\phi}L = \frac{v - \omega \frac{L}{2}}{r} \quad \dot{\phi}R = \frac{v + \omega \frac{L}{2}}{r}$$

Di mana L adalah jarak antara kedua roda dan r adalah jari-jari roda TurtleBot. Nilai $\dot{\phi}$ (dalam radian per detik) ini kemudian dikirim ke *joint motor* PyBullet menggunakan mode kontrol kecepatan (**p.VELOCITY_CONTROL**). Meskipun ini

adalah simulasi, konsep Odometri tetap diperlukan untuk melacak posisi robot di dunia virtual. PyBullet secara akurat mensimulasikan *rotary encoder* dengan menyediakan fungsi `p.getJointState()`. Dengan membaca posisi sudut roda dari *frame* ke *frame*, kita dapat mengestimasi perpindahan robot ($\Delta x, \Delta y, \Delta \theta$). Persamaan update posisi odometri berdasarkan pergerakan roda adalah:

$$\Delta s = 2r \cdot (\Delta \phi_R + \Delta \phi_L)$$

$$\Delta \theta = Lr \cdot (\Delta \phi_R - \Delta \phi_L)$$

$$x_t = x_{t-1} + \Delta s \cdot \cos(\theta_{t-1} + \Delta \theta)$$

$$y_t = y_{t-1} + \Delta s \cdot \sin(\theta_{t-1} + \Delta \theta)$$

Kalibrasi parameter L dan r sangat penting dalam simulasi. Jika nilai L pada kode program tidak sama dengan nilai *wheelbase* yang terdefinisi di dalam file URDF, akumulasi odometri robot virtual akan mengalami *drift* (geseran perhitungan) yang signifikan saat robot berbelok, sama seperti kasus miscalibration pada robot fisik.

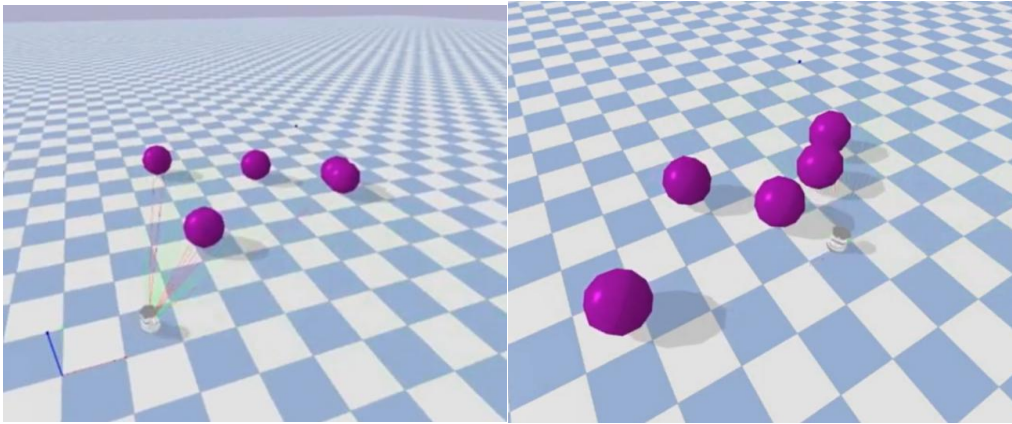
14.3. Implementasi Obstacle Avoidance Berbasis Depth

Algoritma navigasi otonom membutuhkan data persepsi lingkungan. Pada simulasi ini, kita mengimplementasikan sensor kedalaman virtual (menggunakan kamera bawaan PyBullet) yang berperilaku identik dengan sensor Kinect Depth. PyBullet merender citra kedalaman dari sudut pandang robot menggunakan matriks *view* dan *projection*. Nilai *depth buffer* yang dihasilkan oleh PyBullet berupa jarak normalisasi antara 0 hingga 1. Untuk mendapatkan nilai jarak metrik yang sebenarnya (*Z metrik*) dalam satuan milimeter atau meter, kita harus melakukan denormalisasi menggunakan parameter *near plane* (N) dan *far plane* (F) dari kamera virtual:

$$Z_{metrik} = F - (F - N) \cdot Z_{buffer} / F \cdot N$$

Data matriks kedalaman (Depth Map) ini kemudian diiris secara horizontal pada baris piksel tengah untuk menciptakan *Pseudo-LiDAR*. Algoritma membagi irisan ini menjadi tiga zona arah: Kiri, Tengah, dan Kanan. Jika terdapat piksel pada zona Tengah yang bernilai kurang dari ambang batas aman (misal $D_{safe}=800$ mm), robot diharuskan berhenti ($v=0$) dan memeriksa zona Kiri serta Kanan untuk mencari celah (valley) dengan jarak bebas hambatan terjauh untuk berbelok. Mekanisme ini dikenal sebagai *Virtual Bumper*. Implementasi kode di bawah ini menggabungkan akuisisi Depth Image dari PyBullet, pemrosesan *Pseudo-LiDAR*, perhitungan kinematika differensial, dan eksekusi

motor TurtleBot secara *real-time*. Berikut ini merupakan hasil penelitian yang ditujukan pada Gambar 14.1



Gambar 14.1. Obstacle Avoidance mobile robot menggunakan Pybullets

Berikut ini merupakan repository pada penelitian yang dilakukan untuk percobaan Bab 14 sebagai berikut

https://youtu.be/VteC_HYPnDo?si=gpTFdgNfwiVfk4kl
<https://github.com/devanys/DDMR-VFH-Collision-Avoidance>
<https://github.com/devanys/VFH-Collision-Avoidance>

DAFTAR PUSTAKA

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [2] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [3] G. Bradski and A. Kaehler, *Learning OpenCV 4: Computer Vision in C++ with the OpenCV Library*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [4] J. C. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2005.
- [5] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [7] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, UK: Cambridge University Press, 2004.
- [8] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, UK: Springer, 2009.
- [9] Z. Zhang, "Microsoft Kinect sensor and its effect," *IEEE Multimedia*, vol. 19, no. 2, pp. 4-10, Feb. 2012.
- [10] K. Khoshelham and S. O. Elberink, "Accuracy and resolution of Kinect depth data for indoor mapping applications," *Sensors*, vol. 12, no. 2, pp. 1437-1454, Feb. 2012.
- [11] J. Smisek, M. Jancosek, and T. Pajdla, "3D with Kinect," in *Consumer Depth Cameras for Computer Vision*, London, UK: Springer, 2013, pp. 3-25.
- [12] P. Henry, M. Krainin, E. Herbst, X. Ren, and D. Fox, "RGB-D mapping: Using Kinect-style depth cameras for dense 3D modeling of indoor environments," *The International Journal of Robotics Research*, vol. 31, no. 5, pp. 647-663, Apr. 2012.
- [13] F. Endres, J. Hess, J. Sturm, D. Cremers, and W. Burgard, "3-D mapping with an RGB-D camera," *IEEE Transactions on Robotics*, vol. 30, no. 1, pp. 177-187,

Feb. 2014.

- [14] R. B. Rusu and S. Cousins, "3D is here: Point Cloud Library (PCL)," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Shanghai, China, 2011, pp. 1-4.
- [15] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, Kobe, Japan, 2009, vol. 3, no. 3.2, p. 5.
- [16] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34-46, Feb. 2007.
- [17] S. Kohbrecher, J. Meyer, T. Graber, K. Petersen, U. Klingauf, and O. von Stryk, "Hector open source modules for autonomous mapping and navigation with rescue robots," in *Proc. RoboCup 2013: Robot World Cup XVII*, Berlin, Germany, 2014, pp. 624-631.
- [18] E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, "The office marathon: Robust navigation in an indoor office environment," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Anchorage, AK, USA, 2010, pp. 300-307.
- [19] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986.
- [20] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91-110, Nov. 2004.
- [21] H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, "Speeded-Up Robust Features (SURF)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346-359, Jun. 2008.
- [22] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, Barcelona, Spain, 2011, pp. 2564-2571.
- [23] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, San Diego, CA, USA, 2005, vol. 1, pp. 886-893.

- [24] M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179-187, Feb. 1962.
- [25] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- [26] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," *arXiv preprint arXiv:2006.10204*, 2020.
- [27] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. 7th International Joint Conference on Artificial Intelligence (IJCAI)*, Vancouver, Canada, 1981, vol. 2, pp. 674-679.
- [28] J. Borenstein and Y. Koren, "The vector field histogram-fast obstacle avoidance for mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278-288, Jun. 1991.
- [29] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23-33, Mar. 1997.
- [30] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016-2021. [Online]. Available: <https://pybullet.org/>
- [31] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, Kobe, Japan, 2009, vol. 3, no. 3.2, p. 5.
- [32] PrimeSense Ltd., "OpenNI 2.0 Programmer's Guide," 2013. [Online]. Available: <https://github.com/OpenNI/OpenNI2>
- [33] OpenKinect Project, "libfreenect: Open source drivers for the Xbox Kinect device," 2010. [Online]. Available: <https://openkinect.org/>
- [34] Intel Corporation, "Intel RealSense SDK 2.0 Documentation," 2021. [Online]. Available: <https://dev.intelrealsense.com/docs>
- [35] Microsoft Corporation, "Kinect for Windows SDK Programming Guide," 2014. [Online]. Available: <https://developer.microsoft.com/en-us/windows/kinect/>

- [36] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, San Francisco, CA, USA, 2000, vol. 2, pp. 995-1001.
- [37] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566-580, Aug. 1996.
- [38] S. Izadi, D. Kim, O. Hilliges, D. Molyneaux, R. Newcombe, P. Kohli, J. Shotton, S. Hodges, D. Freeman, A. Davison, and A. Fitzgibbon, "KinectFusion: Real-time 3D reconstruction and interaction using a moving depth camera," in *Proc. 24th Annual ACM Symposium on User Interface Software and Technology*, Santa Barbara, CA, USA, 2011, pp. 559-568.
- [39] T. Whelan, M. Kaess, M. Fallon, H. Johannsson, J. Leonard, and J. McDonald, "Continuous: Spatially extended KinectFusion," in *Proc. RSS Workshop on RGB-D: Advanced Reasoning with Depth Cameras*, Sydney, Australia, 2012.
- [40] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. 10th IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, Basel, Switzerland, 2011, pp. 127-136.
- [41] A. Tateno, F. Tombari, and N. Navab, "Real-time and scalable incremental segmentation using dense RGB-D and depth maps," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1161-1172, Oct. 2015.
- [42] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 9, pp. 1627-1645, Sep. 2010.
- [43] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788.
- [44] C. R. Wren, A. Azarbayejani, T. Darrell, and A. P. Pentland, "Pfinder: Real-time tracking of the human body," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 19, no. 7, pp. 780-785, Jul. 1997.

- [45] J. Shotton, T. Sharp, A. Kipman, A. Fitzgibbon, M. Finocchio, A. Blake, M. Cook, and R. Moore, "Real-time human pose recognition in parts from single depth images," *Communications of the ACM*, vol. 56, no. 1, pp. 116-124, Jan. 2013.
- [46] M. R. Andersen, T. Jensen, P. Lisouski, A. K. Mortensen, M. K. Hansen, T. Gregersen, and P. Ahrendt, "Kinect depth sensor evaluation for computer vision applications," *Aarhus University, Department of Engineering, Technical Report*, 2012.
- [47] S. O. Escolano, C. Rhemann, S. Fanello, W. Chang, A. Kowdle, Y. Degtyarev, D. Kim, P. L. Davidson, S. Khamis, M. Dou, and V. Tankovich, "Holoportation: Virtual 3D teleportation in real-time," in *Proc. 29th Annual Symposium on User Interface Software and Technology*, Tokyo, Japan, 2016, pp. 741-754.
- [48] D. H. Ballard, "Generalizing the Hough transform to detect arbitrary shapes," *Pattern Recognition*, vol. 13, no. 2, pp. 111-122, 1981.
- [49] M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381-395, Jun. 1981.
- [50] S. chlorojin, H. Kim, and J. Kim, "Obstacle avoidance using a single RGB-D sensor for mobile robots in indoor environments," *Journal of Institute of Control, Robotics and Systems*, vol. 22, no. 8, pp. 631-637, Aug. 2016.
- [51] Y. Zhang, L. Chen, and X. Wang, "A survey of 3D point cloud processing algorithms," *IEEE Access*, vol. 8, pp. 215123-215142, 2020.
- [52] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 652-660.
- [53] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097-1105, 2012.
- [54] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770-778.

- [55] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 3431-3440.
- [56] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580-587.
- [57] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137-1149, Jun. 2017.
- [58] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conference on Computer Vision (ECCV)*, Amsterdam, Netherlands, 2016, pp. 21-37.